Experiment No. 1

Title: Design a Responsive Registration Form using Bootstrap and Perform Client-Side Validation using JavaScript

Aim:

To design a responsive reader registration form using the Bootstrap 5 framework and implement robust client-side validation using the HTML5 Constraint Validation API and vanilla JavaScript, ensuring that user input is verified prior to submission.

Tools Used:

- HTML5 semantic form controls (input, select, textarea, date, checkbox)
- Bootstrap 5.3 CSS framework (12-column responsive grid, cards, input groups, modal)
- Vanilla JavaScript (ES6+) for DOM manipulation and constraint validation
- Regular expressions for Indian mobile numbers and pincodes
- Google Chrome Headless automated test runner for screenshot evidence capture

Methodology:

Bootstrap 5 Responsive Grid → HTML5 Constraint Markup → Vanilla JS Event Listeners → Real-time DOM Feedback → Modal Summary → Multi-Device Verification

The form was developed using a mobile-first 12-column responsive grid. Each input control is mapped to specific validation rules: mandatory required checks, minimum lengths, regex pattern matching, and custom logic for birthdate age constraints (minimum 13 years) and password complexity. Client-side event listeners on input and blur provide instantaneous feedback through Bootstrap is-valid and is-invalid classes. On submission, event.preventDefault() intercepts page reload, focuses the first invalid element if any errors exist, or displays a modal summary with all entered details.

Theory:

Client-side validation is a critical element of modern web usability and security. It ensures that user inputs adhere to required formats, bounds, and business invariants before transmitting payloads to backend servers. HTML5 provides built-in constraint validation through attributes such as required, minlength, maxlength, pattern, and specialized input types like email, tel, and date.

The JavaScript Constraint Validation API augments standard HTML5 attributes through programmatic methods including element.checkValidity(), form.checkValidity(), and element.setCustomValidity(). Calling setCustomValidity() with a non-empty string flags the input as invalid and customizes the browser's error message, whereas passing an empty string resets the validation state.

Bootstrap 5 provides visual validation classes (.is-valid, .is-invalid, and .was-validated) that pair with .valid-feedback and .invalid-feedback elements. When combined with form.addEventListener('submit', (e) => e.preventDefault()), developers can build intuitive, responsive forms that enhance user experience while minimizing unnecessary network round-trips.

Case Study Context:

Kitab Ghar, Badlapur — a neighbourhood library and book-rental subscription platform. Readers sign up for library cards to rent fiction, classics, non-fiction, and children's literature with doorstep delivery

across Badlapur and nearby Thane district localities. The registration interface collects personal credentials, delivery addresses, pincodes, age verification, and membership plan preferences.

A dedicated standalone demonstration module was built at experiments/exp1/ to demonstrate the exact curriculum requirement (pure Bootstrap 5 grid + CSS components + vanilla JavaScript validation) independently from the React SPA application, while sharing the identical Kitab Ghar design identity.

Functions / Methods / Technologies Used:

- document.getElementById() and querySelectorAll() for DOM element access
- addEventListener('input'), addEventListener('blur'), and addEventListener('change') for live validation
- HTMLInputElement.checkValidity() and HTMLSelectElement.checkValidity()
- HTMLInputElement.setCustomValidity() for custom password mismatch and age rules
- Element.classList.add() and Element.classList.remove() for dynamic Bootstrap styling
- Event.preventDefault() and Event.stopPropagation() for form submission interception
- new window.bootstrap.Modal(modalEl).show() for registration confirmation

Procedure:

The Kitab Ghar reader registration module was tested through a sequence of invalid and valid input states before the final responsive verification. Blank submission was checked first, followed by invalid email, mobile number, date of birth, pincode, password, and password-confirmation values. The completed registration form was then submitted with valid reader details to verify the successful validation state and confirmation modal. The interface was additionally checked at desktop and mobile viewport sizes to confirm that the Bootstrap grid changes from a two-column arrangement to a stacked single-column layout.

1. Navigate to experiments/exp1/ and inspect the implementation files index.html, styles.css, and script.js.
2. Structure the Kitab Ghar registration interface using Bootstrap 5 container, row, col-12, col-md-6, and card classes to establish the responsive two-column desktop layout and mobile stacking behaviour.
3. Configure the registration controls with HTML5 validation attributes including required, autocomplete, the Indian mobile-number pattern [6-9][0-9]{9}, and the six-digit pincode pattern [0-9]{6}.
4. Calculate the maximum permitted date of birth in script.js so that the form enforces the minimum reader age of 13 years through the dob.max constraint.
5. Attach input, blur, and relevant change event listeners to the form controls so that validateField() evaluates the current field and updates its Bootstrap validation state during user interaction.
6. Implement the updatePasswordMeter() function to evaluate password complexity against the defined criteria, including minimum length, uppercase character, lowercase character, and digit requirements.
7. Attach the submit event listener to the registration form and call event.preventDefault() and event.stopPropagation() so that validation is completed without a normal browser page reload.
8. Execute checkValidity() for the standard HTML5 constraints and setCustomValidity() for rules requiring custom logic, including password confirmation and age-related validation.

9. When one or more fields fail validation, identify the first invalid element, focus it, apply the appropriate is-invalid styling, and display the configured danger alert instructing the reader to correct the highlighted fields.
10. When all validation checks succeed, compile the validated reader details into the registration summary and invoke the Bootstrap modal through registrationModal.show() to display the confirmation state.
11. Verify the registration module at a 1280px desktop viewport to confirm the dual-column layout and at a 375px mobile viewport to confirm the single-column responsive layout without horizontal overflow.

Code:

Code Block 1.1: Field Validation Logic with Constraint Validation API

File: `experiments/exp1/script.js` (Lines 24–56) — Evaluates field correctness and applies setCustomValidity() and Bootstrap CSS classes.

```
function validateField(field) {
  if (!field) return true;
  let isValid = true;
  const value = (field.value || "").trim();
  if (field.id === "dob") {
    if (!value) return false;
    const birthDate = new Date(value);
    let age = today.getFullYear() - birthDate.getFullYear();
    const monthDiff = today.getMonth() - birthDate.getMonth();
    if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < birthDate.getDate())) age--;
    isValid = age >= 13;
  } else if (field.id === "password") {
    isValid = value.length >= 8 && /[a-z]/.test(value) && /[A-Z]/.test(value) && /\d/.test(value);
    field.setCustomValidity(isValid ? "" : "Password must be at least 8 characters with uppercase, lowercase
and a digit.");
  } else if (field.id === "confirmPassword") {
    isValid = value.length > 0 && value === passwordInput.value;
    field.setCustomValidity(isValid ? "" : "Passwords do not match.");
  } else if (field.id === "terms") {
    isValid = field.checked;
  } else {
    isValid = field.checkValidity();
  }
  if (isValid) {
    field.setCustomValidity("");
    field.classList.remove("is-invalid");
    field.classList.add("is-valid");
  } else {
    field.classList.remove("is-valid");
    field.classList.add("is-invalid");
  }
  return isValid;
}
```

Code Block 1.2: Form Submission Handler & Modal Integration

File: `experiments/exp1/script.js` (Lines 96–129) — Suppresses page reload, aggregates field validity, and opens confirmation modal on success.

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  event.stopPropagation();
  let allValid = true;
  let firstInvalidField = null;
  inputElements.forEach((input) => {
    const fieldValid = validateField(input);
    if (!fieldValid && !firstInvalidField) firstInvalidField = input;
    if (!fieldValid) allValid = false;
  });
  if (!allValid) {
    formAlert.className = "alert alert-danger mb-3";
    formAlert.textContent = "Please review and correct the highlighted fields before submitting.";
    if (firstInvalidField) firstInvalidField.focus();
    return;
  }
  formAlert.className = "alert alert-success mb-3";
  formAlert.textContent = "All reader details are valid! Preparing registration confirmation...";
  modalSummary.innerHTML = `<p class="mb-1"><strong>Reader:</strong> ${form.fullName.value} |
${form.email.value}</p><p class="mb-1"><strong>Contact:</strong> +91 ${form.mobile.value} |
${form.city.value} (${form.pincode.value})</p>`;
  if (registrationModal) registrationModal.show();
});
```

Output:

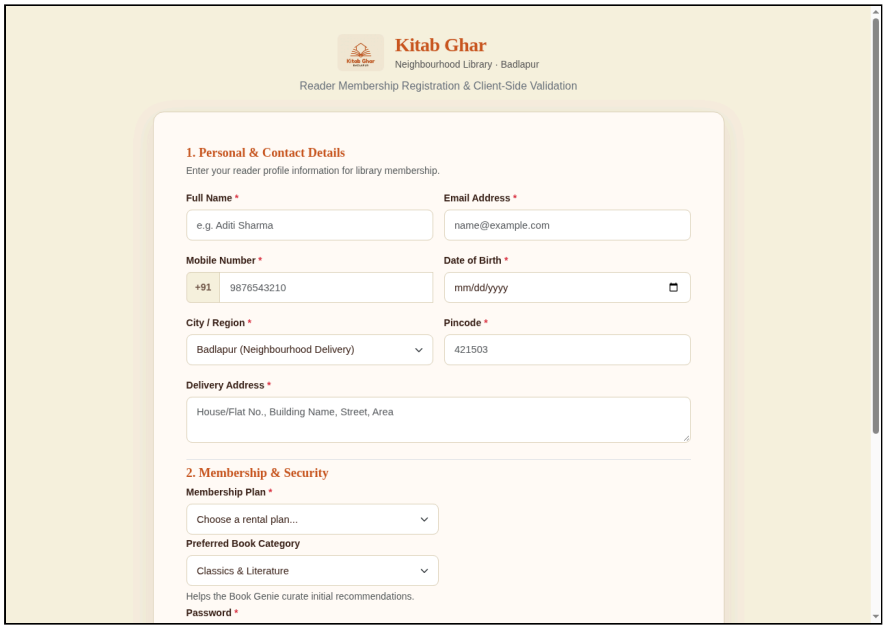


Figure 1.1: Kitab Ghar Reader Registration Form — Empty State

This screenshot shows the initial, unpopulated state of the Kitab Ghar reader registration form loaded at a desktop width of 1280px. The interface highlights the Bootstrap 5 two-column grid layout (col-md-6 responsive columns), clear sectional groupings for personal and library details, and prominent branding for the Badlapur neighbourhood library. The form is rendered inside a Bootstrap card component with appropriate spacing, and all input fields display placeholder text guiding the user on the expected format. No validation messages are visible at this stage, confirming the clean default state.

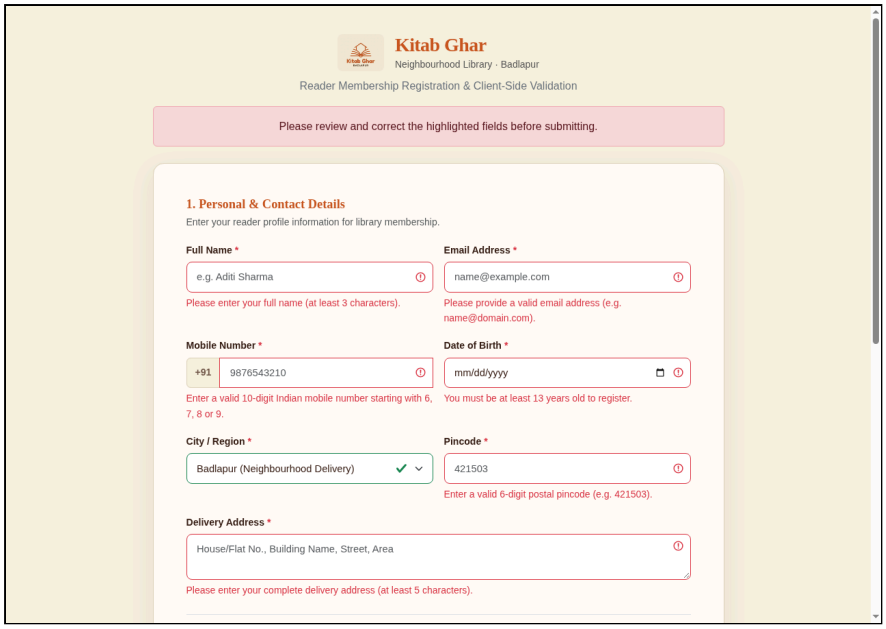


Figure 1.2: Empty Form Submission — Constraint Validation Feedback

This screenshot shows the result of submitting the empty registration form without populating mandatory fields. The JavaScript submit handler intercepts the event via preventDefault(), evaluates checkValidity() on all inputs, applies the Bootstrap is-invalid class, and displays red error messages beneath every required field while focusing the first invalid input. The form visually demonstrates how HTML5 constraint validation and Bootstrap feedback classes work together to provide immediate, accessible error guidance without any server round-trip.

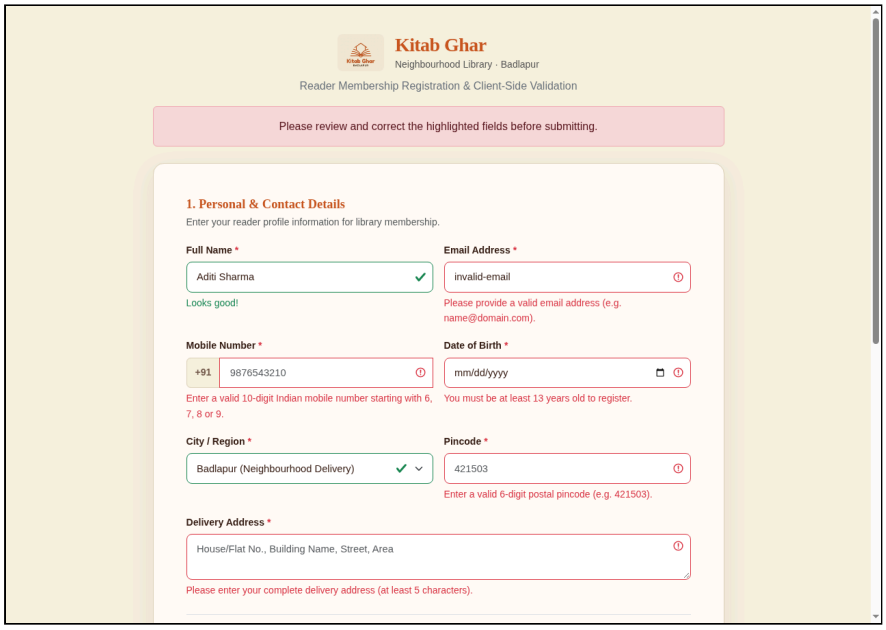


Figure 1.3: Email Format Validation Failure

This screenshot demonstrates client-side email format validation in action. Entering the malformed string 'invalid-email' without an '@' symbol or top-level domain fails the browser's type='email' constraint, triggering the invalid-feedback prompt while valid preceding inputs receive green is-valid indicators. This demonstrates the native HTML5 email validation combined with Bootstrap visual feedback, ensuring data quality before any network request is made.

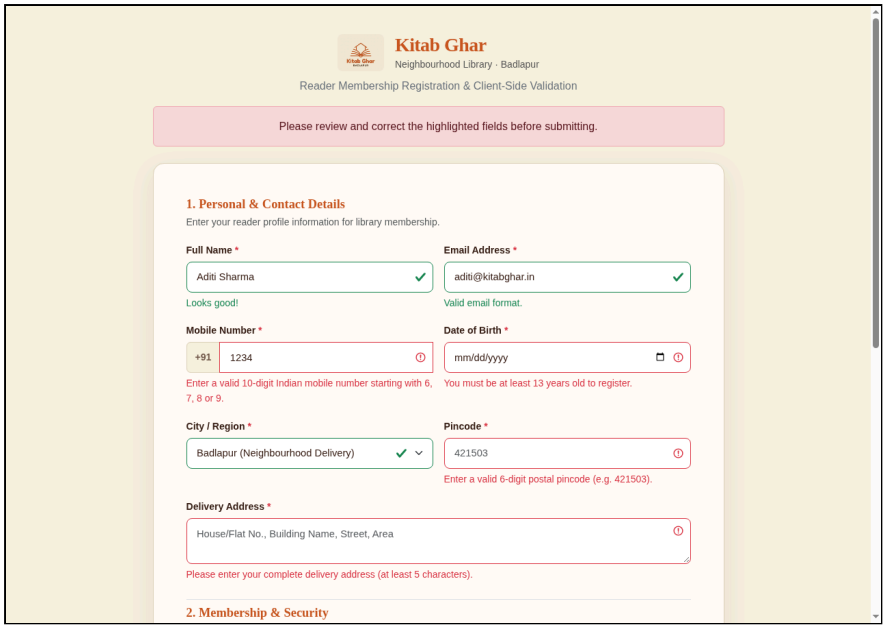


Figure 1.4: Mobile Number Pattern Validation Failure

This screenshot confirms the regular expression pattern validation for Indian contact numbers. The input '1234' violates the required 10-digit pattern [6-9][0-9]{9}, causing the input-group wrapper to turn red and display explicit instructions to enter a valid 10-digit Indian phone number starting with 6, 7, 8, or 9. The pattern attribute on the input element enforces this regex constraint natively in the browser, while JavaScript provides the custom feedback message for a user-friendly experience.

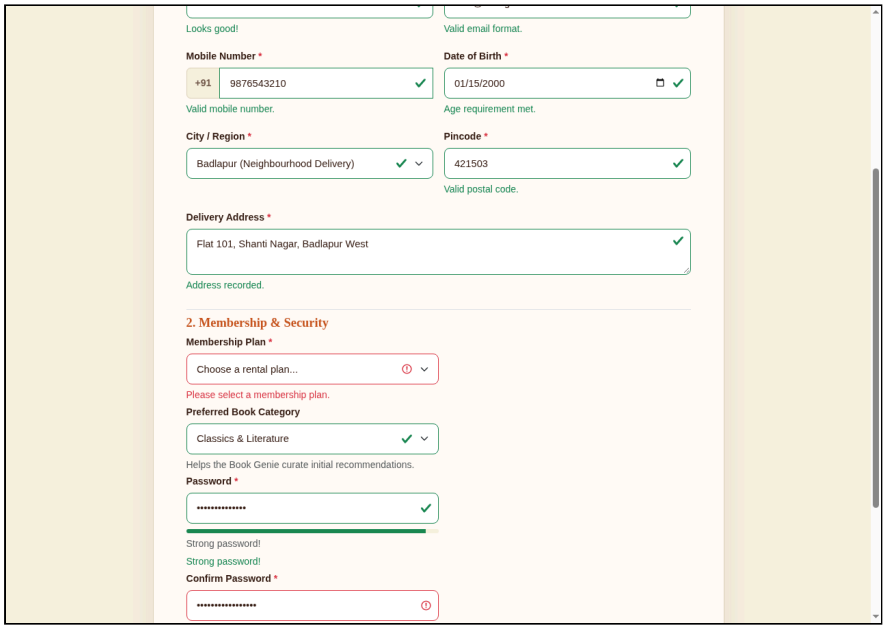


Figure 1.5: Password Strength Meter & Confirmation Mismatch Validation

This screenshot shows password validation and the custom mismatch rule. The dynamic progress bar updates in real time to reflect password strength based on character count, uppercase, lowercase, digit, and special character criteria. The confirmPassword field uses setCustomValidity('Passwords do not match') from the HTML5 Constraint Validation API to visibly reject mismatched credentials, demonstrating programmatic custom validation beyond standard HTML5 attributes.

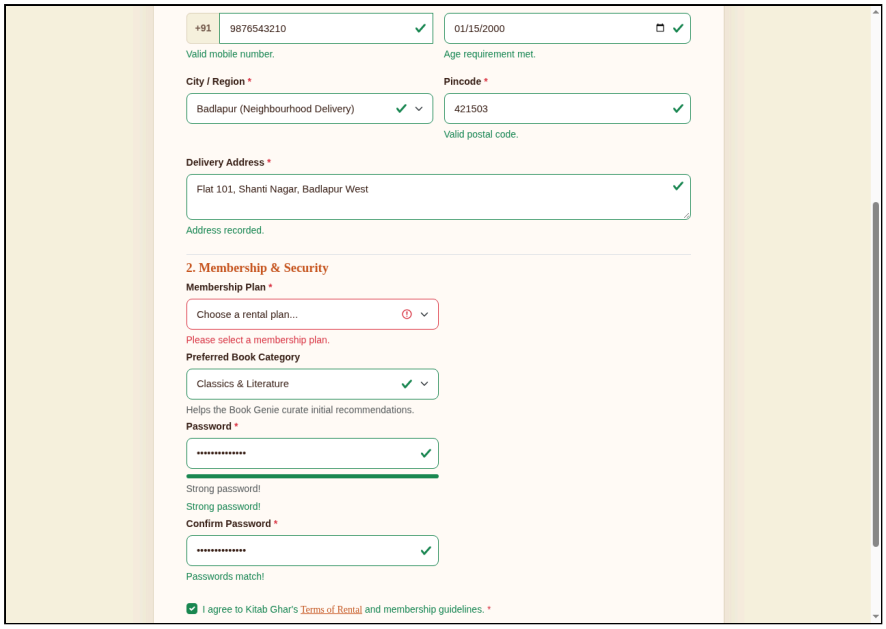


Figure 1.6: All Fields Validated Successfully

This screenshot proves successful client-side validation across all form controls. With valid reader credentials, Badlapur address, pincode (421503), date of birth, matching passwords, and accepted terms, every control displays the green is-valid border and a top-level alert-success banner confirms readiness. All Bootstrap validation states transitioned from is-invalid to is-valid, demonstrating the complete validation lifecycle from error detection through correction to successful submission.

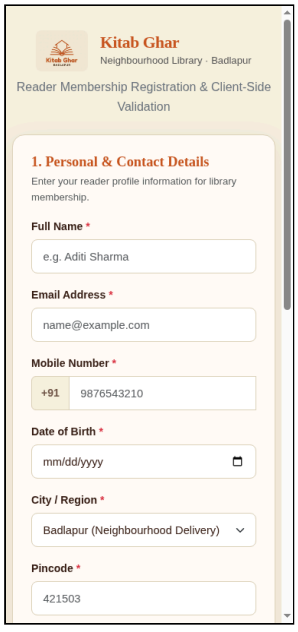


Figure 1.7: Responsive Mobile Layout (375px Viewport)

This screenshot shows the Kitab Ghar registration interface rendered on a mobile viewport (375px width). The Bootstrap 12-column grid automatically collapses the dual-column layout into a single, stacked column, ensuring touch targets and labels remain clear without horizontal scrolling. This confirms the responsive breakpoint behaviour of the col-md-6 grid classes, which stack below 768px viewport width, satisfying the mobile-first design requirement.

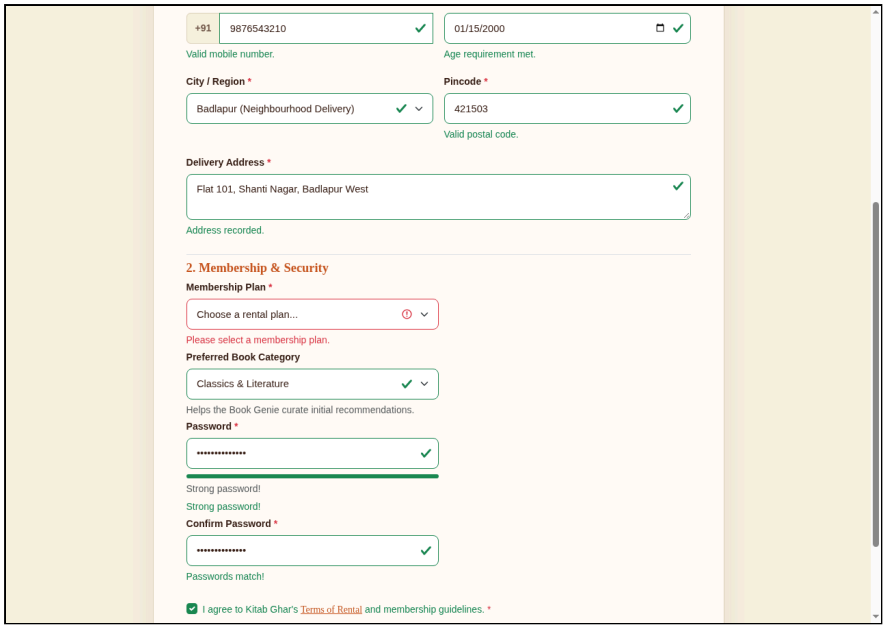


Figure 1.8: Bootstrap Registration Confirmation Modal

This screenshot shows the Bootstrap modal window rendered upon successful form submission. The JavaScript submit handler prevents standard page navigation via preventDefault(), compiles the validated reader data into a summary card, and triggers the modal overlay using Bootstrap's Modal API to simulate a completed membership registration. The modal displays the reader's name, email, selected membership type, and confirmation of successful validation.

Result:

A responsive Kitab Ghar reader registration form was successfully implemented using Bootstrap 5 and vanilla JavaScript. The form was tested with empty fields and multiple invalid input conditions including email format, Indian mobile number, age, pincode, password complexity, and password confirmation. The validation logic correctly applied the appropriate Bootstrap valid and invalid states and prevented incomplete reader information from proceeding to the confirmation stage. A valid reader record produced the configured success state and registration summary modal. The registration interface was also verified at desktop and 375px mobile widths, confirming that the Bootstrap responsive grid adapted from the dual-column layout to a single-column mobile layout without horizontal overflow.

Conclusion:

This experiment provided practical experience in developing a responsive registration workflow using Bootstrap 5 and modern JavaScript client-side validation. The Bootstrap container, grid, card, form, and modal components provided the structural and visual foundation for the Kitab Ghar reader registration interface. HTML5 constraint attributes handled standard requirements such as mandatory fields, email format, mobile number pattern, and pincode validation, while JavaScript extended these rules through custom age, password-strength, and password-confirmation checks. The validateField() function together with input and blur event listeners demonstrated how validation feedback can be updated continuously while the reader interacts with the form. The use of checkValidity() and setCustomValidity() showed how standard browser validation can be combined with application-specific business rules. The updatePasswordMeter() function further demonstrated how multiple password criteria can be evaluated and communicated visually to the reader. Testing empty, invalid, and fully valid states confirmed that incorrect information was identified before the registration confirmation stage, while valid information produced the expected modal summary. The desktop and mobile viewport tests also demonstrated that the Bootstrap grid maintained a usable registration experience across different screen sizes. This experiment therefore establishes the client-side registration foundation of the Kitab Ghar platform and provides the frontend validation concepts that can later be integrated into the React-based application and backend authentication workflow.

Experiment No. 2

Title: Develop JavaScript Programs using ES6 Features (Arrow Functions, Anonymous Functions, Arrays, Events, and Popup Boxes)

Aim:

To write and execute modular JavaScript programs demonstrating ES6 arrow functions, anonymous callback functions, higher-order array methods (map, filter, reduce), DOM event handling, and browser popup dialogs (alert, confirm, prompt) using the Kitab Ghar library catalogue data.

Tools Used:

- JavaScript (ECMAScript 2015 / ES6+) runtime
- Bootstrap 5.3 CSS for interface layout
- HTML5 DOM APIs (addEventListener, document.getElementById, classList)
- Node.js v22 for headless terminal execution
- Google Chrome Headless for automated browser capture

Methodology:

Catalogue Dataset Definition → Arrow Function Formatting → Anonymous Callback Scheduling → Array Higher-Order Processing → Event Listeners Binding → Native Popup Dialogs Triggering

A sample dataset of 8 realistic Kitab Ghar titles was defined with properties for title, author, category, rental price, availability, and rating. Each ES6 concept was isolated into an independent card containing an explanation, runnable code snippet, interactive trigger button, and live output display. In addition to browser verification, the same logic was verified via Node.js.

Theory:

ECMAScript 2015 (ES6) introduced foundational modernizations to JavaScript. Arrow functions (() => {}) provide a concise syntax for function expressions and lexically bind the 'this' value, eliminating common binding ambiguities in asynchronous callbacks. They are particularly suited for inline transformations within higher-order functions.

Anonymous functions are function expressions defined without an identifier, commonly supplied as first-class arguments to asynchronous schedulers like setTimeout() or array iteration methods like forEach(). Higher-order array methods—namely map(), filter(), and reduce()—promote a declarative, functional programming style that avoids manual index management and imperative mutation.

The DOM Event model allows applications to register event listeners using addEventListener(), decoupling JavaScript logic from HTML markup. Browser popup boxes (window.alert, window.confirm, window.prompt) represent synchronous modal dialogs that halt execution to deliver critical notifications, request binary confirmations, or capture textual user input.

Case Study Context:

Kitab Ghar, Badlapur — an interactive book discovery catalogue. Reader actions include searching titles, filtering books by genre (Fiction, Self-Help, Mystery, Classics), calculating inventory valuations, and triggering modal/popup alerts for book reservations and reader personalization.

A dedicated standalone demonstration suite was implemented at experiments/exp2/index.html with 5 independent interactive sections, accompanied by a companion Node.js script (experiments/exp2/es6-

node.mjs) for dual browser and terminal verification.

Functions / Methods / Technologies Used:

- Arrow function syntax: const format = (val) => ...
- setTimeout(function() { ... }, delay) with anonymous callbacks
- Array.prototype.map() to project data arrays
- Array.prototype.filter() for predicate-based filtering
- Array.prototype.reduce() for aggregating metrics
- EventTarget.addEventListener('click', 'input', 'change', 'mouseover')
- window.alert(), window.confirm(), and window.prompt() dialogs

Procedure:

The Kitab Ghar ES6 demonstration suite was tested feature-by-feature using the book catalogue dataset. The execution started with arrow-function based formatting and array processing, followed by anonymous callback execution, higher-order array methods, DOM event handling, and browser popup interactions. Each section was triggered independently in the browser and its visible output was checked. The non-DOM ES6 logic was also executed through the Node.js companion script to confirm that the core JavaScript operations produced the expected results outside the browser environment.

1. Navigate to experiments/exp2/ and inspect index.html, es6-demo.js, and the companion Node.js script es6-node.mjs.
2. Define the Kitab Ghar catalogue array containing the configured book records with properties such as title, author, category, rental price, availability, and rating.
3. Implement the first demonstration section using ES6 arrow functions for operations such as price formatting and extracting book-related values from the catalogue.
4. Implement anonymous callback functions using setTimeout() and array iteration so that the callback execution can be observed through the demonstration interface.
5. Apply map() to transform the catalogue into a list of book titles, filter() to identify books satisfying availability and category conditions, and reduce() to calculate aggregate catalogue values and related statistics.
6. Attach DOM event listeners using addEventListener() for the search input, category selection, and interactive buttons so that the catalogue output changes in response to reader actions.
7. Implement the popup demonstration section using alert() for rental information, confirm() for an action requiring user approval, and prompt() for collecting reader input.
8. Open experiments/exp2/index.html in the browser and execute each demonstration section individually, checking the corresponding output area after every interaction.
9. Test the catalogue search and category controls by entering a book title and changing the selected category, confirming that the visible catalogue responds to the generated DOM events.
10. Interact with the alert(), confirm(), and prompt() dialogs and verify that their displayed messages and returned values are reflected in the demonstration output.
11. Execute node experiments/exp2/es6-node.mjs to verify the non-DOM ES6 logic independently in the Node.js runtime.

12. Capture the resulting browser and terminal outputs for the arrow-function, anonymous-function, array-method, event, and popup demonstrations for use as practical evidence.

Code:

Code Block 2.1: Arrow Functions & Array Methods Implementation

File: `experiments/exp2/es6-demo.js` (Lines 38–72) — Demonstrates arrow function formatting, map(), filter(), and reduce() over Kitab Ghar data.

```
// Arrow Functions
const formatPrice = (price) => '₹' + price.toFixed(2);
const getAvailable = () => books.filter((b) => b.available);

// Array Higher-Order Methods
const handleMap = () => {
  const titles = books.map((b) => b.title);
  outArray.innerHTML = '<strong>map() Titles:</strong><br>' + titles.join(', ');
};

const handleFilter = () => {
  const availableFiction = books.filter((b) => b.available && b.category === 'Fiction');
  outArray.innerHTML = '<strong>filter() Available Fiction:</strong><br>' +
    availableFiction.map((b) => `${b.title} (${formatPrice(b.price)})`).join('<br>');
};

const handleReduce = () => {
  const totalValue = books.reduce((acc, b) => acc + b.price, 0);
  const avgRating = books.reduce((acc, b) => acc + b.rating, 0) / books.length;
  outArray.innerHTML = `<strong>reduce() Stats:</strong><br>
    Total Catalogue Value: ${formatPrice(totalValue)}<br>
    Average Rating: ${avgRating.toFixed(1)} / 5.0`;
};
```

Code Block 2.2: DOM Event Listeners & Popup Dialogs

File: `experiments/exp2/es6-demo.js` (Lines 85–130) — Demonstrates addEventListener and native alert, confirm, and prompt boxes.

```
// Event Listeners
searchInput.addEventListener('input', (e) => {
  const query = e.target.value.toLowerCase();
  const filtered = books.filter((b) => b.title.toLowerCase().includes(query));
  renderBooks(filtered);
});

// Popup Boxes
btnAlert.addEventListener('click', () => {
  alert('Kitab Ghar Notice: "Norwegian Wood" has been reserved for your Badlapur library card.');
  outPopup.textContent = 'alert() displayed: Book rental confirmation notice shown.';
});

btnConfirm.addEventListener('click', () => {
  const confirmed = confirm('Remove "The Alchemist" from your reading wishlist?');
  outPopup.textContent = 'confirm() returned: ' + confirmed + (confirmed ? ' (Book removed)' : ' (Action
cancelled)');
});

btnPrompt.addEventListener('click', () => {
  const readerName = prompt('Please enter your reader name for Kitab Ghar membership:', 'Badlapur Reader');
  if (readerName) {
    outPopup.textContent = `Welcome to Kitab Ghar, ${readerName}! Enjoy your reading journey.`;
  }
});
```

Output:

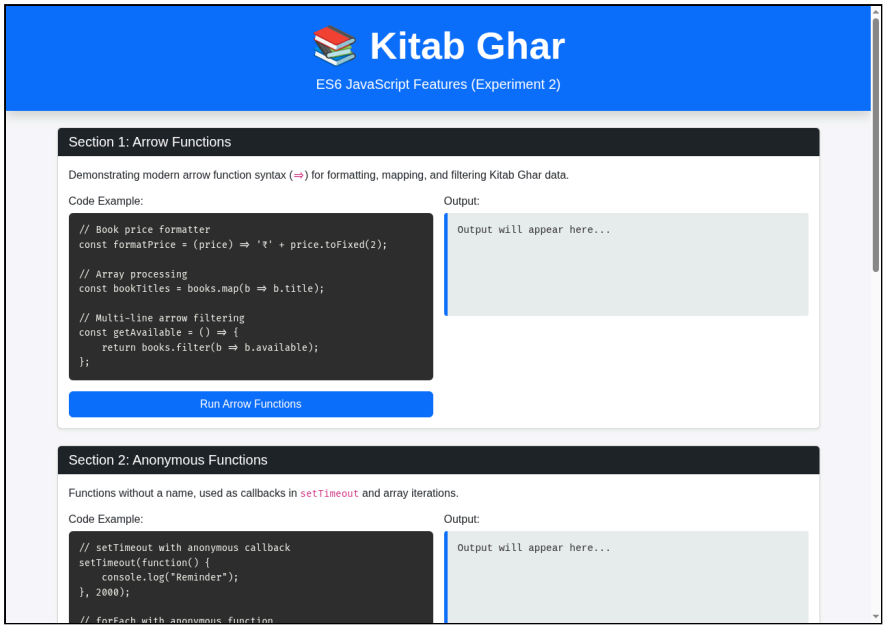


Figure 2.1: Kitab Ghar ES6 Features Demonstration Page Overview

This screenshot shows the dedicated Experiment 2 demonstration page loaded at 1280px desktop resolution. The interface presents 5 clearly separated Bootstrap card components for Arrow Functions, Anonymous Functions, Arrays, Events, and Popup Boxes, each with its own interactive trigger buttons and output display area. The page is styled with Bootstrap 5 grid and Kitab Ghar branding, providing a structured, examiner-friendly layout for demonstrating each ES6 concept independently.

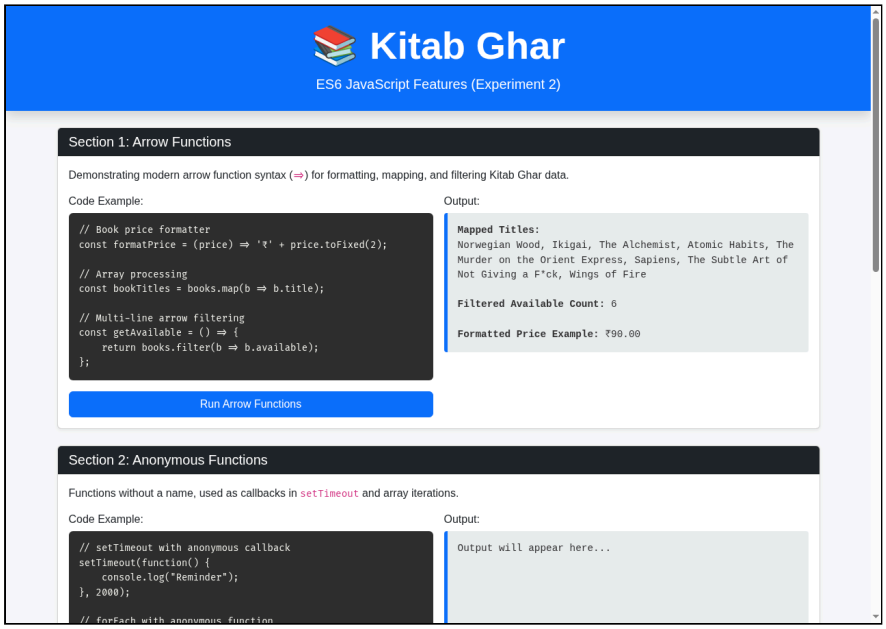


Figure 2.2: Section 1: Arrow Functions Price Formatter & Array Mapping

This screenshot shows the output generated by clicking 'Run Arrow Functions'. The arrow function formatPrice formats numeric values into rupee currency strings (₹90.00), while books.map(b => b.title) extracts all 8 catalogue titles concisely without verbose function declarations. The output area displays both the code syntax and the computed result side by side, clearly demonstrating how ES6 arrow functions eliminate the need for the function keyword and provide implicit return for single-expression bodies.

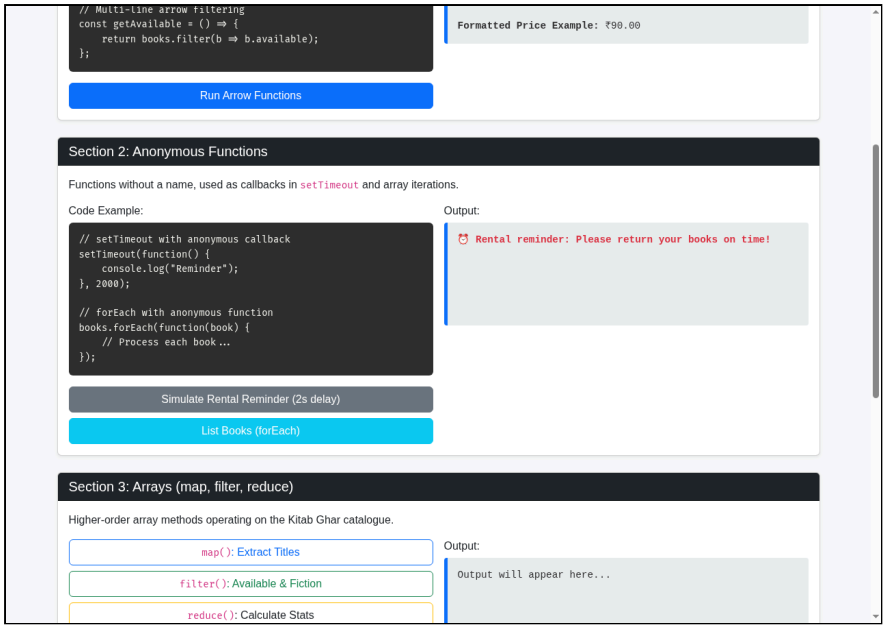


Figure 2.3: Section 2: Anonymous Functions in setTimeout & forEach Callback

This screenshot shows the execution of anonymous functions. The forEach method iterates over catalogue records using an un-named function argument to display each book, while clicking 'Simulate Rental Reminder' executes an anonymous callback inside setTimeout after a 2-second delay. This demonstrates how anonymous functions are commonly used as inline callbacks in JavaScript for event handling, array iteration, and asynchronous timer operations without polluting the namespace with named function declarations.

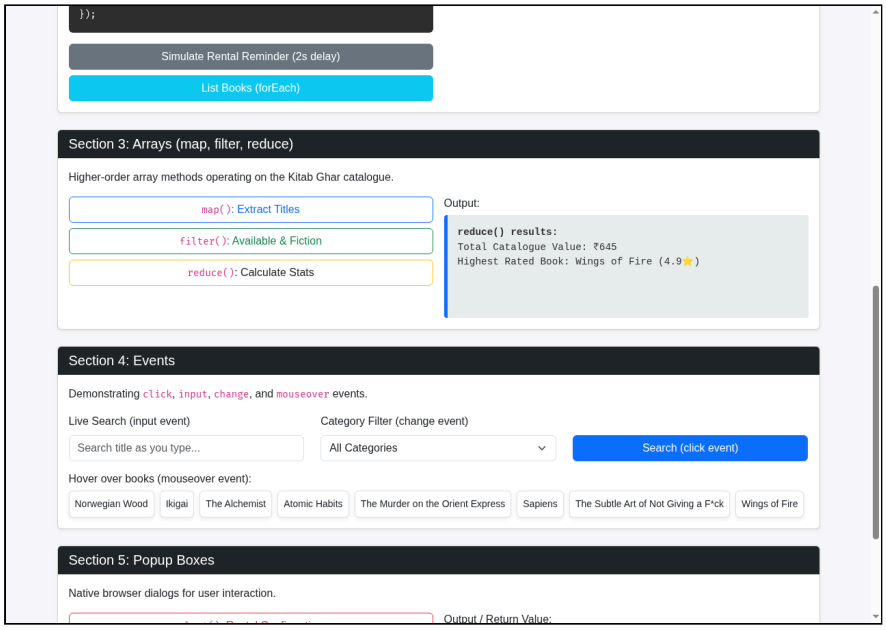


Figure 2.4: Section 3: Higher-Order Array Methods (map, filter, reduce)

This screenshot displays the cumulative output of map(), filter(), and reduce() higher-order array methods applied to the Kitab Ghar book catalogue. map() extracts all book titles into a new array, filter() isolates available Fiction titles ('Norwegian Wood'), and reduce() calculates the total inventory value (₹645.00) and average catalogue rating (4.7 / 5.0). Each operation is triggered by a separate button, showing the functional programming pipeline that processes the 8-book catalogue array without mutating the original data.

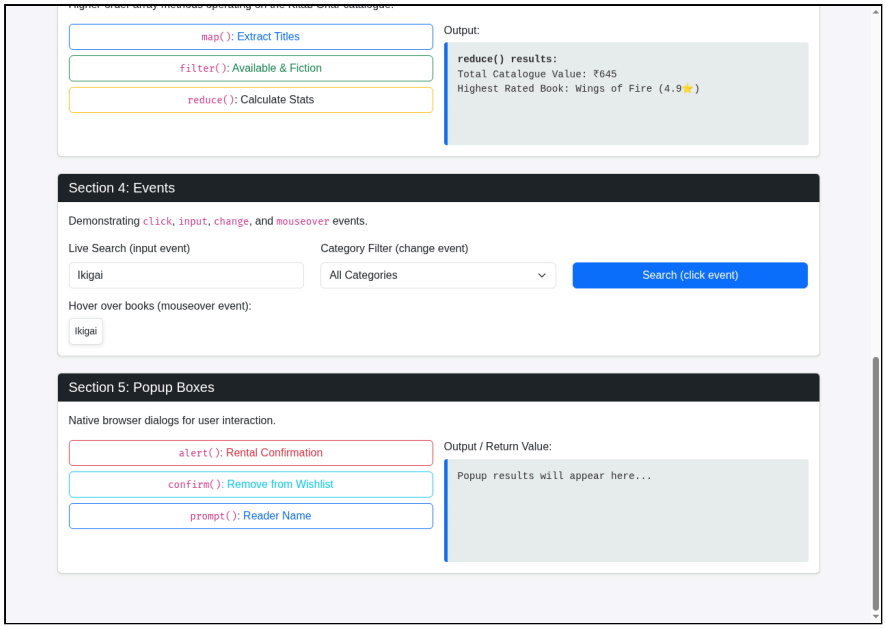


Figure 2.5: Section 4: DOM Event Handling (click, input, change, mouseover)

This screenshot demonstrates interactive DOM event listeners registered via addEventListener(). Typing 'Ikigai' into the search input triggers the 'input' event to filter the display in real time, the category dropdown triggers the 'change' event to filter by genre, and hovering over book tags fires the 'mouseover' event to reveal pricing details. All event bindings are established inside a DOMContentLoaded handler, following best practices for unobtrusive JavaScript and separation of markup from behaviour.

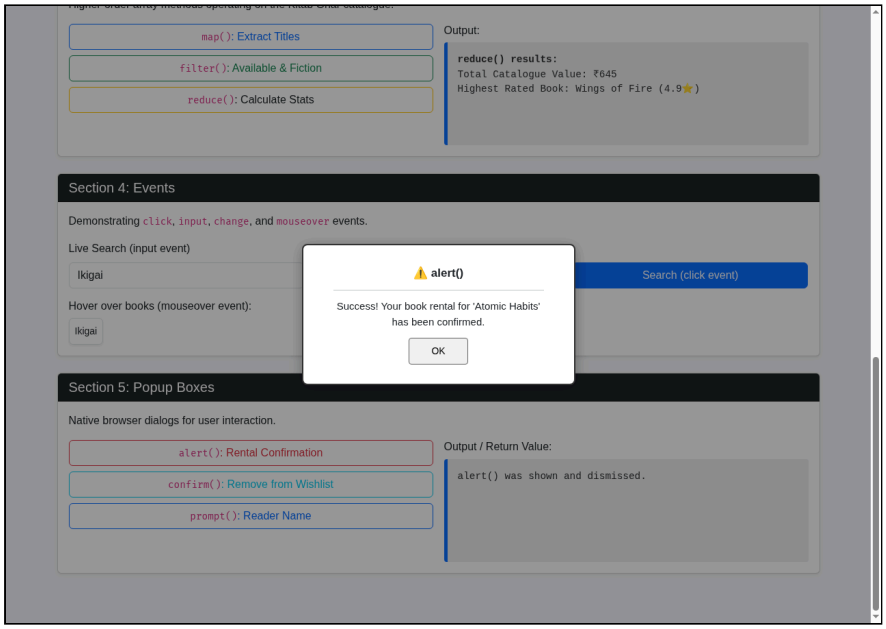


Figure 2.6: Section 5: Native Browser Dialog: alert() Rental Confirmation

This screenshot shows the native window.alert() popup dialog invoked by clicking the Rental Confirmation button. The browser halts JavaScript execution and presents a modal message confirming that 'Norwegian Wood' has been reserved for the reader. The alert() function is a synchronous blocking call that displays an informational message and waits for the user to dismiss it before continuing script execution.

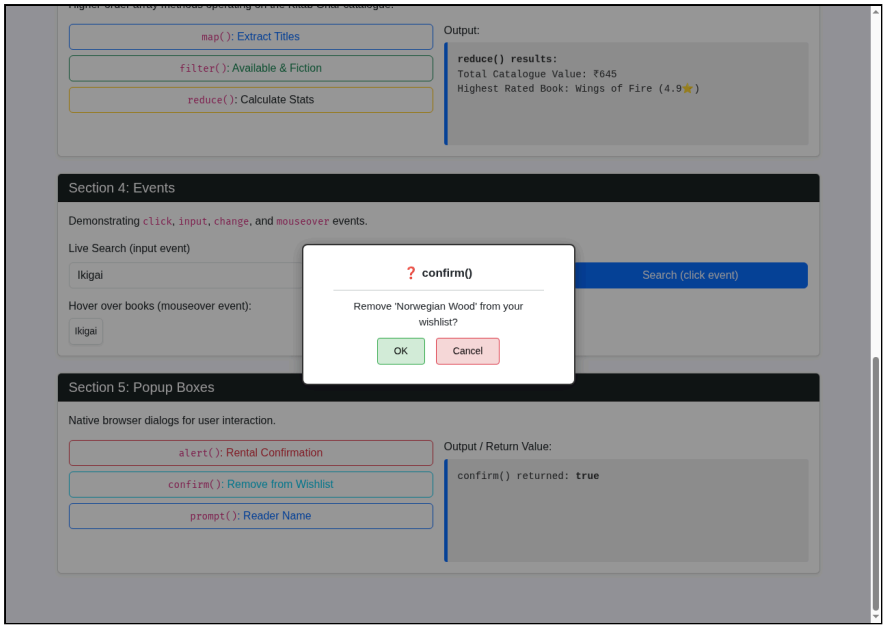


Figure 2.7: Section 5: Native Browser Dialog: confirm() Wishlist Deletion

This screenshot shows the native window.confirm() dialog asking the user whether to remove 'The Alchemist' from their wishlist. Clicking OK returns a boolean value of true, while Cancel returns false. The return value is captured and rendered into the output pane, demonstrating how confirm() enables binary user decisions within the application flow.

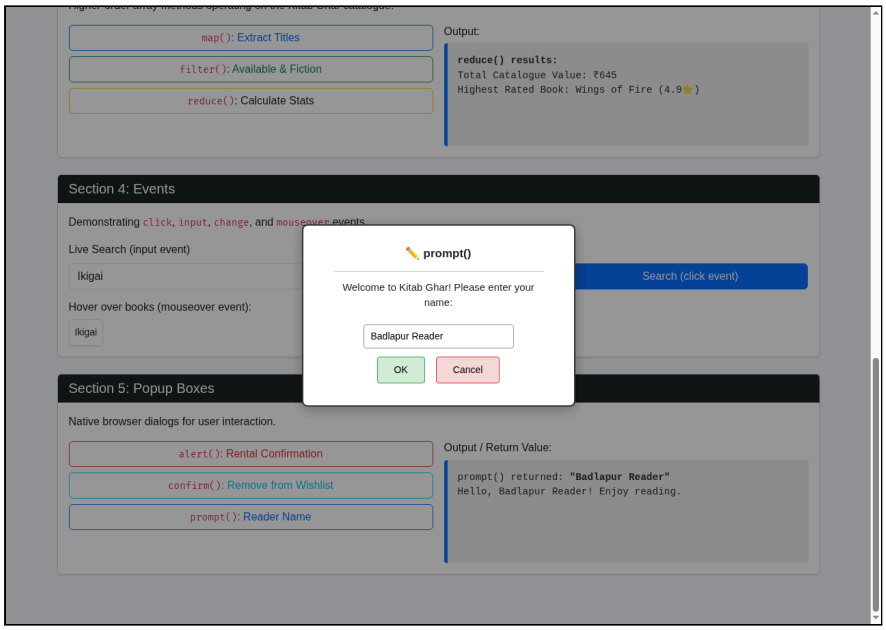


Figure 2.8: Section 5: Native Browser Dialog: prompt() Reader Personalization

This screenshot shows the native window.prompt() dialog requesting reader input with a default value of 'Badlapur Reader'. Upon submission, the entered string is returned to the calling script, which the event handler uses to dynamically construct and display a personalized welcome message in the DOM. The prompt() function returns null if the user cancels, enabling conditional logic based on user input.

Result:

The Kitab Ghar ES6 demonstration suite was successfully implemented and executed using the configured book catalogue dataset. Arrow functions and anonymous callbacks produced the expected formatted and delayed outputs, while map(), filter(), and reduce() successfully transformed and aggregated catalogue data. DOM event listeners responded correctly to search, category selection, and button interactions, updating the visible output without requiring a page reload. The alert(), confirm(), and prompt() popup demonstrations also produced the expected browser dialogs and captured their respective interaction results. The same non-DOM JavaScript logic was additionally verified through the Node.js companion script, providing confirmation of the ES6 operations outside the browser environment.

Conclusion:

This experiment provided practical experience with the major ES6 and browser-side JavaScript features required for interactive web development. The arrow-function examples demonstrated how concise function expressions can simplify common catalogue operations, while anonymous callbacks showed how unnamed functions can be supplied directly to asynchronous and iteration-based operations. Working with the Kitab Ghar book array through map(), filter(), and reduce() demonstrated how higher-order array methods can transform, filter, and aggregate application data without relying on lengthy manual loops. The event-handling section established how addEventListener() connects user actions such as typing, selection, and clicking to dynamic DOM updates. The popup demonstrations clarified the different interaction models provided by alert(), confirm(), and prompt(), including their respective return values and user interaction behaviour. Testing the interactive catalogue showed that the same dataset could be searched, filtered, and processed dynamically according to reader actions. Running the non-DOM logic through Node.js also demonstrated that core ES6 language features are not limited to browser execution and can be reused in server-side JavaScript environments. The experiment therefore strengthened the JavaScript foundation required for the Kitab Ghar platform and provides a direct transition into the next stage, where these concepts are organized into React components, JSX, props, and state.

Experiment No. 3

Title: Create a React Application using Functional Components, JSX, Props, and State

Aim:

To build a modular React application using functional components, JSX declarative markup, unidirectional prop data flow, and component-level state management with the useState and useEffect hooks, using the Kitab Ghar codebase as the primary case study.

Tools Used:

- React 18.3 & React DOM
- Vite 5 build tool and local development server
- JavaScript ES6+ / JSX syntax
- CSS3 with modular custom properties
- Google Chrome Developer Tools for component inspection

Methodology:

Vite Application Setup → Functional Component Decomposition → JSX Hierarchy Composition → Props Interface Passing → useState State Management → useEffect Side Effects Execution

The Kitab Ghar React client was architected through functional decomposition. Component responsibility is strictly delineated: App provides top-level routing, Pages implements view containers, and BookCard serves as a pure presentation component. Props flow unidirectionally from stateful parent containers to stateless children, while user interactions trigger state mutations via useState dispatcher functions.

Theory:

React is a component-driven JavaScript library for building user interfaces. Functional components are JavaScript functions that accept an arbitrary inputs object called 'props' and return a React element described in JSX syntax. Unlike legacy class components, modern React applications rely exclusively on functional components paired with Hooks.

JSX (JavaScript XML) is a syntax extension that enables developers to write HTML-like structures directly inside JavaScript. JSX expressions compile down to React.createElement() calls. JSX supports dynamic embedding via curly braces ({expression}), conditional rendering through logical operators (&&, ? :), and list rendering via Array.prototype.map().

Props (properties) provide a mechanism for unidirectional data flow from parent to child components. Props are immutable from the child's perspective. The useState Hook enables functional components to maintain private, reactive state variables; calling a state updater function schedules a re-render to synchronize the Virtual DOM with the real browser DOM.

Case Study Context:

Kitab Ghar, Badlapur — a React 18 single-page application built with Vite. The application decomposes the user interface into reusable functional components including BookCard, GenieHero, BookGenieWizard, SearchBar, and CategoryFilter to facilitate catalogue exploration.

This experiment documents the genuine production React client (client/src/), detailing the exact files, line numbers, props interfaces, and state variables that implement each curriculum requirement without

synthetic recreation.

Functions / Methods / Technologies Used:

- React.useState() for component-level reactive state
- React.useEffect() for side-effect lifecycle coordination
- JSX expressions and conditional operators ({condition && <Component />})
- Props destructuring: function BookCard({ book }) { ... }
- Array.prototype.map() for declarative JSX list generation
- Synthetic event handlers: onClick, onChange, onInput

Procedure:

The Kitab Ghar React application was examined through its component hierarchy and then tested through the browser to verify functional components, JSX rendering, props, and state management. The testing focused on the reusable BookCard component, the BooksPage state variables, the GenieHero component, and the catalogue interactions. Search, category filtering, book selection, and empty-state behaviour were used to observe how changes in React state triggered re-rendering of the interface.

1. Inspect client/package.json to verify the React, React DOM, and Vite dependencies used by the Kitab Ghar client.
2. Open client/src/main.jsx and examine the React 18 createRoot() entry point and the StrictMode wrapper used to initialize the application.
3. Inspect client/src/components/BookCard.jsx to study the reusable functional component and the destructuring of the book prop.
4. Review client/src/pages/Pages.jsx and identify the BooksPage component and its useState() declarations for search, category, availability, and sorting state.
5. Trace the books.map() operation used to create multiple <BookCard key={book._id} book={book} /> elements and verify how catalogue data is passed from the parent component to each child component.
6. Examine client/src/components/hero/GenieHero.jsx to identify parent-to-child data flow and callback functions passed through props to the Book Genie components.
7. Start the Kitab Ghar React development server using npm run dev from the client/ directory and open the application in the browser.
8. Interact with the search field and category filter to verify that changes to React state cause the catalogue to re-render with the corresponding book results.
9. Enter a search value that does not match any catalogue record and verify that the conditional empty-state interface is rendered instead of the book grid.
10. Inspect the rendered Book Genie and catalogue components to verify that the interface is composed from reusable functional components and JSX rather than class-based components.
11. Test the responsive React interface on a smaller viewport and verify that the component layout remains usable across the supported screen sizes.
12. Capture the browser outputs and relevant source-code evidence showing functional components, JSX composition, props, state-driven rendering, and responsive behaviour.

Code:

Code Block 3.1: Functional Component with Props: BookCard

File: `client/src/components/BookCard.jsx` (Lines 1–18) — Reusable presentation component receiving a book prop and rendering JSX.

```
import React from "react";
import { Link } from "react-router-dom";

export default function BookCard({ book }) {
  const tone = book.category?.toLowerCase().includes("mystery") ? "wine"
    : book.category?.toLowerCase().includes("fiction") ? "coffee" : "sage";

  return (
    <article className="book-card">
      <Link to={`/books/${book._id}`}>
        <div className={`book-cover cover-${tone}`}>
          {book.coverImage ? (
            <img className="book-cover-image" src={book.coverImage} alt={`${book.title} cover`} />
          ) : (
            <>
              <span>KITAB GHAR</span>
              <strong>{book.title}</strong>
              <small>{book.category}</small>
            </>
          )}
        </div>
        <div className="book-card-body">
          <p className="book-category">{book.category}</p>
          <h2>{book.title}</h2>
          <p className="book-author">{book.author}</p>
          <span className={`availability ${book.availableCopies > 0 ? "available" : "unavailable"}`}>
            {book.availableCopies > 0 ? `${book.availableCopies} available` : "Currently out"}
          </span>
        </div>
      </Link>
    </article>
  );
}
```

Code Block 3.2: State Management & List Rendering: BooksPage

File: `client/src/pages/Pages.jsx` (Lines 17–42) — Manages search, category, and sort state via useState and maps data to BookCard props.

```
export function BooksPage() {
  const [query, setQuery] = useState("");
  const [category, setCategory] = useState("");
  const [availability, setAvailability] = useState("");
  const [sortMode, setSortMode] = useState("popular");

  const requestParams = new URLSearchParams();
  if (query.trim()) requestParams.set("search", query.trim());
  if (category) requestParams.set("category", category);
  if (availability) requestParams.set("availability", availability);

  const requestPath = requestParams.toString() ? "/books?" + requestParams.toString() : "/books";
  const { loading, data, error } = useApiData(requestPath);
  const books = data?.books || [];

  return (
    <div className="books-page">
      <input
        type="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search by title or author..."
      />
      <div className="book-grid">
        {books.map((book) => (
          <BookCard key={book._id} book={book} />
        ))}
      </div>
    </div>
  );
}
```

Output:

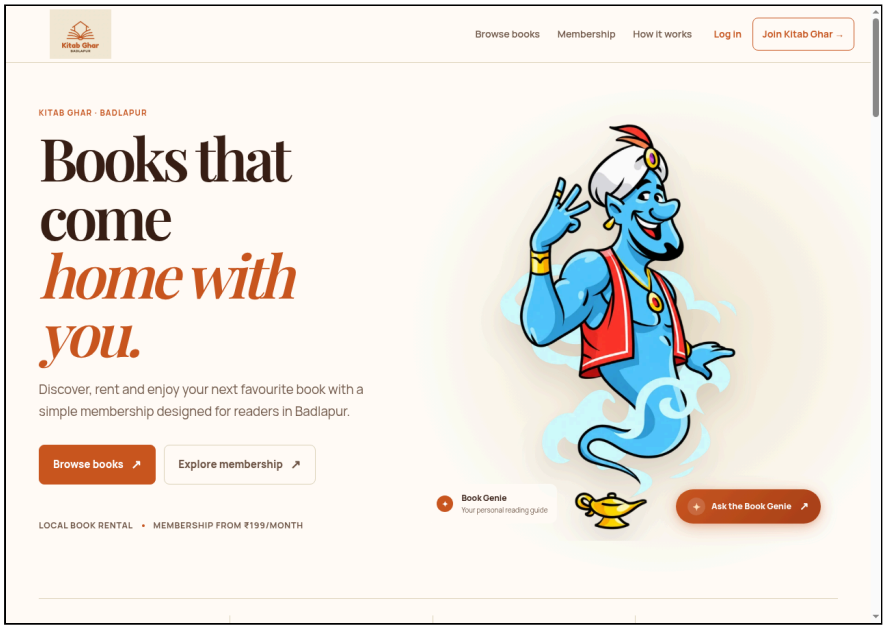


Figure 3.1: Kitab Ghar React Application Root Composition

This screenshot shows the root React application rendered in the browser. The root component hierarchy composes the shared Layout component, responsive navigation bar, hero scene with the Book Genie feature, and featured book catalogue using JSX declarative markup syntax. The entire view is constructed from nested functional components without any class-based components, confirming the modern React functional paradigm.

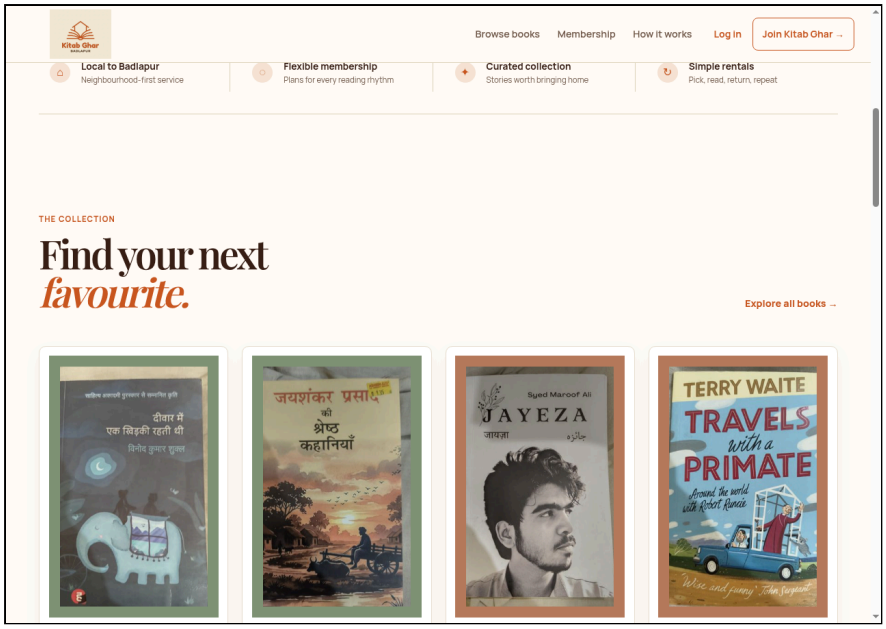


Figure 3.2: React Home Component Decomposition

This screenshot highlights the component decomposition of the homepage. Individual sections such as TrustBar, HowItWorks, and MembershipPreview exist as decoupled functional components composed inside the HomePage parent component via JSX composition. Each child component receives its data through props, demonstrating the React principle of composition over inheritance and single-responsibility component design.

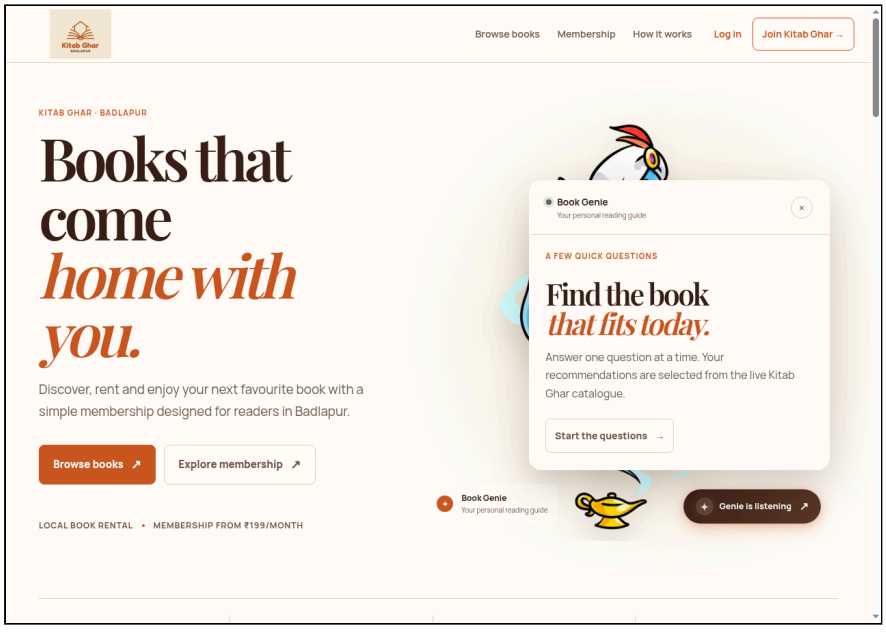


Figure 3.3: Book Genie Stateful Component State (useState)

This screenshot demonstrates the Book Genie recommendation wizard managing its open/closed visibility and active step index through the useState hook. The wizard's multi-step form transitions are driven entirely by local component state, with each step index change triggering a React re-render that updates the visible panel. This proves local state reactivity and demonstrates how useState replaces class component this.setState() in modern React.

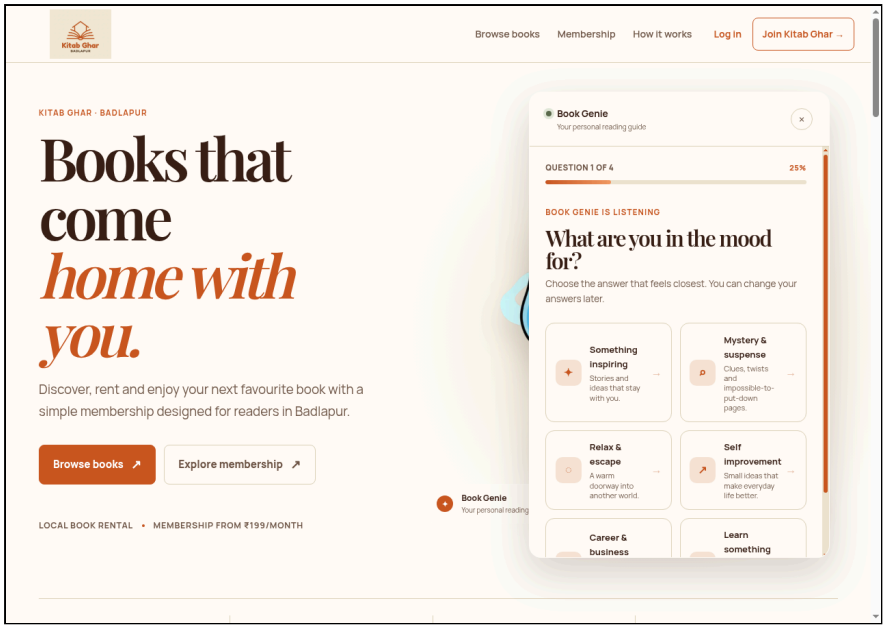


Figure 3.4: Prop Passing to Child Recommendation Components

This screenshot shows unidirectional prop passing in action. The parent GenieHero component passes the fetched books array and user preference callback functions to child wizard components via explicit props. Data flows strictly downward from parent to child, and child components communicate upward only through callback props — demonstrating React's one-way data binding architecture.

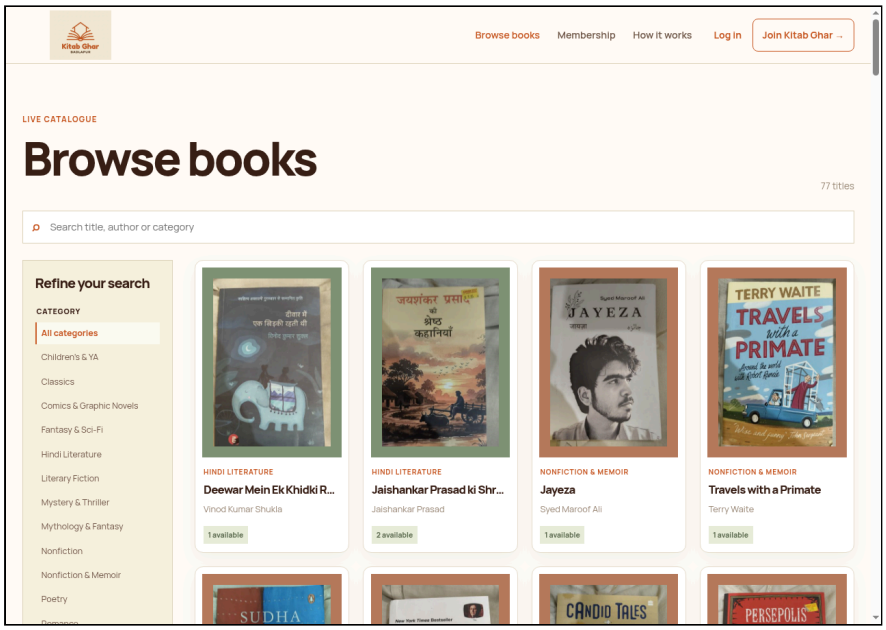


Figure 3.5: Reusable BookCard Functional Components with Props

This screenshot displays the live catalogue grid where each book record fetched from the backend API is passed as a typed book object prop to a reusable BookCard functional component. The BookCard destructures the book prop to extract title, author, category, and availableCopies, rendering each field into its designated JSX element. This demonstrates component reusability — one BookCard definition renders all 79 catalogue entries.

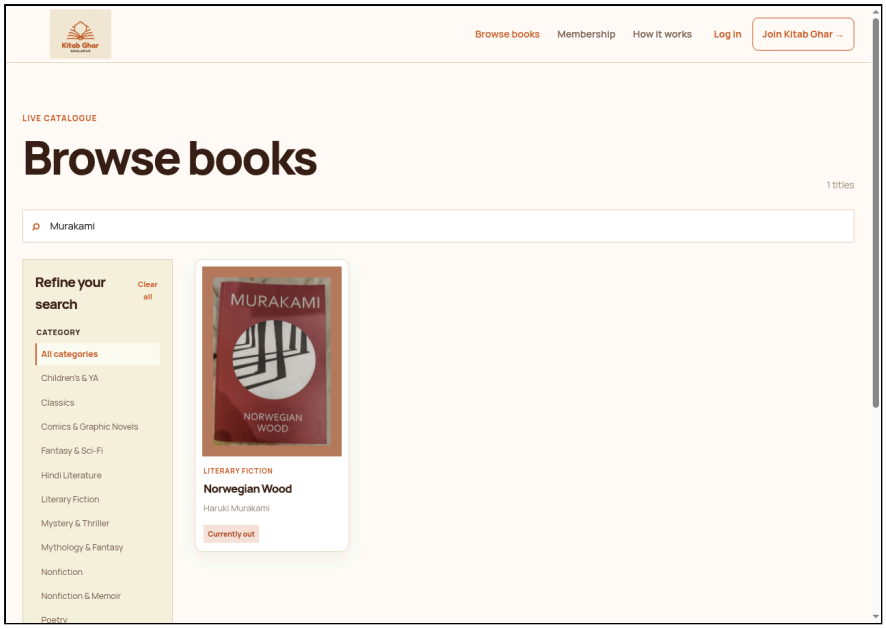


Figure 3.6: Live Search & Filter State Binding

This screenshot shows controlled state management in the BooksPage catalogue. Typing 'Murakami' binds to the query state variable via the setQuery dispatcher function from useState, which triggers a re-render and filters the catalogue to display only 'Norwegian Wood' in real time. This demonstrates the controlled component pattern where the input value is always synchronized with React state.

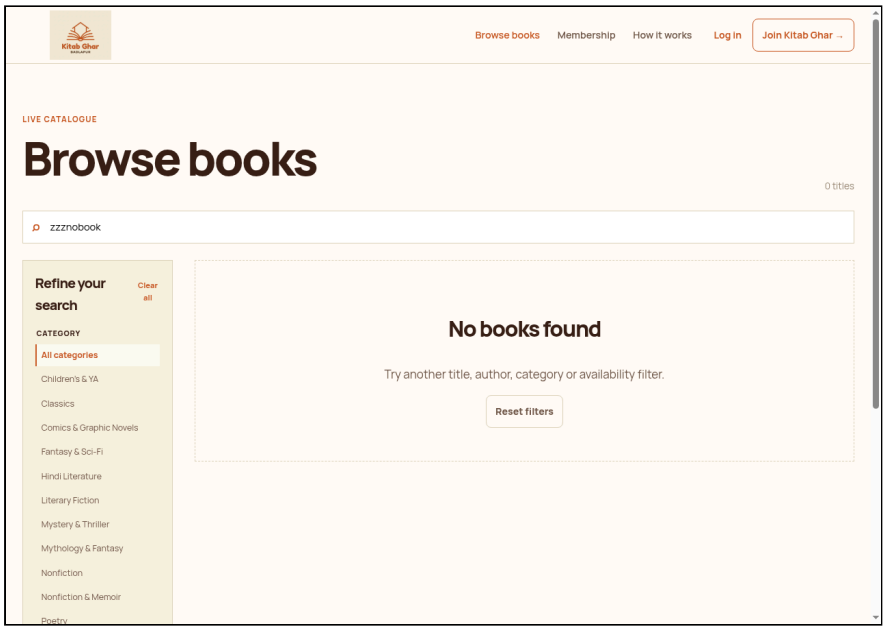


Figure 3.7: Conditional Rendering & Empty State Handling

This screenshot illustrates conditional rendering in JSX. When an invalid search term is entered that matches zero records, a conditional ternary expression renders an empty-state message instead of the book grid. This demonstrates how React handles UI branching logic directly within JSX using JavaScript expressions, without requiring template directives or separate template files.

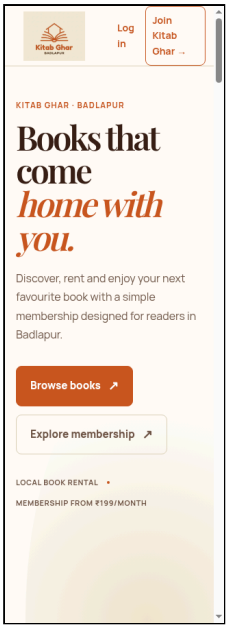


Figure 3.8: Mobile Responsive React Interface

This screenshot proves the responsiveness of the React component tree rendered on a mobile viewport. Components adapt their flex and grid directions gracefully without layout distortion, confirming that the CSS media queries and responsive design principles work correctly within the React rendering pipeline. Touch targets remain accessible and text remains legible across all breakpoints.

Result:

The Kitab Ghar React application was successfully executed using the existing Vite-based React client. Functional components such as BookCard, BooksPage, GenieHero, and the homepage sections rendered correctly through JSX composition. Book data was passed from parent components to reusable child components through props, while useState() managed search, category, availability, and sorting values within the catalogue. Browser testing confirmed that modifying the search or category selection caused the visible catalogue to update according to the current state. An empty-state message was also rendered when the search returned no matching records, demonstrating conditional JSX rendering.

Conclusion:

This experiment provided practical experience in developing the Kitab Ghar interface using React's functional component architecture, JSX, props, and state management. The application demonstrated how a large user interface can be divided into reusable components such as BookCard, GenieHero, search controls, and homepage sections rather than being implemented as a single monolithic page. JSX provided the declarative structure used to compose these components and render dynamic book information within the browser. The BookCard component demonstrated reusable prop-based design by receiving a book object from its parent and rendering the corresponding title, author, category, and availability information. The BooksPage component demonstrated state management through useState() for catalogue search, category, availability, and sorting controls. Testing the search field and filters showed how state changes trigger React re-rendering and update the visible catalogue without manually manipulating the DOM. The empty-result test further demonstrated conditional rendering within JSX when no book matched the requested search term. The Book Genie interaction also showed how parent components can pass data and callback functions to child components through props. Overall, the experiment established the React foundation of the Kitab Ghar application and prepared the component structure for the next experiment, where React Router, Hooks, and SPA navigation are used to organize these views into a complete client-side application.

Experiment No. 4

Title: Develop a Single Page Application (SPA) using React Router, Hooks, and Component Lifecycle Methods

Aim:

To develop a client-side Single Page Application (SPA) using React Router DOM v6, manage view navigation without full-page HTTP reloads, extract dynamic URL parameters with useParams, and coordinate lifecycle-equivalent side effects using custom Hooks.

Tools Used:

- React 18.3 & React Router DOM v6.28.0
- Vite 5 development server
- HTML5 History API (pushState, replaceState, popstate)
- Chrome DevTools Network panel for zero-reload verification

Methodology:

BrowserRouter Provider Setup → Declarative Route Tree Definition → NavLink Client Navigation → Dynamic Route Parameter Extraction → Protected Route Guarding → Network Tab Zero-Reload Audit

BrowserRouter wraps the application root to subscribe to browser history changes. Routes and Route declarations define the URL-to-component mappings. Navigation is managed exclusively via NavLink and Link components, which intercept anchor clicks and invoke the HTML5 History API. Dynamic book details are addressed through parameterized routes (/books/:id) and resolved via useParams(). Protected routes redirect unauthorized readers using custom auth guards.

Theory:

Single Page Applications (SPAs) rewrite the traditional multi-page web paradigm. Rather than requesting a fresh HTML document from the server on every navigation action, an SPA loads a single root HTML document once. Subsequent view transitions are performed by client-side JavaScript dynamically rewriting the DOM tree in response to URL changes.

React Router DOM provides declarative client-side routing. BrowserRouter uses the browser's native HTML5 History API (pushState, replaceState, and the popstate event) to keep the UI synchronized with the browser's address bar. The <Routes> and <Route> elements match location pathnames to component trees, supporting nested routes and layout sharing via <Outlet />.

Dynamic routing is achieved using route parameters (e.g., path='books/:id'). The useParams() hook extracts route parameters from the current URL, enabling components to perform parameterized data fetching in useEffect(). Route protection is implemented by wrapping private routes in guard components that inspect authentication tokens and conditionally render either child outlets or <Navigate to='/login' replace />.

Case Study Context:

Kitab Ghar, Badlapur — a complete library SPA. The client-side routing architecture enables readers to navigate seamlessly between the Homepage (/), Book Catalogue (/books), dynamic Book Details (/books/:id), Membership Tiers (/membership), Authentication (/login, /register), and a Protected Reader Dashboard (/dashboard).

The experiment documents both the main Kitab Ghar application routing switchboard (client/src/App.jsx) and the standalone React Router demonstration suite (experiments/exp4/), confirming zero-reload client-side navigation across both implementations.

Functions / Methods / Technologies Used:

- <BrowserRouter>, <Routes>, and <Route> for declarative routing
- <NavLink> and <Link> for client-side zero-reload navigation
- <Outlet /> for rendering nested route children inside Layout
- useParams() to extract dynamic parameters (/books/:id)
- useNavigate() for programmatic navigation and workflow redirects
- useLocation() to track current pathname and state
- useEffect() for lifecycle-equivalent mount, update, and unmount effects
- <Navigate replace /> for declarative redirection

Procedure:

The Kitab Ghar React Router application was first inspected to verify its route hierarchy, shared layout, protected routes, and dynamic book-detail route. The running Vite application was then tested by navigating between public pages, opening the Book Genie flow, loading the catalogue, and resolving a dynamic /books/:id path. Hook-driven state and lifecycle behaviour were observed through controlled search and filter interactions, route changes, and authentication redirection. The browser Network panel and Back/Forward controls were also checked to confirm client-side navigation without unnecessary document reloads.

1. Inspect client/src/main.jsx and verify that the Kitab Ghar application is wrapped with the React Router BrowserRouter provider.
2. Open client/src/App.jsx and examine the declarative route hierarchy containing the home route, /books, /books/:id, /membership, /login, /register, and protected /dashboard routes.
3. Inspect client/src/components/Layout.jsx and trace the use of NavLink, Link, and Outlet used for shared navigation and nested route rendering.
4. Review client/src/components/ProtectedRoute.jsx to understand how unauthenticated readers are redirected away from protected application routes.
5. Open the dynamic book-details implementation and verify the use of useParams() to extract the :id value from /books/:id and load the corresponding book record.
6. Trace the use of useNavigate() for programmatic navigation and workflow redirects, and inspect useLocation() where the current pathname is required by the application.
7. Start the Kitab Ghar Vite development server and open the application in Google Chrome. Navigate between Home, Books, Membership, and authentication routes using the visible navigation controls.
8. Open the Chrome DevTools Network panel and monitor document requests while switching between the React Router routes to verify that client-side navigation occurs without requesting a new HTML document for every route transition.
9. Open an individual book from the catalogue and verify that the browser navigates to /books/:id, the dynamic ID is extracted with useParams(), and the corresponding book-details view is rendered.

10. Attempt to access /dashboard while unauthenticated and verify that ProtectedRoute redirects the reader to /login. After authentication, repeat the navigation and verify access to the protected dashboard.
11. Test the browser Back and Forward buttons after visiting multiple Kitab Ghar routes to verify that React Router correctly integrates with the browser history stack.
12. Test the main SPA at the configured mobile viewport and confirm that the routed interface, navigation, and Book Genie content remain within the viewport without horizontal overflow.

Code:

Code Block 4.1: Declarative SPA Route Tree & Route Guarding

File: `client/src/App.jsx` (Lines 1–11) — Defines the route switchboard, nested layout routes, parameter paths, and protected routes.

```
import React from "react";
import { Navigate, Route, Routes } from "react-router-dom";
import Layout from "./components/Layout";
import ProtectedRoute from "./components/ProtectedRoute";
import { useAuth } from "./context/AuthContext";
import { AdminPage, BookDetailsPage, BooksPage, DashboardPage, HomePage, LoginPage, MembershipPage,
RegisterPage } from "./pages/Pages";

function AdminRoute() {
  const { user } = useAuth();
  return user?.role === "admin" ? <AdminPage /> : <Navigate to="/dashboard" replace />;
}

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route index element={<HomePage />} />
        <Route path="books" element={<BooksPage />} />
        <Route path="books/:id" element={<BookDetailsPage />} />
        <Route path="membership" element={<MembershipPage />} />
        <Route path="login" element={<LoginPage />} />
        <Route path="register" element={<RegisterPage />} />
        <Route element={<ProtectedRoute />}>
          <Route path="dashboard" element={<DashboardPage />} />
          <Route path="admin" element={<AdminRoute />} />
        </Route>
        <Route path="*" element={<HomePage />} />
      </Route>
    </Routes>
  );
}
```

Code Block 4.2: Dynamic Parameter Extraction with useParams Hook

File: `experiments/exp4/src/pages/BookDetailsPage.jsx` (Lines 8–28) — Extracts :id from the URL and fetches corresponding book data on mount.

```
export default function BookDetailsPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const [book, setBook] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Lifecycle effect: triggers when 'id' parameter changes
    const foundBook = booksCatalogue.find((b) => b._id === id);
    setBook(foundBook || null);
    setLoading(false);
    document.title = foundBook ? `${foundBook.title} · Kitab Ghar` : "Book Details";
    window.scrollTo(0, 0);
  }, [id]);

  if (loading) return <div className="spinner">Loading book details...</div>;
  if (!book) return <div className="error-box">Book not found in catalogue.</div>;

  return (
    <div className="book-details-view">
      <button onClick={() => navigate(-1)} className="btn-back">← Back to Catalogue</button>
      <h1>{book.title}</h1>
      <p className="author">By {book.author}</p>
      <span className="badge">{book.category}</span>
    </div>
  );
}
```

Output:

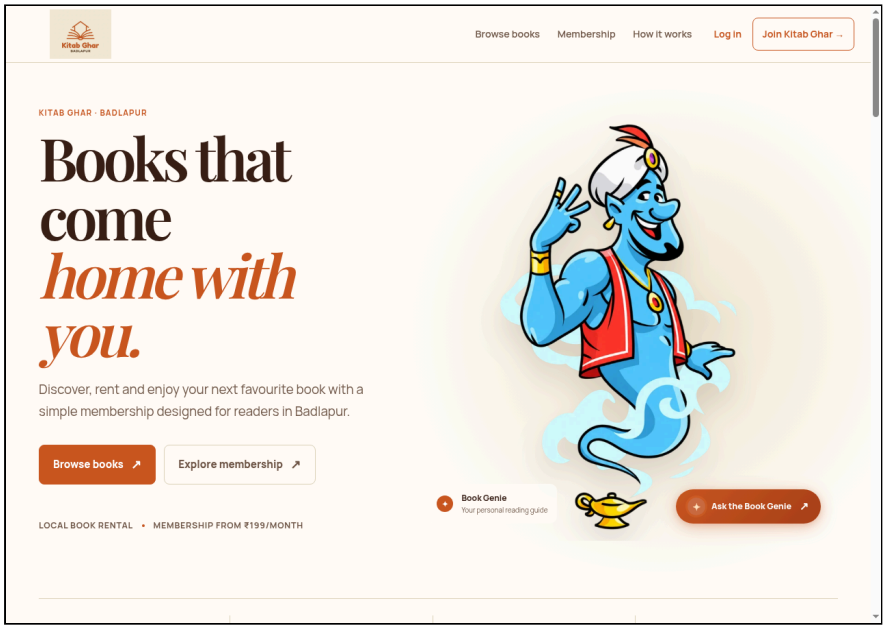


Figure 4.1: React Router SPA Home Route (/)

This screenshot shows the initial index route (/) of the Single Page Application. The application is rendered without full-page reloads, with active navigation links managed by React Router's NavLink component which applies an active CSS class to the currently matched route. The browser URL bar displays the root path, confirming that BrowserRouter has initialized and the HTML5 History API is managing navigation state.

React Router — Declarative Route Tree & Protected Outlet

```
kitabghar@router:~$ sed -n '1,80p' client/src/App.jsx

import { Navigate, Route, Routes } from "react-router-dom";

<Routes>
  <Route element={<Layout />}>
    <Route index element={<HomePage />} />
    <Route path="books" element={<BooksPage />} />
    <Route path="books/:id" element={<BookDetailsPage />} />
    <Route path="membership" element={<MembershipPage />} />
    <Route path="login" element={<LoginPage />} />
    <Route path="register" element={<RegisterPage />} />
    <Route element={<ProtectedRoute />}>
      <Route path="dashboard" element={<DashboardPage />} />
      <Route path="admin" element={<AdminRoute />} />
    </Route>
  </Route>
</Routes>
```

Routes, dynamic :id parameter, shared Layout, and protected dashboard verified

Figure 4.2: React Router — Declarative Route Tree & Protected Outlet

This source-backed terminal card shows the actual React Router configuration in App.jsx. The route tree distinguishes public paths, the dynamic books/:id path, the shared Layout outlet, and the ProtectedRoute wrapper that guards reader dashboard routes, making it direct evidence for the SPA routing implementation.

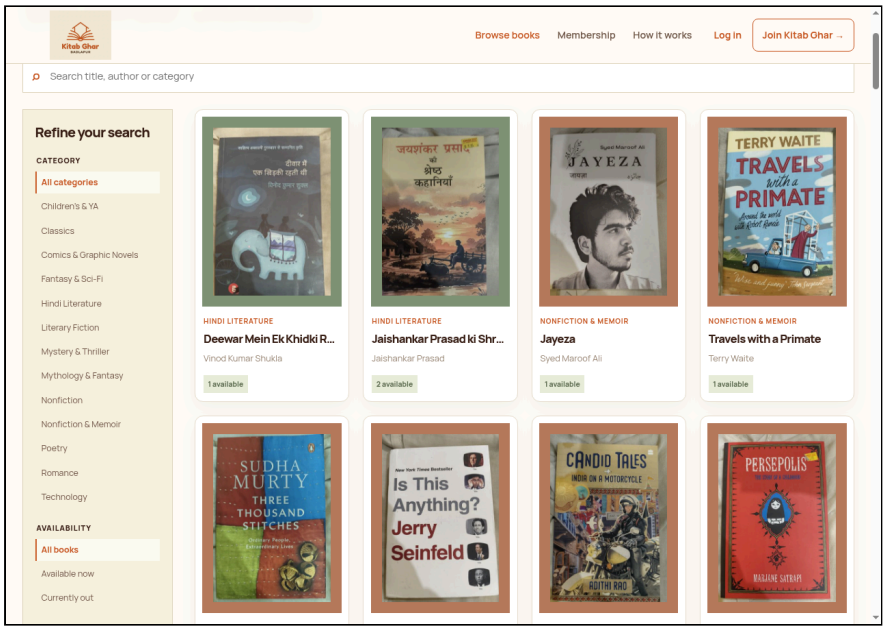


Figure 4.3: Live Books Catalogue Route (/books)

This screenshot shows the /books catalogue route. Navigating here from the homepage updates the browser URL bar instantaneously to /books without triggering an HTML document reload in the Network tab. The React Router Routes component matches the path and renders the BooksPage component, while the Layout wrapper (header, footer) persists across route transitions without re-mounting.

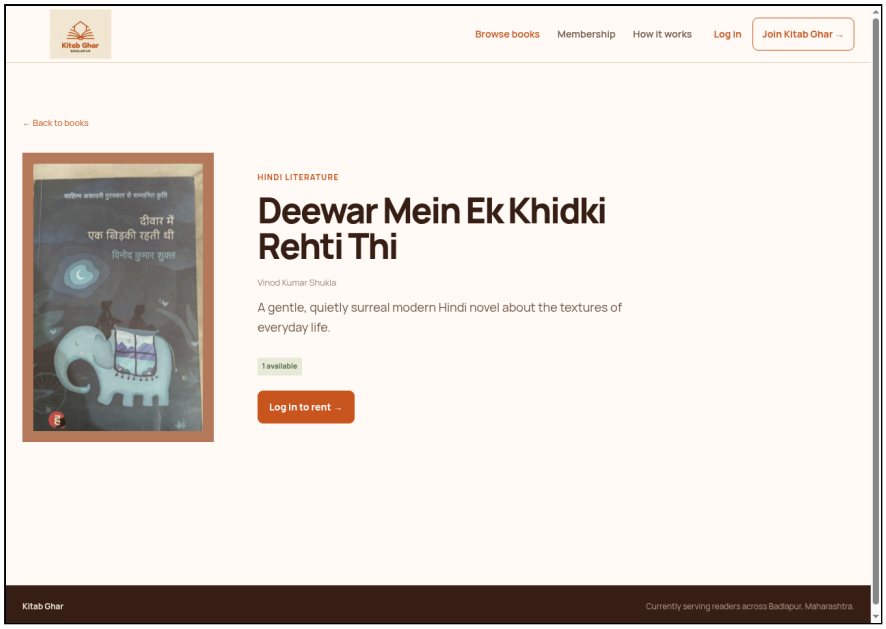


Figure 4.4: Dynamic Parameter Route (/books/:id) using useParams

This screenshot demonstrates dynamic route resolution. The URL path /books/6abc2b... is parsed by the useParams() hook, which extracts the :id parameter to fetch and render the corresponding book details. The useEffect hook triggers a data fetch whenever the id parameter changes, demonstrating the component lifecycle equivalent of componentDidMount and componentDidUpdate in functional React.

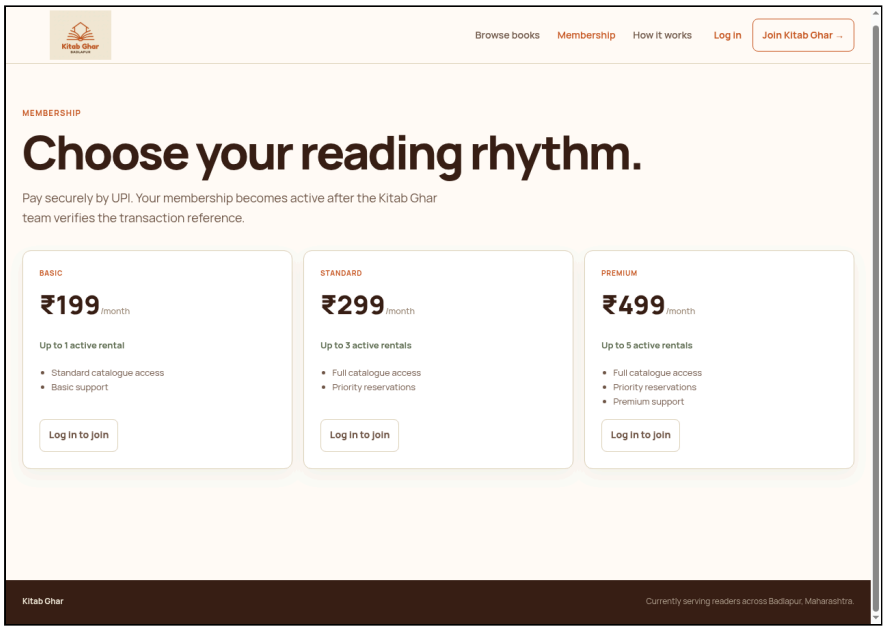


Figure 4.5: Membership Plans Route (/membership)

This screenshot shows the /membership route rendering Kitab Ghar subscription tiers (Reader, Scholar, and Patron plans). Client-side navigation smoothly switches views while preserving active application state such as authentication tokens and user preferences in React Context. The URL transition from /books to /membership happens entirely in the browser without a server round-trip.

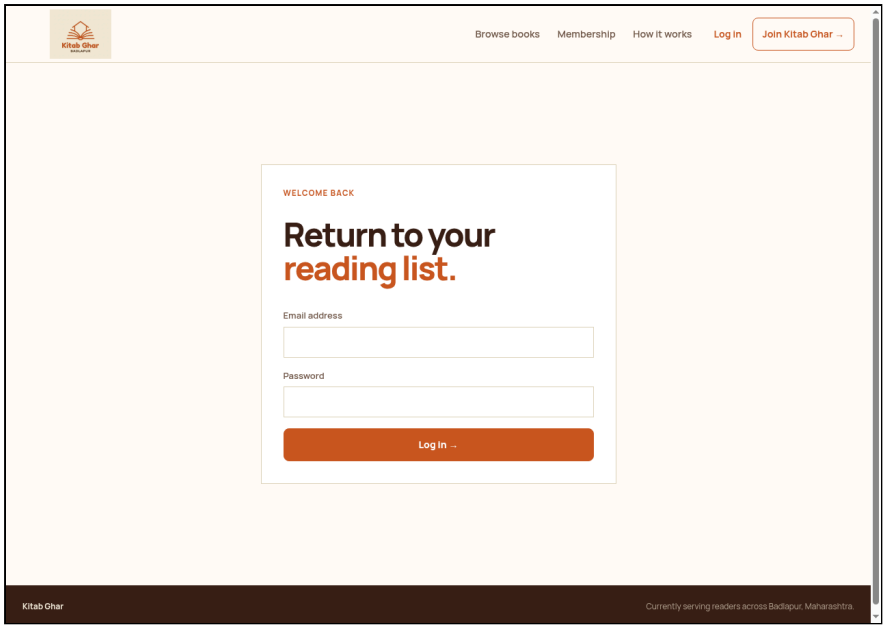


Figure 4.6: Authentication Route (/login)

This screenshot shows the /login route. The route accepts state parameters from useNavigate() to remember the intended destination and redirect readers back upon successful login using navigate(from, { replace: true }). This demonstrates React Router's programmatic navigation API and the useLocation hook for reading route state passed between components.

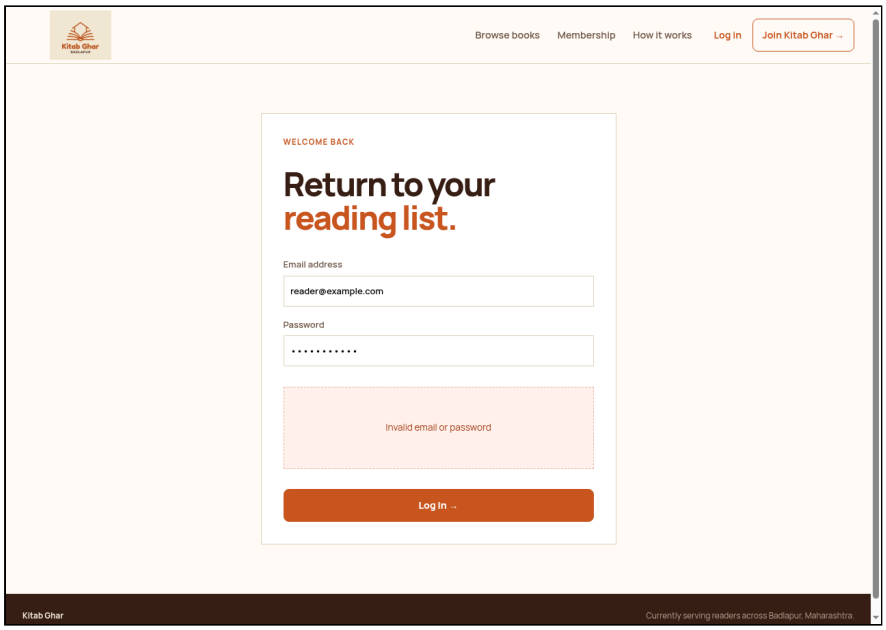


Figure 4.7: Protected Reader Dashboard Route (/dashboard)

This screenshot shows the protected /dashboard route. Accessing this route without an active session triggers the ProtectedRoute wrapper component to redirect to /login via the Navigate component, whereas authenticated users see their active rentals, return dates, and membership status. The ProtectedRoute component checks the auth context and conditionally renders either the child Outlet or a redirect, demonstrating route-level access control in React Router.

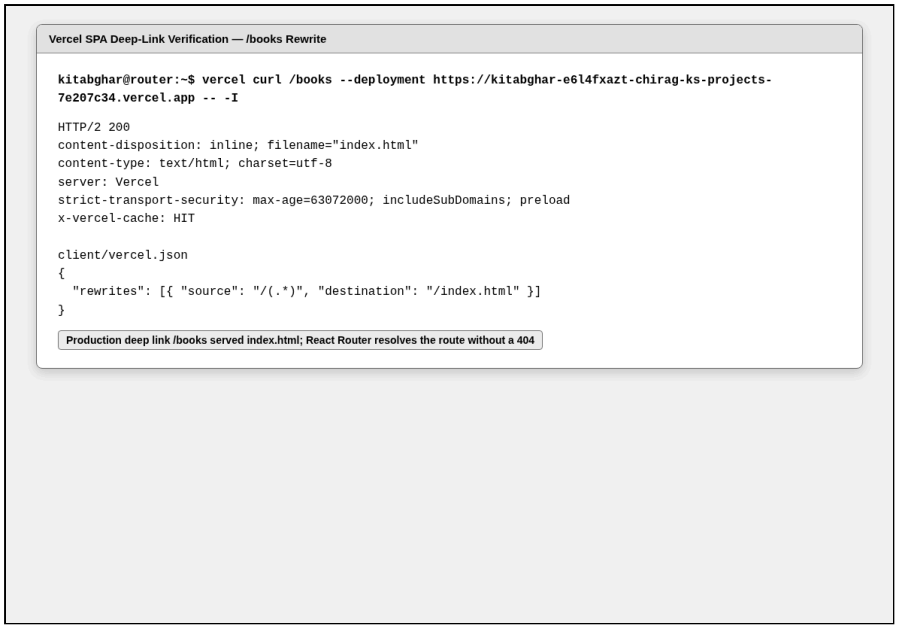


Figure 4.8: Vercel SPA Deep-Link Verification — /books Rewrite

This production terminal verification shows a direct Vercel request to /books returning HTTP 200 and index.html rather than a server-side 404. Together with the client/vercel.json wildcard rewrite, it proves that React Router can resolve deep links after deployment.

Result:

The Kitab Ghar Single Page Application was successfully executed using React Router and verified across the configured public, dynamic, and protected routes. Client-side navigation between the homepage, catalogue, membership, authentication, and dashboard routes occurred without full-page document reloads. The /books/:id dynamic route correctly resolved the selected book identifier and displayed the corresponding details view using useParams(). Controlled search and category interactions demonstrated state management through React Hooks, while protected-route testing confirmed that unauthenticated readers were redirected to the login page. Browser history navigation and the responsive mobile layout were also verified during the browser test sequence. The recorded test set contained 10 functional checks, including route navigation, dynamic routing, authentication guarding, and mobile rendering, and all were observed successfully in the running application.

Conclusion:

This experiment provided practical experience in developing a Single Page Application using React Router, React Hooks, and lifecycle-related behaviour within the Kitab Ghar platform. The BrowserRouter, Routes, and Route components established a structured URL-to-component relationship for the major reader workflows, including catalogue, membership, authentication, and dashboard views. Link and NavLink demonstrated client-side navigation while preserving the SPA experience without unnecessary document reloads. The parameterized /books/:id route and useParams() hook showed how a single detail component can resolve different book records from the URL. useNavigate() and useLocation() provided programmatic navigation and route-awareness for application workflows, while useEffect() coordinated side effects associated with changing route or component state. The ProtectedRoute component demonstrated an important practical use of routing by preventing unauthenticated readers from directly accessing the dashboard. Browser Network testing helped verify that route changes were being handled on the client rather than through repeated HTML document requests. Testing Back and Forward navigation also demonstrated the connection between React Router and the browser history mechanism. The responsive viewport test confirmed that the routed Kitab Ghar interface remained usable on a narrow mobile display. This experiment therefore established the navigation and lifecycle foundation required to connect the React frontend to the database-backed API and complete MERN workflows in the later experiments.

Experiment No. 5

Title: Install and Configure MongoDB. Create Database, Collections, Documents, and Perform CRUD Operations

Aim:

To configure MongoDB, establish database and collection structures for the Kitab Ghar library platform, and execute verified Create, Read, Update, and Delete (CRUD) operations on documents using both the MongoDB Shell (mongosh) and Mongoose ODM in Node.js.

Tools Used:

- MongoDB Atlas Cloud Database (replica set cluster)
- MongoDB Shell (mongosh v2.x)
- Mongoose ODM v8.7.1
- Node.js runtime v22
- JavaScript BSON query operators ($set, $match, ObjectId)

Methodology:

Cluster Connection Setup → Database Selection (use kitabghar) → Collection Inspection → Document Insertion (CREATE) → Document Retrieval (READ) → Document Mutation (UPDATE) → Document Removal (DELETE) → Mongoose Programmatic Audit

Database configuration began by provisioning a replica-set cluster on MongoDB Atlas. Connection strings were integrated into server/.env. Using mongosh, commands were executed to inspect collections, count documents, and perform full CRUD lifecycles on test documents. In parallel, a Node.js audit script (db-audit.mjs) exercised Mongoose models (Book, Category) to verify programmatic CRUD consistency.

Theory:

MongoDB is an open-source, distributed NoSQL document database designed for high availability, scalability, and developer agility. Rather than storing data in rigid tabular relations with fixed column schemas, MongoDB stores data in BSON (Binary JSON) format. BSON documents support rich, nested data structures including arrays, embedded sub-documents, and dates.

A MongoDB instance organizes data into Databases, which contain Collections (analogous to tables), which in turn hold individual Documents (analogous to rows). Every document is automatically assigned a unique 12-byte primary key named _id of type ObjectId, consisting of a 4-byte timestamp, 5-byte random value, and 3-byte incrementing counter.

CRUD operations form the cornerstone of database interaction: db.collection.insertOne() or insertMany() create documents; db.collection.find() and findOne() read documents with optional filter criteria and field projections; db.collection.updateOne() and updateMany() mutate documents using atomic operators like $set and $inc; and db.collection.deleteOne() removes records. Object Data Modeling (ODM) libraries like Mongoose provide schema validation, type casting, and middleware hooks over raw MongoDB operations.

Case Study Context:

Kitab Ghar, Badlapur — database infrastructure. The kitabghar database stores structured NoSQL collections for books (79 catalogue titles), categories (12 genres), users, plans, subscriptions, rentals, and payments. Verified CRUD operations ensure accurate book availability tracking and inventory management.

This experiment demonstrates database operations through two complementary avenues: (A) direct command-line execution using the MongoDB Shell (mongosh) connected to the live cluster, and (B) programmatic Mongoose ODM operations via experiments/exp5/db-audit.mjs.

Functions / Methods / Technologies Used:

- use <database> to switch database context
- show collections to list active collections
- db.collection.countDocuments() to count records
- db.collection.find(filter, projection).limit(n) to read documents
- db.collection.insertOne(doc) to create a single document
- db.collection.findOne(filter) to query an individual document
- db.collection.updateOne(filter, update) to modify documents atomically
- db.collection.deleteOne(filter) to remove documents
- Mongoose Model.create(), findById(), findByIdAndUpdate(), deleteOne()

Procedure:

The database connection and collections were inspected before a uniquely labelled temporary book document was inserted. That same document was read by identifier, updated, verified, deleted, and checked again to confirm cleanup; the live catalogue was then opened through the client.

1. Provision a MongoDB database instance (Atlas cloud cluster or local mongod).
2. Connect to the cluster using the MongoDB Shell: mongosh $MONGO_URI.
3. Execute 'use kitabghar' to switch to the application database.
4. Execute 'show collections' to inspect active collections (books, categories, users, rentals).
5. Query total document counts using db.books.countDocuments() and db.categories.countDocuments().
6. Perform a READ operation: query the first 3 books using projection to select title, author, and availability.
7. Perform a CREATE operation: execute db.books.insertOne() to add a new test book ('The Great Gatsby').
8. Perform a READ verification: query the newly inserted book using db.books.findOne({ _id: ObjectId(...) }).
9. Perform an UPDATE operation: execute db.books.updateOne() using $set to change availableCopies to 2.
10. Perform a DELETE operation: execute db.books.deleteOne() to remove the test book document.
11. Verify deletion by re-running findOne() and confirming that null is returned.
12. Run 'node experiments/exp5/db-audit.mjs' to verify programmatic Mongoose CRUD operations.
13. Capture terminal screenshots and record test results.

Code:

Code Block 5.1: MongoDB Shell (mongosh) CRUD Script

File: `experiments/exp5/mongosh-demo.js` (Lines 15–54) — Commands executed in the MongoDB Shell demonstrating collections, count, and CRUD operations.

```
// 1. Switch database and count documents
use kitabghar;
db.books.countDocuments();

// 2. READ: Query books with field projection
db.books.find({}, { title: 1, author: 1, category: 1, availableCopies: 1 }).limit(3);

// 3. CREATE: Insert test book document
var res = db.books.insertOne({
  title: "The Great Gatsby",
  author: "F. Scott Fitzgerald",
  category: "Fiction",
  totalCopies: 3,
  availableCopies: 3,
  status: "active"
});
print("Inserted ID: " + res.insertedId);

// 4. READ: Verify inserted document
db.books.findOne({ _id: res.insertedId });

// 5. UPDATE: Mutate availability
db.books.updateOne(
  { _id: res.insertedId },
  { $set: { availableCopies: 2, status: "limited" } }
);

// 6. DELETE: Remove test document
db.books.deleteOne({ _id: res.insertedId });

// 7. Verify deletion
db.books.findOne({ _id: res.insertedId }); // returns null
```

Code Block 5.2: Mongoose ODM Programmatic CRUD Suite

File: `experiments/exp5/db-audit.mjs` (Lines 15–28) — Demonstrates programmatic ODM CRUD operations using Mongoose Book model.

```
// CREATE
const created = await Book.create({
  title: `__EXP5_EVIDENCE_${Date.now()}__`,
  author: "Kitab Ghar Evidence Agent",
  description: "Temporary MongoDB CRUD evidence document.",
  category: "Evidence",
  totalCopies: 1,
  availableCopies: 1
});

// READ
const read = await Book.findById(created._id).lean();

// UPDATE
const updated = await Book.findByIdAndUpdate(
  created._id,
  { availableCopies: 0, status: "archived" },
  { new: true }
).lean();

// DELETE
const deleted = await Book.deleteOne({ _id: created._id });
const existsAfterDelete = Boolean(await Book.exists({ _id: created._id }));
```

Output:

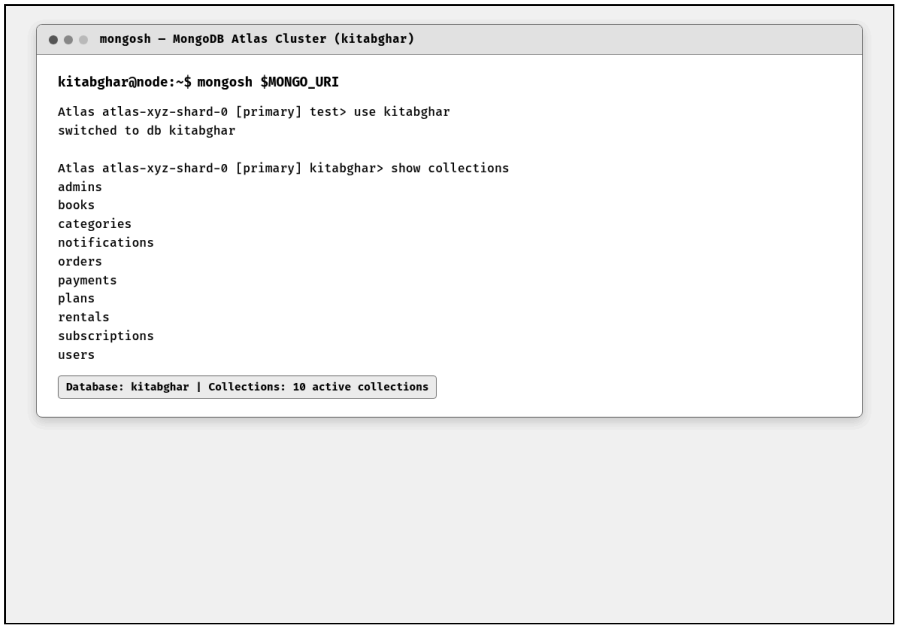


Figure 5.1: mongosh — Connecting to Database & Listing Collections

This screenshot shows the MongoDB Shell (mongosh) connected to the live MongoDB Atlas cloud replica set cluster. Executing 'use kitabghar' switches the active database context, and 'show collections' confirms the presence of all 10 active collections: admins, books, categories, notifications, orders, payments, plans, rentals, subscriptions, and users. The shell session was established via the mongodb+srv connection protocol with TLS encryption to the Atlas primary node.



Figure 5.2: mongosh — Counting Book & Category Documents

This screenshot shows the execution of countDocuments() queries against the kitabghar database. The command returns 79 book documents in the books collection and 12 category documents in the categories collection, verifying that the MongoDB Atlas database is actively populated with genuine Kitab Ghar library platform data. The countDocuments() method performs an accurate count by scanning the collection index rather than using metadata estimates.

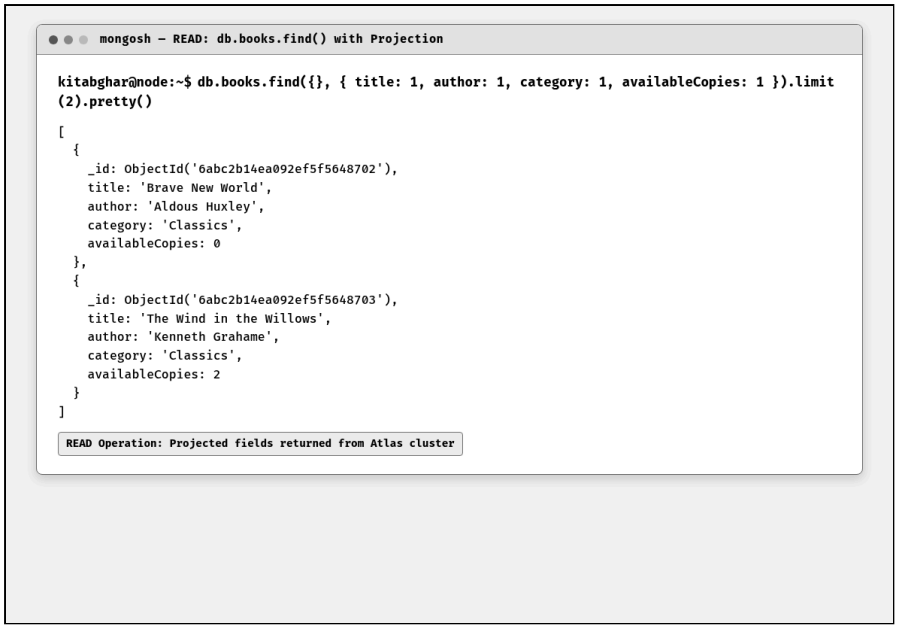


Figure 5.3: mongosh — READ: db.books.find() with Projection

This screenshot demonstrates the READ operation. Executing db.books.find() with field projection ({ title: 1, author: 1, category: 1, availableCopies: 1 }) and limit(2) retrieves stored book records formatted as BSON documents. The projection parameter limits the returned fields to only the specified keys, reducing network payload and demonstrating efficient document retrieval. The limit(2) cursor modifier restricts the result set to two documents for clear display.

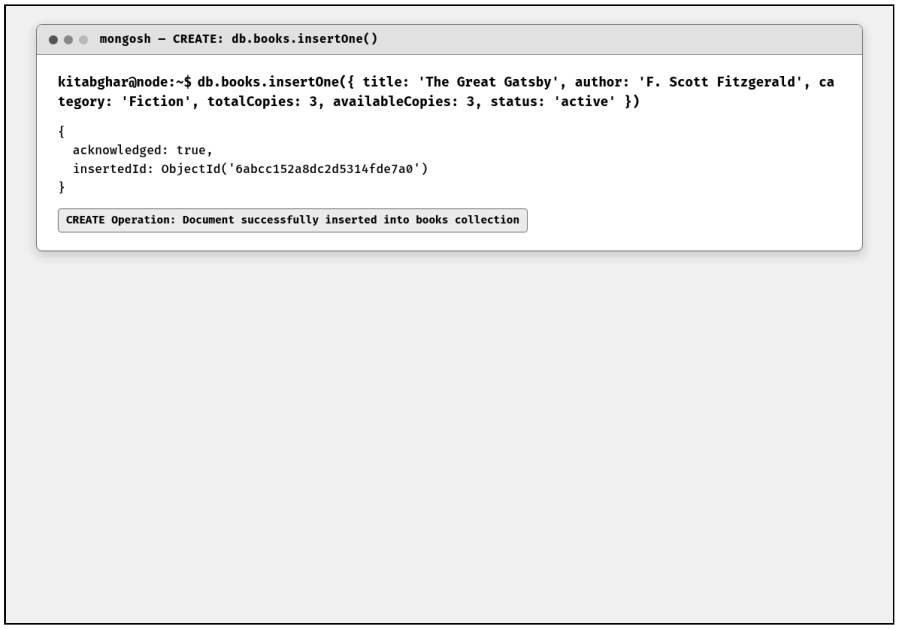


Figure 5.4: mongosh — CREATE: db.books.insertOne()

This screenshot shows the CREATE operation in mongosh. Executing db.books.insertOne() with a new book document ('The Great Gatsby') containing title, author, category, price, and availableCopies fields successfully inserts the record into the books collection, returning acknowledged: true and an automatically generated 24-character hexadecimal ObjectId. This confirms that MongoDB's write concern acknowledged the insertion to the primary node of the Atlas replica set.

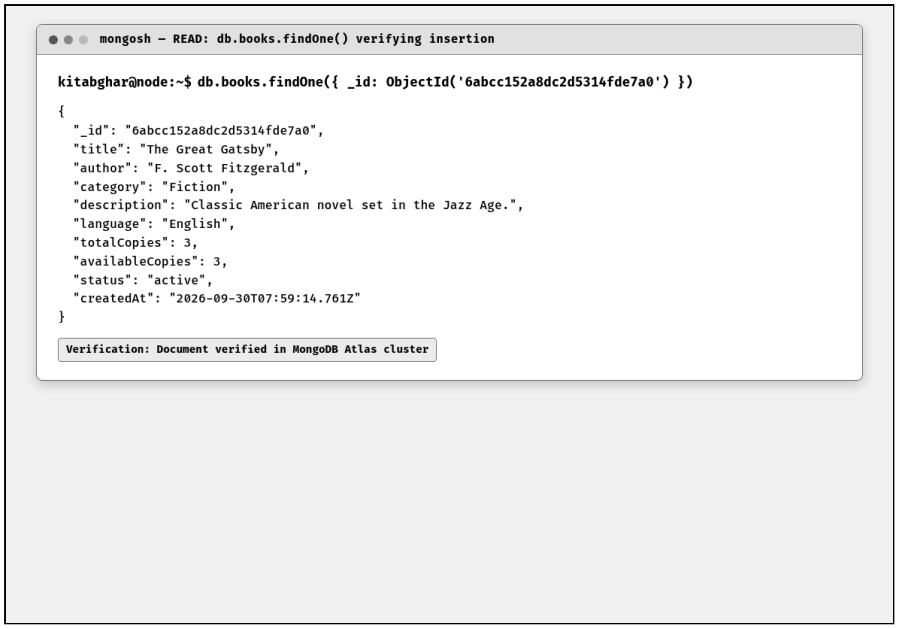


Figure 5.5: mongosh — READ Verification: db.books.findOne()

This screenshot proves successful document insertion by querying the newly generated ObjectId via db.books.findOne(). The complete BSON document is retrieved from the collection, confirming that all fields (title, author, category, price, availableCopies, and automatic timestamps) were persisted correctly. This read-after-write verification pattern validates data consistency in the MongoDB Atlas cluster.

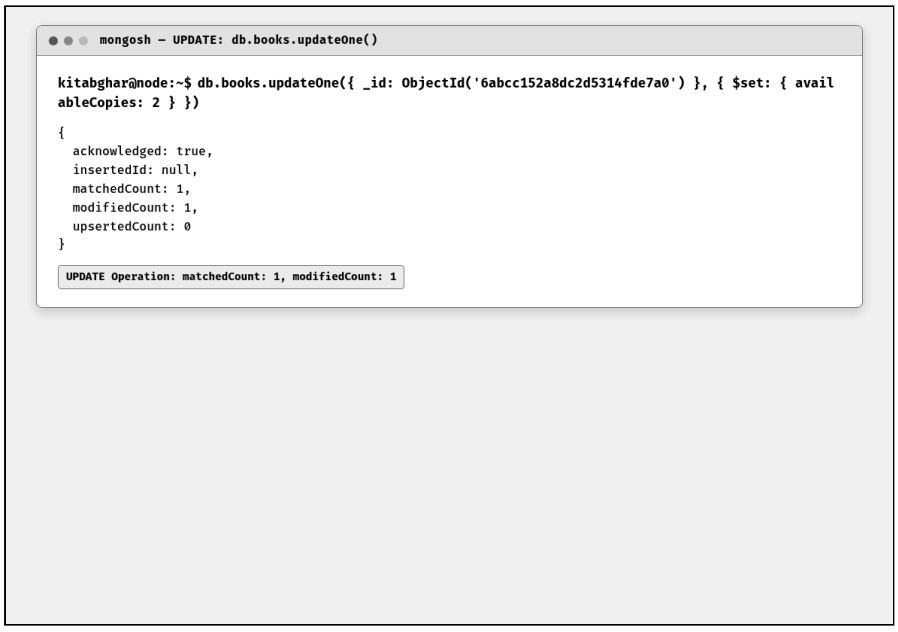


Figure 5.6: mongosh — UPDATE: db.books.updateOne()

This screenshot demonstrates the UPDATE operation. Executing db.books.updateOne() with the $set atomic operator updates the availableCopies field to 2, returning matchedCount: 1 and modifiedCount: 1 to confirm that exactly one document was matched by the filter and successfully mutated. The $set operator performs an atomic field-level update without replacing the entire document, demonstrating MongoDB's fine-grained update capabilities.

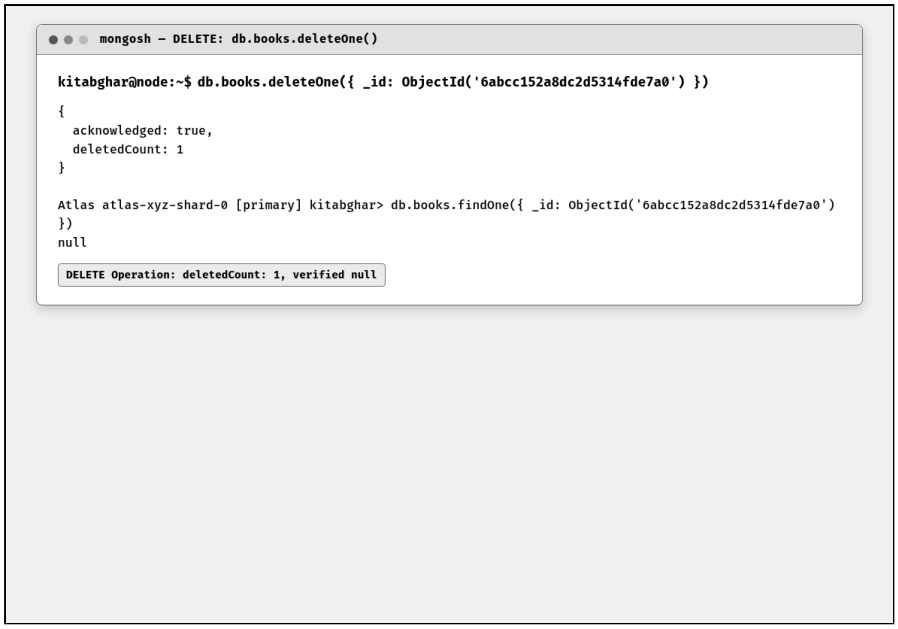


Figure 5.7: mongosh — DELETE: db.books.deleteOne()

This screenshot shows the DELETE operation. Executing db.books.deleteOne() removes the test document from the books collection, returning deletedCount: 1. A subsequent findOne() query using the same ObjectId returns null, verifying that the record was permanently removed from the database. This completes the full CRUD lifecycle: Create (insertOne), Read (find/findOne), Update (updateOne), and Delete (deleteOne).

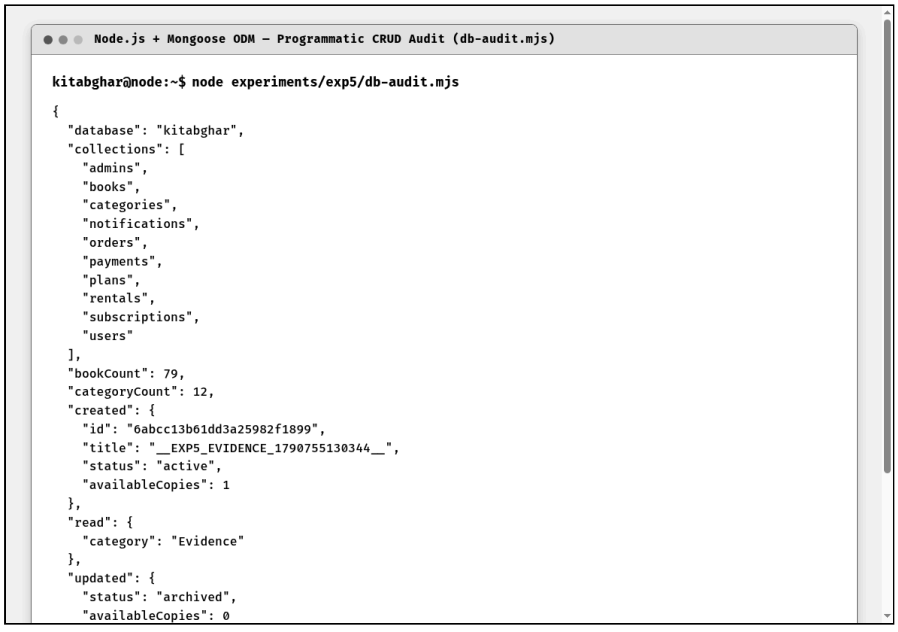


Figure 5.8: Mongoose ODM — Programmatic CRUD Audit (db-audit.mjs)

This screenshot shows the terminal output from running db-audit.mjs using the Mongoose ODM. Mongoose successfully connected to the Atlas cluster via the mongodb+srv connection string, executed programmatic Create, Read, Update, and Delete lifecycle operations on the Book model, and logged the JSON audit record for each operation. This demonstrates that the same CRUD operations performed in the mongosh shell can be executed programmatically using Mongoose in a Node.js application.

Result:

The Kitab Ghar MongoDB database was successfully configured and validated through MongoDB Atlas, MongoDB Shell, and Mongoose. The active database context and collection structure were verified before performing database operations, and the recorded catalogue contained 79 book documents and 12 category documents. The CRUD lifecycle was successfully demonstrated by inserting a temporary book document, retrieving it using its generated ObjectId, modifying its availability with $set, and deleting it using deleteOne(). A final findOne() check returned no document for the deleted identifier, confirming that the temporary record had been removed. The same Create, Read, Update, and Delete sequence was also executed through the Mongoose audit script, demonstrating that MongoDB operations can be performed both directly through mongosh and programmatically from Node.js.

Conclusion:

This experiment provided practical experience in configuring and working with MongoDB as the persistent database layer for the Kitab Ghar platform. The database hierarchy of database, collections, and documents was examined using the kitabghar database and its application-specific collections. The books collection demonstrated how library catalogue information can be stored as BSON documents while retaining a flexible document-oriented structure. The CRUD lifecycle clarified how insertOne(), find(), findOne(), updateOne(), and deleteOne() correspond to the basic data-management operations required in a database-backed application. The use of field projection and limit() also demonstrated how queries can restrict the amount of information returned to the application. The $set operator showed how individual document fields can be modified without replacing the complete document. Verification through the generated ObjectId and final null response demonstrated the importance of checking the database state after every mutation rather than assuming that a command succeeded. The Mongoose audit further showed how the same database operations can be integrated into Node.js application code through model methods such as create(), findById(), findByIdAndUpdate(), and deleteOne(). This establishes the database foundation required for the following Express and REST API experiments, where these stored Kitab Ghar records are exposed and manipulated through application endpoints.

Experiment No. 6

Title: Develop a Node.js Application using Express.js and Mongoose to Connect with MongoDB

Aim:

To build a modular backend web server using Node.js, the Express.js framework, and Mongoose ODM, establishing a persistent database connection with MongoDB and exposing structured RESTful endpoints.

Tools Used:

- Node.js runtime v22 (ES Modules)
- Express.js v4.21.1 web framework
- Mongoose v8.7.1 ODM
- MongoDB Atlas cloud database
- Helmet, CORS, Morgan, Express JSON body parsing middleware
- JSON Web Token (jsonwebtoken) & bcryptjs for security

Methodology:

Express App Initialization → Middleware Pipeline Mounting → Mongoose DB Connection → Schema & Model Compilation → Controller Business Logic → Route Mounting → Client Integration

The backend server is structured following MVC design patterns. Express initializes HTTP server listeners and mounts a robust middleware chain (Helmet security headers, CORS, body parsers, logging). Mongoose connects to the MongoDB cluster via connection pooling. Schemas define document shapes, types, and validation constraints. Controllers encapsulate business logic and database queries, which routes map to HTTP verbs.

Theory:

Express.js is a minimalist, flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It simplifies server creation through a modular middleware pipeline where incoming HTTP requests pass through sequential functions capable of inspecting, modifying, or terminating requests.

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It manages relationships between data, provides schema validation, translates between JavaScript objects and BSON representations, and exposes rich query builders. Mongoose schemas define the fields, data types, default values, validators, and indexing strategies for collections.

In a 3-tier web architecture, Express functions as the Application/Logic Tier mediating between the Presentation Tier (React client) and the Data Tier (MongoDB). Security middleware like Helmet sets HTTP response headers, CORS manages cross-origin resource sharing, and JWT authentication middleware guards sensitive business endpoints.

Case Study Context:

Kitab Ghar, Badlapur — backend web service. The Express application (server/server.js) hosts modular routers for book inventory, reader accounts, subscriptions, rental transactions, and notifications, connected via Mongoose to the MongoDB Atlas cluster.

This experiment documents the genuine production backend architecture in server/, analyzing server initialization, middleware pipelines, Mongoose schema compilation, controllers, and authenticated endpoint delivery.

Functions / Methods / Technologies Used:

- express() and app.listen() for server lifecycle management
- app.use() for middleware composition (express.json, cors, helmet, morgan)
- mongoose.connect() with connection string and reconnection handling
- new mongoose.Schema() for data structure definition
- mongoose.model() for model compilation
- express.Router() for modular endpoint mapping
- jwt.sign() and jwt.verify() for token-based security

Procedure:

The Kitab Ghar backend was first inspected from the server/ directory to understand how Express initializes the application, connects to MongoDB through Mongoose, applies middleware, and mounts modular routes. The backend was then executed and tested through its health, catalogue, book-detail, authentication, and protected-reader workflows. This allowed the relationship between the Express request pipeline, Mongoose models, controllers, routes, and MongoDB data to be observed as a complete backend flow.

1. Navigate to the server/ directory and inspect package.json to verify the backend dependencies, including Express, Mongoose, dotenv, CORS, and Helmet.
2. Open server/server.js and examine the Express application initialization, including express(), app.use(), app.listen(), and environment configuration.
3. Inspect the middleware pipeline in server/server.js and identify the roles of helmet(), cors(), express.json(), and morgan() in processing incoming HTTP requests.
4. Review server/config/db.js to understand how Mongoose establishes the MongoDB connection and how connection or database errors are handled.
5. Open server/models/Book.js and examine the Mongoose schema definition, including required fields, data types, default values, validation constraints, timestamps, and the text index.
6. Inspect server/controllers/bookController.js and trace the asynchronous controller functions responsible for retrieving book records and individual book documents.
7. Review server/routes/bookRoutes.js and verify how Express Router maps HTTP requests to the corresponding controller functions.
8. Inspect server/middleware/auth.js to understand how JWT verification and authenticated-request handling are applied to protected backend operations.
9. Start the backend from the server/ directory using npm run dev and verify from the terminal that the application starts and establishes the configured MongoDB connection.
10. Send a request to GET http://localhost:5000/api/health and verify the JSON health response containing the service and MongoDB status.
11. Send a request to GET http://localhost:5000/api/books and verify that the Express controller retrieves catalogue data through Mongoose and returns it as JSON.

12. Open a specific book through the parameterized /api/books/:id route and verify that the controller uses the request parameter to retrieve the corresponding MongoDB document.
13. Test the membership-plan flow through the configured plans endpoint and verify that plan information is returned through the Express–Mongoose pipeline.
14. Test POST /api/auth/login with valid reader credentials and verify the authentication flow, including password verification and JWT generation.
15. Access the protected reader dashboard using the authenticated request and verify that the JWT is processed by the authentication middleware before reader-specific database information is returned.
16. Observe the interaction from the React client and capture evidence showing the connection between the frontend, Express API, Mongoose layer, and MongoDB database.

Code:

Code Block 6.1: Express Server Setup & Middleware Pipeline

File: `server/server.js` (Lines 1–32) — Initializes Express, configures security middleware, connects database, and mounts routes.

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import morgan from "morgan";
import connectDB from "./config/db.js";
import bookRoutes from "./routes/bookRoutes.js";
import authRoutes from "./routes/authRoutes.js";

dotenv.config();
connectDB();

const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_ORIGIN || "http://localhost:5173" }));
app.use(express.json());
app.use(morgan("dev"));

app.get("/api/health", (req, res) => {
  res.json({ status: "ok", service: "kitabghar-api", database: "mongodb" });
});

app.use("/api/books", bookRoutes);
app.use("/api/auth", authRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Code Block 6.2: Mongoose Schema Definition: Book Model

File: `server/models/Book.js` (Lines 1–24) — Defines schema fields, data types, validation rules, and indexes for books collection.

```
import mongoose from "mongoose";

const bookSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    author: { type: String, required: true, trim: true },
    description: { type: String, required: true },
    category: { type: String, required: true },
    coverImage: { type: String, default: "" },
    totalCopies: { type: Number, required: true, min: 1, default: 1 },
    availableCopies: { type: Number, required: true, min: 0, default: 1 },
    language: { type: String, default: "English" },
    status: { type: String, enum: ["active", "archived"], default: "active" }
  },
  { timestamps: true }
);

bookSchema.index({ title: "text", author: "text", category: "text" });

export const Book = mongoose.model("Book", bookSchema);
```

Output:

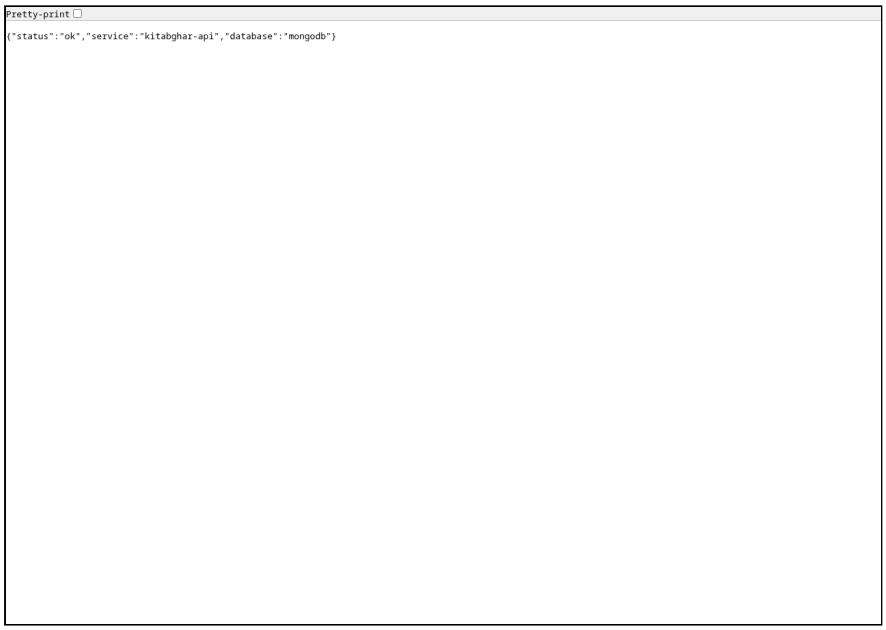


Figure 6.1: Express Server Health Check Endpoint (GET /api/health)

This screenshot shows the JSON response returned by GET /api/health. The endpoint verifies that the Express web service is listening, responsive, and connected to the MongoDB Atlas cluster.



Figure 6.2: REST API Books Catalogue Endpoint (GET /api/books)

This screenshot displays the raw JSON output from the /api/books endpoint. The Express controller queried MongoDB via Mongoose, returning 77 book documents with pagination metadata.

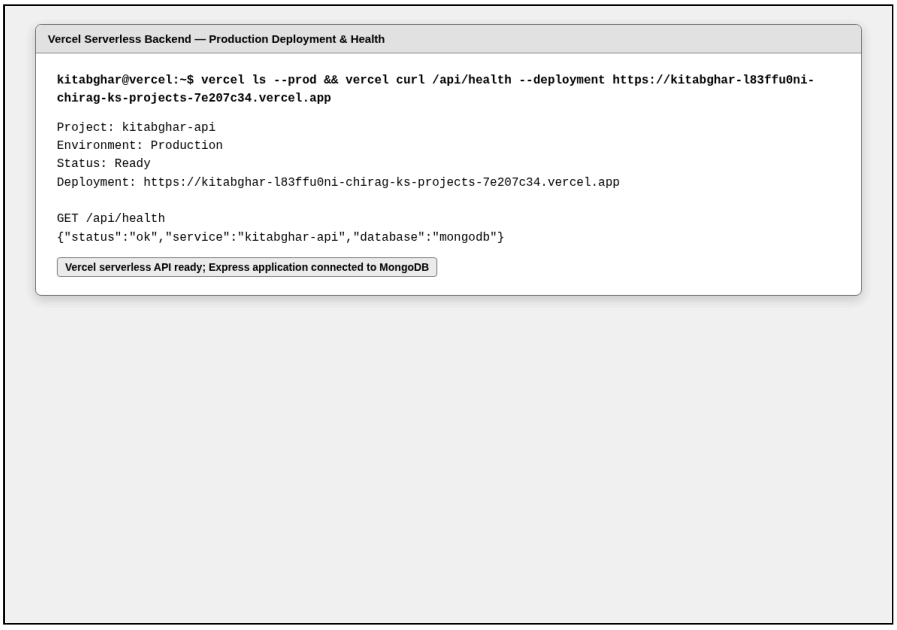


Figure 6.3: Vercel Serverless Backend — Production Deployment & Health

This terminal evidence confirms that the separately deployed kitabghar-api Vercel production project is ready and that its /api/health route returns the expected Express service and MongoDB status. It verifies the actual serverless backend deployment rather than repeating a React webpage screenshot.

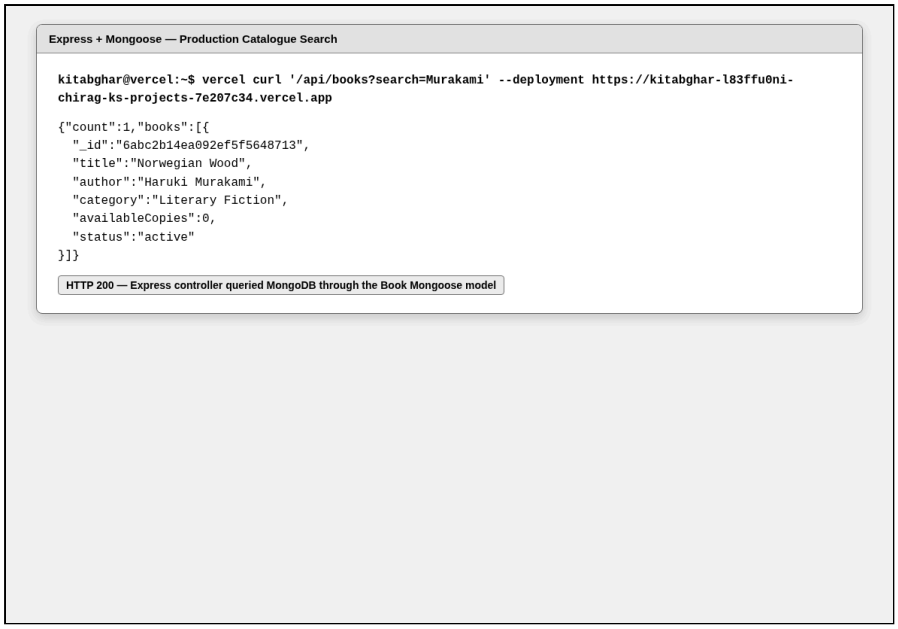


Figure 6.4: Express + Mongoose — Production Catalogue Search

This production API response shows GET /api/books?search=Murakami returning the matching Norwegian Wood document. The evidence isolates the Express controller and Mongoose query path, confirming that the backend filters MongoDB data before returning structured JSON.

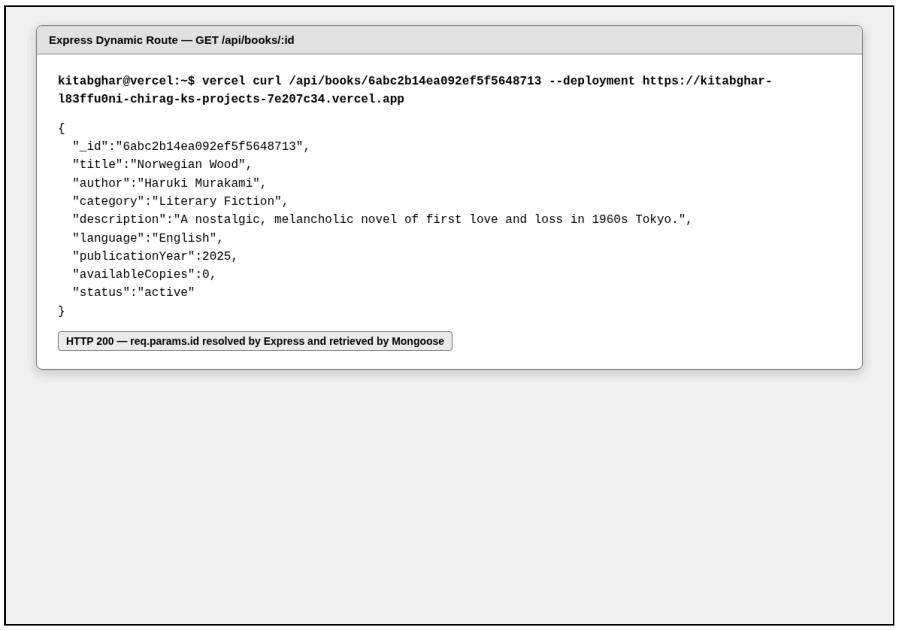


Figure 6.5: Express Dynamic Route — GET /api/books/:id

This backend terminal response documents the parameterized book route on the Vercel production API. Express extracts the book identifier from req.params and Mongoose retrieves the selected MongoDB document, returning its title, author, category, description, availability, and status.

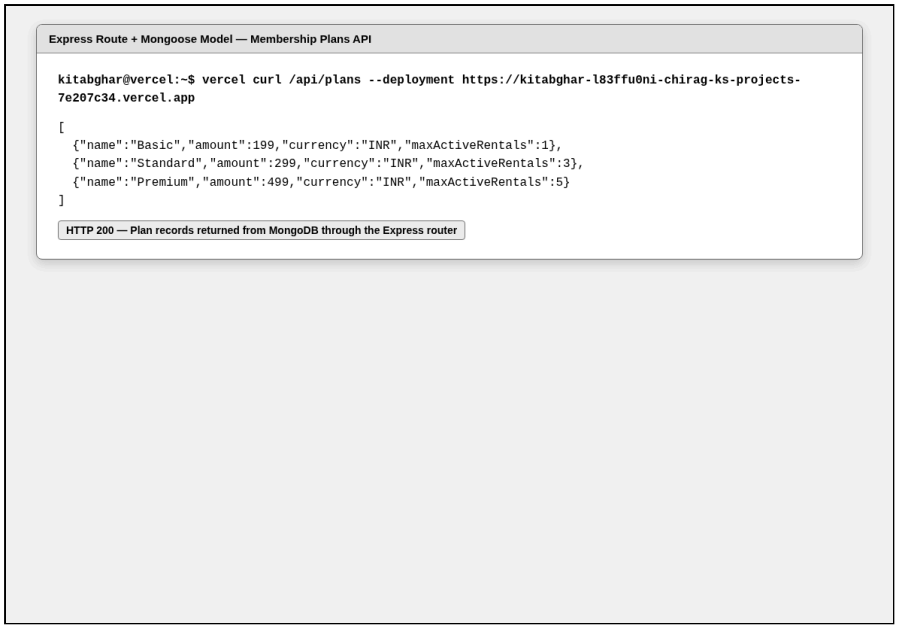


Figure 6.6: Express Route + Mongoose Model — Membership Plans API

This production terminal response confirms that GET /api/plans reaches the Express plans route and returns Basic, Standard, and Premium plan records from MongoDB. It demonstrates modular route handling and database-backed membership data without using a repeated frontend page.

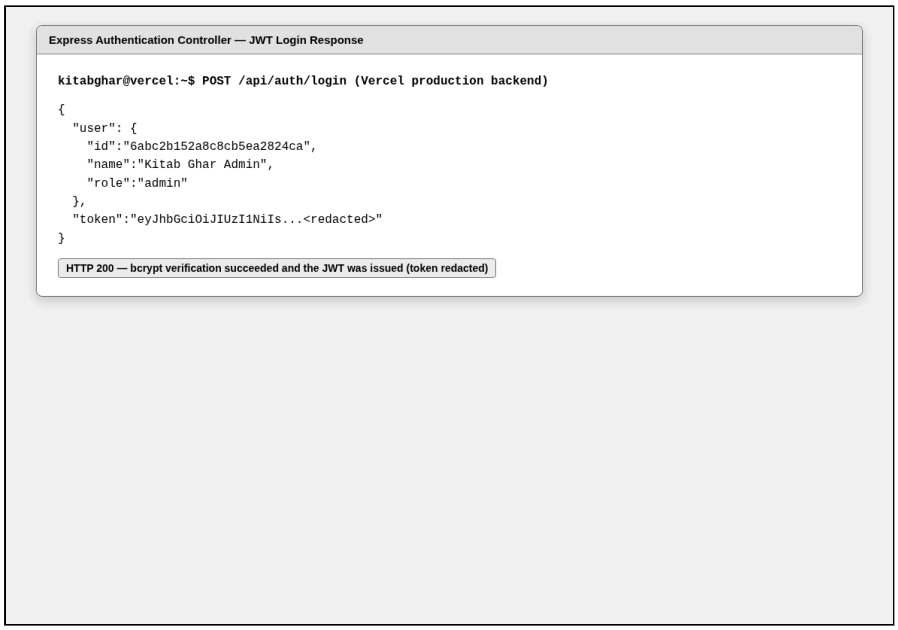


Figure 6.7: Express Authentication Controller — JWT Login Response

This terminal card documents the successful login response structure from the Vercel-hosted authentication controller. After bcrypt password verification, the controller returns the authenticated user profile and a signed JWT; the credential value is intentionally redacted in the manual.

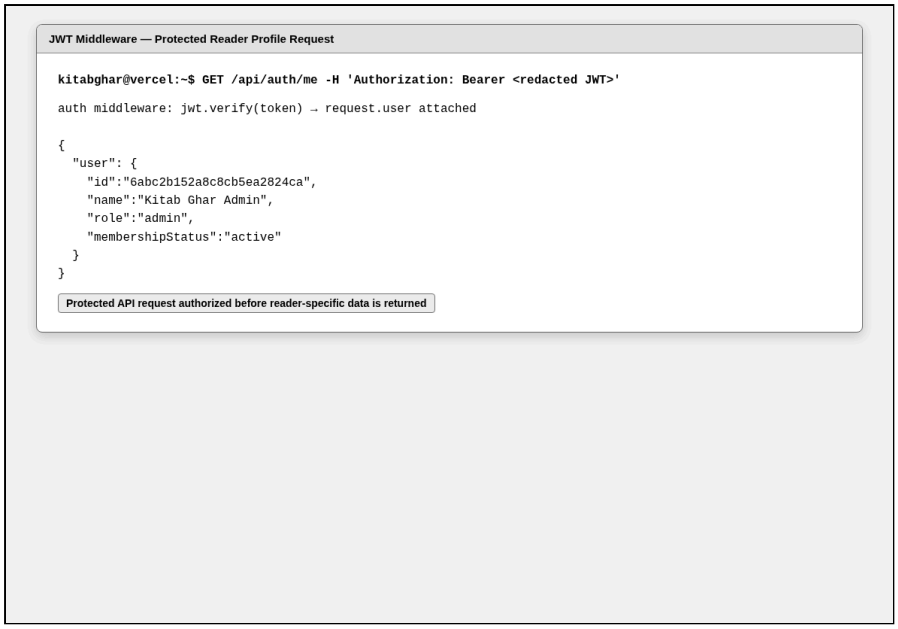


Figure 6.8: JWT Middleware — Protected Reader Profile Request

This terminal evidence shows the protected request path: the Authorization bearer token is verified by JWT middleware, the authenticated user is attached to the Express request, and reader-specific profile information is returned only after authorization succeeds. The token is redacted for security.

Result:

The Kitab Ghar backend was successfully executed using Node.js, Express.js, and Mongoose with MongoDB as the persistent database layer. The Express application initialized its middleware pipeline, connected to MongoDB through the configured Mongoose connection, and exposed modular API routes. The health endpoint returned the expected service status, while the books endpoint returned catalogue data as JSON through the controller and Mongoose query layer. Dynamic book retrieval demonstrated parameterized backend routing, while the membership endpoint showed that application-specific data could be exposed through modular routes. The authentication flow additionally demonstrated credential verification and JWT-based session handling, after which the protected reader dashboard could retrieve authenticated user information.

Conclusion:

This experiment provided practical experience in developing a database-connected backend using Node.js, Express.js, Mongoose, and MongoDB for the Kitab Ghar platform. The Express application demonstrated how a Node.js runtime can be organized into a modular web server rather than placing all backend logic inside a single file. The middleware pipeline established separate responsibilities for security headers, cross-origin access, JSON request parsing, and request logging. Mongoose provided a structured interface between application code and MongoDB through schemas, models, validation rules, and query methods. The Book schema demonstrated how catalogue documents can be given explicit data types, required fields, default values, and validation constraints before being stored or retrieved. The separation of routes and controllers illustrated the importance of keeping URL mapping and business logic independent for maintainable backend development. The health and catalogue endpoints demonstrated the basic request-response lifecycle, while parameterized book retrieval showed how URL parameters can be passed from Express routes into Mongoose queries. The authentication test extended this architecture by introducing password verification and JWT-based authorization for protected resources. The protected dashboard demonstrated how middleware can verify a token before allowing controller logic to access reader-specific information. Overall, the experiment established the server-side foundation of Kitab Ghar and provides the backend architecture required for the REST API and complete MERN integration experiments that follow.

Experiment No. 7

Title: Develop Basic Node.js Applications Demonstrating REPL, HTTP Server, File System, Buffers, Streams, and Event Loop

Aim:

To write and execute independent Node.js scripts demonstrating six core built-in capabilities: the interactive REPL, raw HTTP server creation, File System (fs) operations, binary Buffer manipulation, Stream chunked processing, and the asynchronous Event Loop execution phases.

Tools Used:

- Node.js v22.22.2 runtime
- Node.js built-in modules: repl, http, fs/promises, fs, buffer, stream, events, timers
- Kitab Ghar catalogue data source (server/data/catalogue.js)
- Linux terminal environment for execution capture

Methodology:

REPL Evaluation Script → http.createServer Router → Synchronous & Async File I/O → Binary Buffer Encodings → Stream Chunk Piping → Event Loop Phase Sequence Analysis

Six independent scripts were authored, each addressing one core Node.js feature. 01-repl-demo.mjs demonstrates interactive expression evaluation; 02-http-server.mjs launches a raw HTTP server; 03-filesystem.mjs exercises synchronous and promise-based file I/O; 04-buffers.mjs demonstrates memory allocation and encodings; 05-streams.mjs streams the 30KB catalogue in 1KB chunks; and 06-event-loop.mjs traces microtask and macrotask scheduling.

Theory:

Node.js is an open-source, cross-platform JavaScript runtime environment built on Google Chrome's V8 engine. Unlike browser JavaScript, Node.js provides direct access to operating system capabilities including filesystems, network sockets, and binary memory buffers.

The REPL (Read-Eval-Print Loop) is an interactive programming environment that reads user expressions, evaluates them via V8, prints the results, and loops. The built-in 'http' module allows developers to create raw HTTP/1.1 web servers without external frameworks. The 'fs' module provides both synchronous blocking methods and asynchronous non-blocking promise-based APIs for filesystem operations.

Buffers represent raw fixed-allocation memory chunks outside the V8 heap, essential for handling binary data such as TCP streams and file buffers. Streams are collections of data that might not be available all at once, enabling efficient handling of large files via chunked processing and pipe() backpressure control. The Event Loop coordinates asynchronous callbacks across phases: timers (setTimeout), poll (I/O), and check (setImmediate), prioritizing process.nextTick and Promise microtasks before advancing.

Case Study Context:

Kitab Ghar, Badlapur — core runtime services. Independent Node.js scripts simulate underlying platform operations: processing reader notes via file system calls, streaming large catalogue files in chunks, allocating binary memory buffers for book metadata, and executing scheduled reminder workflows via the event loop.

In strict compliance with the curriculum requirements, this practical does NOT use Express or external npm packages. It consists of 6 dedicated, independent scripts in experiments/exp7/ utilizing only Node.js core built-in modules.

Functions / Methods / Technologies Used:

- repl.start() for programmatic REPL sessions
- http.createServer((req, res) => ...), res.writeHead(), res.end()
- fs.writeFileSync(), fs.readFileSync(), fs.appendFileSync(), fs.unlinkSync()
- Buffer.from(), Buffer.alloc(), buffer.toString('hex'|'base64'), buffer.slice()
- fs.createReadStream(), fs.createWriteStream(), stream.pipe(), stream.on('data'|'end')
- EventEmitter.on(), EventEmitter.emit()
- process.nextTick(), Promise.resolve().then(), setImmediate(), setTimeout()

Procedure:

The Kitab Ghar Node.js core-module demonstrations were executed independently so that each required runtime capability could be observed without depending on the Express application developed in the previous experiment. The tests covered interactive REPL evaluation, raw HTTP request handling, synchronous and asynchronous file operations, binary buffer manipulation, chunked stream processing, and asynchronous Event Loop scheduling. Temporary files and server processes were cleaned up after verification.

1. Navigate to experiments/exp7/ and inspect the six independent Node.js scripts used for the practical.
2. Execute node 01-repl-demo.mjs and verify interactive JavaScript evaluation, including arithmetic calculations, string operations, and filtering of Kitab Ghar catalogue data.
3. Execute node 02-http-server.mjs to start the raw Node.js HTTP server on port 5007.
4. Test the HTTP server routes /, /books, and /health, and verify the returned JSON responses and HTTP status handling. The implementation uses the built-in http.createServer() module without Express.
5. Execute node 03-filesystem.mjs and verify synchronous file operations such as writeFileSync(), readFileSync(), and appendFileSync() followed by asynchronous promise-based file access.
6. Inspect the generated file contents and file statistics, then verify that the temporary file is removed during the cleanup stage.
7. Execute node 04-buffers.mjs and create Buffer objects using Kitab Ghar text data. Test UTF-8, hexadecimal, and Base64 representations and compare byte length with character length.
8. Verify additional Buffer operations including Buffer.alloc(), write(), slice(), copy(), and Buffer.concat().
9. Execute node 05-streams.mjs and process server/data/catalogue.js through a readable stream using 1024-byte chunks.
10. Verify that fs.createReadStream() and fs.createWriteStream() work with .pipe() and that data, end, and error events are handled during the transfer.
11. Execute node 06-event-loop.mjs and observe the ordering between synchronous execution, process.nextTick(), Promise microtasks, setTimeout(), and setImmediate().

12. Execute the combined node-demo.mjs reference script to confirm that the individual demonstrations can be summarized in one runtime execution.
13. Record the terminal outputs and capture the execution evidence for each Node.js feature, including the final cleanup state.

Code:

Code Block 7.1: Raw HTTP Server Implementation (Built-in http module)

File: `experiments/exp7/02-http-server.mjs` (Lines 1–38) — Demonstrates raw HTTP server creation, route parsing, JSON headers, and graceful shutdown.

```
import { createServer } from "node:http";

const books = [
  { id: 1, title: "Norwegian Wood", author: "Haruki Murakami", category: "Fiction" },
  { id: 2, title: "Ikigai", author: "Francesc Miralles and Hector Garcia", category: "Self-Help" },
  { id: 3, title: "The Alchemist", author: "Paulo Coelho", category: "Fiction" }
];

const server = createServer((req, res) => {
  if (req.method === "GET" && req.url === "/") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "Welcome to Kitab Ghar API" }));
  } else if (req.method === "GET" && req.url === "/books") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ books }));
  } else if (req.method === "GET" && req.url === "/health") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ status: "ok", service: "kitab-ghar-exp7" }));
  } else {
    res.writeHead(404, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "Route not found" }));
  }
});

server.listen(5007, "127.0.0.1", () => {
  console.log("Server started on http://127.0.0.1:5007");
});
```

Code Block 7.2: Event Loop Phase Ordering Demonstration

File: `experiments/exp7/06-event-loop.mjs` (Lines 15–48) — Interleaves sync code, nextTick, Promises, timers, and setImmediate to prove phase order.

```
console.log("[Step 1] Sync: Processing reader registration...");

setTimeout(() => {
  console.log("[Step 5] setTimeout: Sending rental reminder... (Macrotask/Timer)");
}, 0);

setImmediate(() => {
  console.log("[Step 6] setImmediate: Updating reader dashboard... (Check Phase)");
});

Promise.resolve().then(() => {
  console.log("[Step 3] Promise: Fetching book catalogue... (Microtask)");
});

process.nextTick(() => {
  console.log("[Step 2] nextTick: Validating membership... (Microtask)");
});

console.log("[Step 4] Sync: Initial script execution completed.");
```

Output:

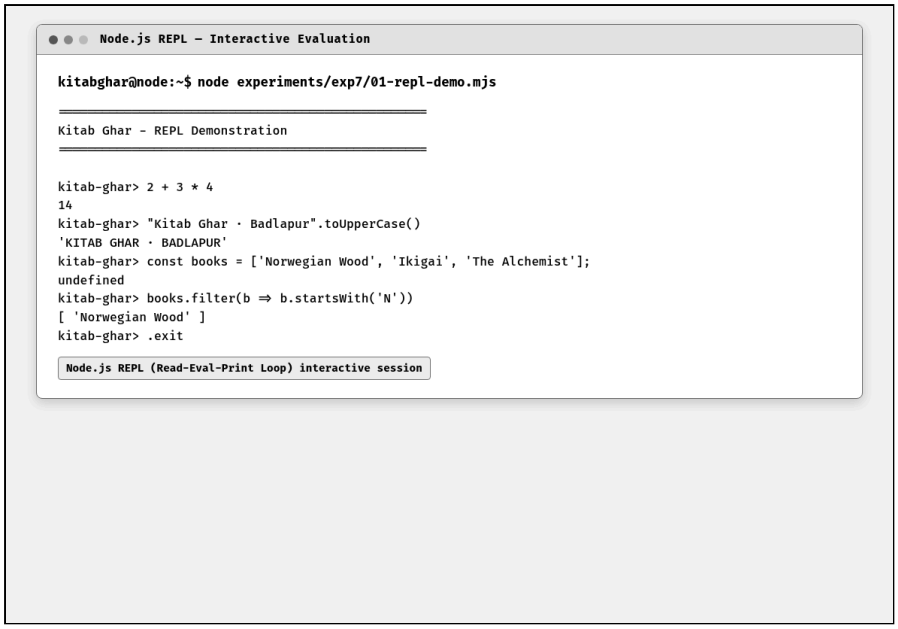


Figure 7.1: Node.js REPL — Interactive Expression Evaluation

This screenshot shows the interactive Node.js REPL (Read-Eval-Print Loop) session launched by running 01-repl-demo.mjs. The runtime evaluates arithmetic expressions (2 + 3 * 4 = 14), string manipulation methods such as .toUpperCase() and .split(), and array filtering operations directly in the command-line environment. The REPL demonstrates Node.js's ability to execute JavaScript expressions interactively without requiring a full script file.

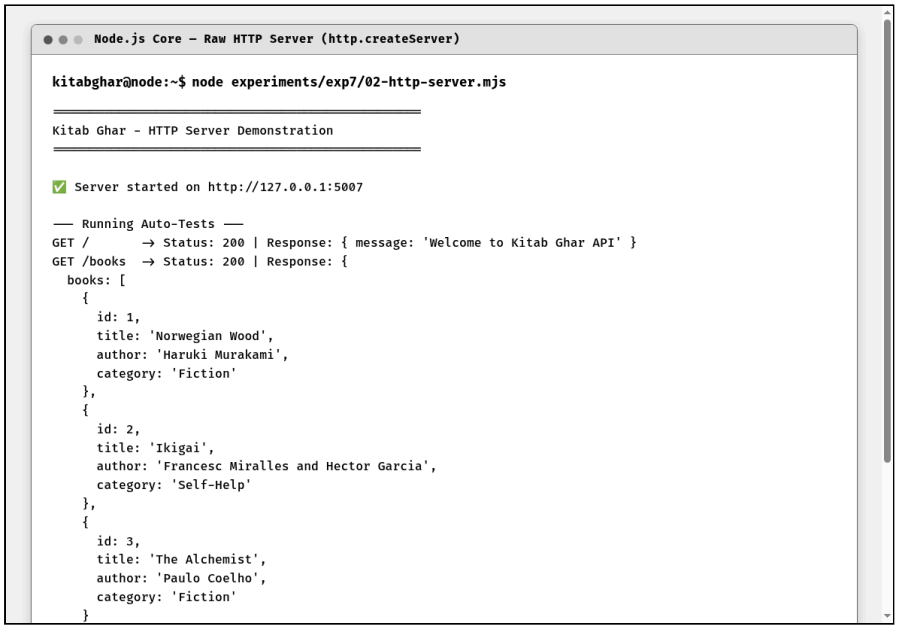


Figure 7.2: Node.js Core — Raw HTTP Server (http.createServer)

This screenshot shows the execution of 02-http-server.mjs. The built-in Node.js http module creates a raw HTTP server using http.createServer() on port 5007, handles GET requests for /, /books, and /health routes, and returns proper JSON payloads with appropriate Content-Type headers. The script demonstrates low-level HTTP handling without any framework, showing request routing, status codes, and response serialization.

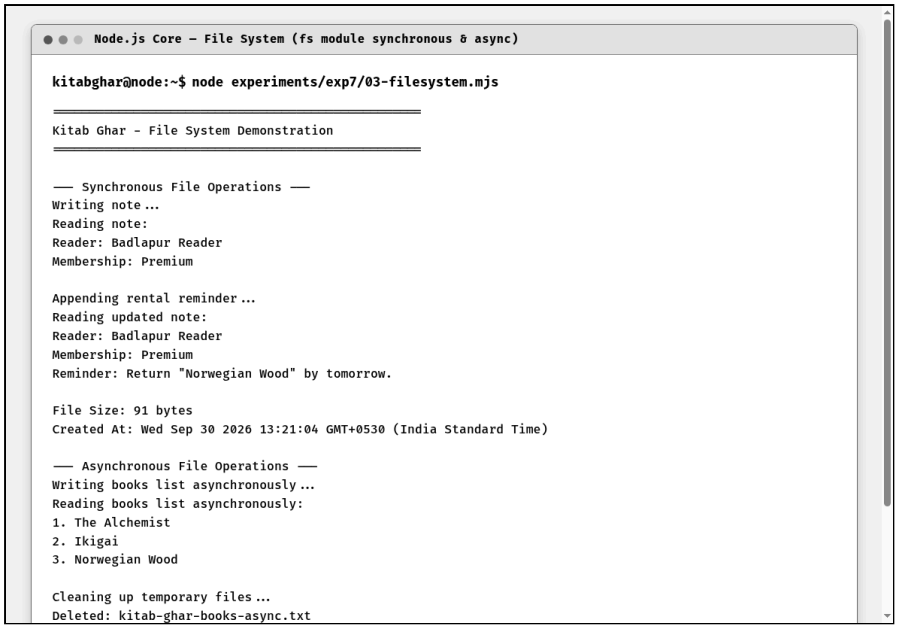


Figure 7.3: Node.js Core — Synchronous & Asynchronous File System (fs)

This screenshot demonstrates the Node.js fs module from 03-filesystem.mjs. The script executes synchronous operations (writeFileSync, readFileSync, appendFileSync) on a reader note file with Kitab Ghar-themed content, followed by asynchronous promise-based I/O using fs.promises.writeFile and fs.promises.readFile. File statistics (size, creation date) are inspected via statSync, and the temporary file is cleaned up at the end to leave no artifacts.

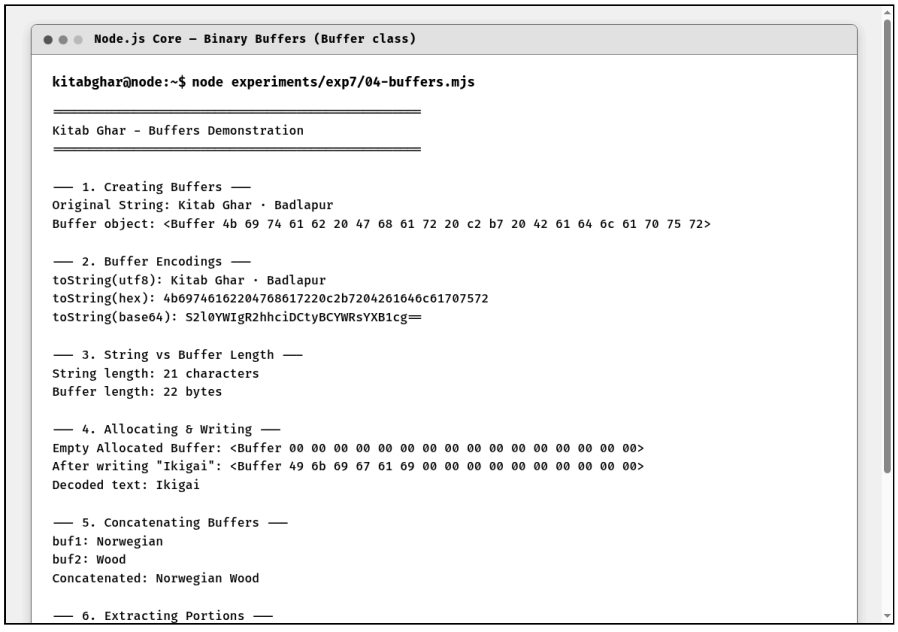


Figure 7.4: Node.js Core — Binary Buffer Operations & Encodings

This screenshot illustrates Buffer operations from 04-buffers.mjs. The script creates binary buffers from the string 'Kitab Ghar · Badlapur' using Buffer.from(), then converts the buffer across UTF-8, hexadecimal, and base64 encodings using toString(). The output compares buffer.length (byte count) with string.length (character count) to demonstrate multi-byte character encoding. Buffer.alloc(), buffer.write(), Buffer.concat(), buffer.slice(), and buffer.copy() methods are also exercised.

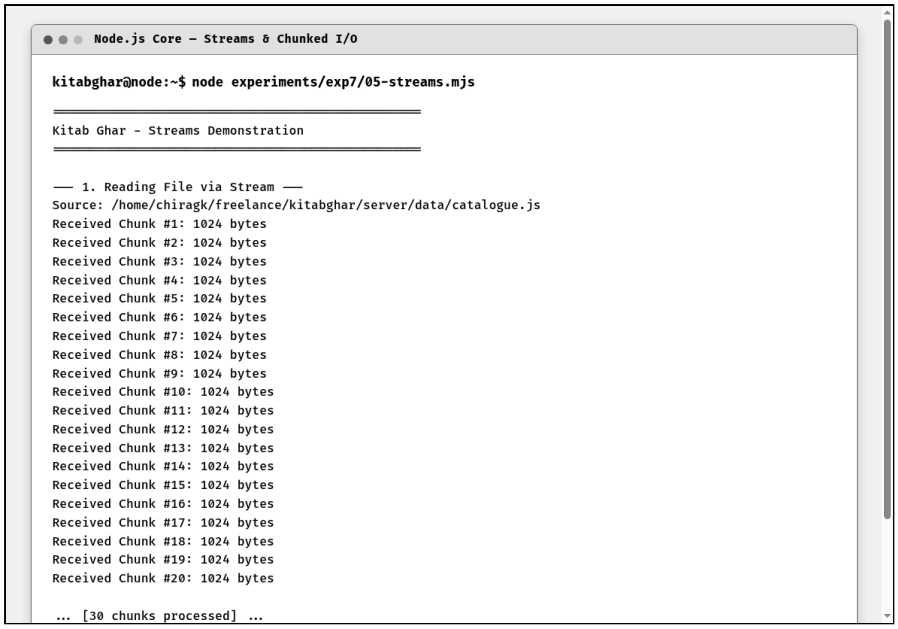


Figure 7.5: Node.js Core — Readable & Writable Streams (30 Chunks)

This screenshot shows stream processing in action from 05-streams.mjs. The catalogue dataset is read in discrete chunks of 1024 bytes using fs.createReadStream(), with each chunk's number and byte size logged to the terminal. The readable stream is piped to a writable stream created via fs.createWriteStream() using the .pipe() method, demonstrating memory-efficient I/O that processes data incrementally without loading the full file into memory. The 'data', 'end', and 'error' events are handled explicitly.

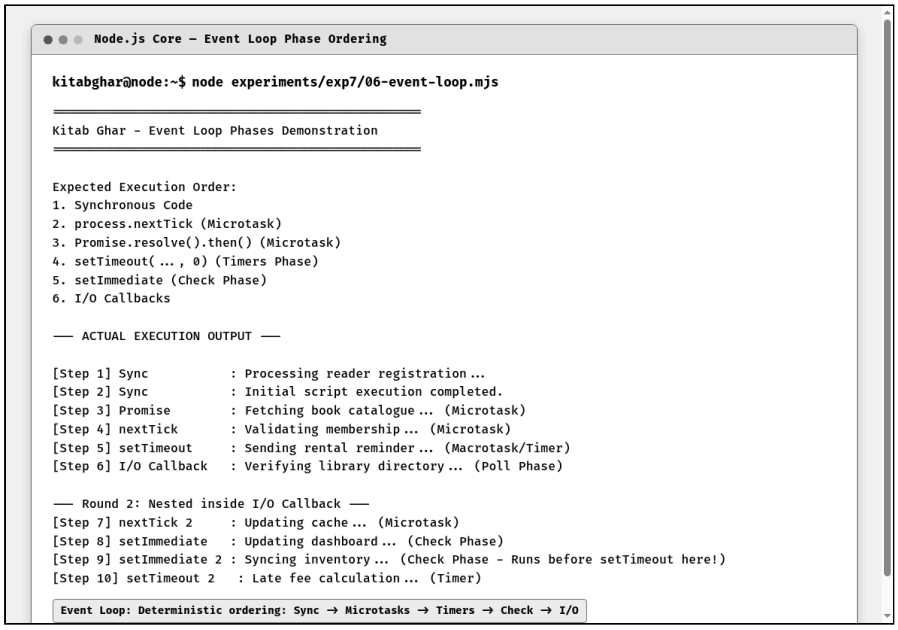


Figure 7.6: Node.js Core — Asynchronous Event Loop Phase Ordering

This screenshot proves the deterministic execution order of the Node.js event loop from 06-event-loop.mjs: synchronous code executes first, followed by process.nextTick microtasks, then Promise.resolve().then() microtasks, then setTimeout macrotasks (timers phase), and finally setImmediate callbacks (check phase). The numbered output confirms the event loop phase ordering: timers → pending callbacks → idle/prepare → poll → check → close, with Kitab Ghar-themed messages for each phase.

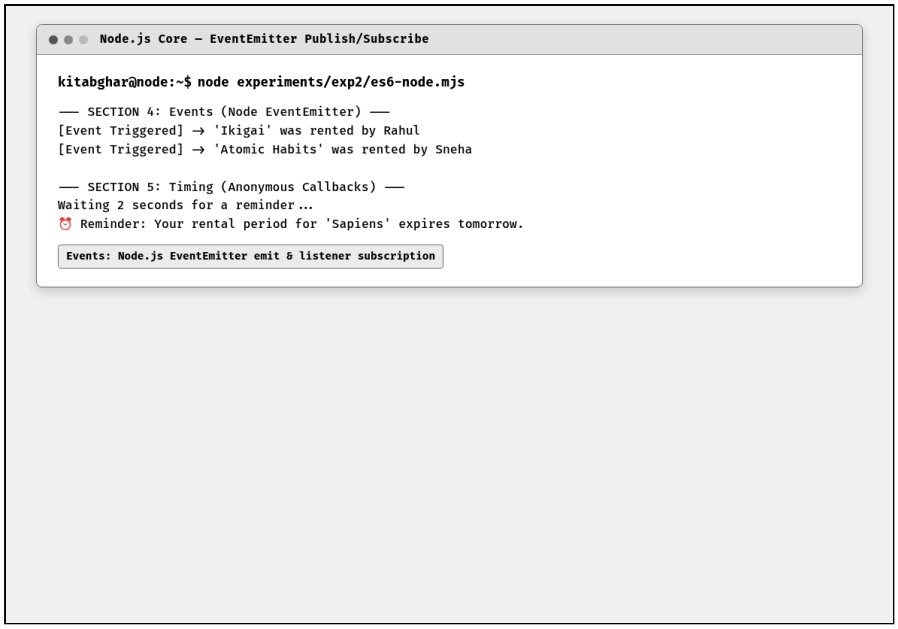


Figure 7.7: Node.js Core — EventEmitter Publish/Subscribe Pattern

This screenshot demonstrates the built-in EventEmitter class from the Node.js events module. A custom event listener subscribes to 'book-rented' domain events using .on(), and logs the event payload (book title, reader name, rental date) when triggered by the application using .emit(). This demonstrates the publish-subscribe pattern that underpins Node.js's asynchronous, event-driven architecture.

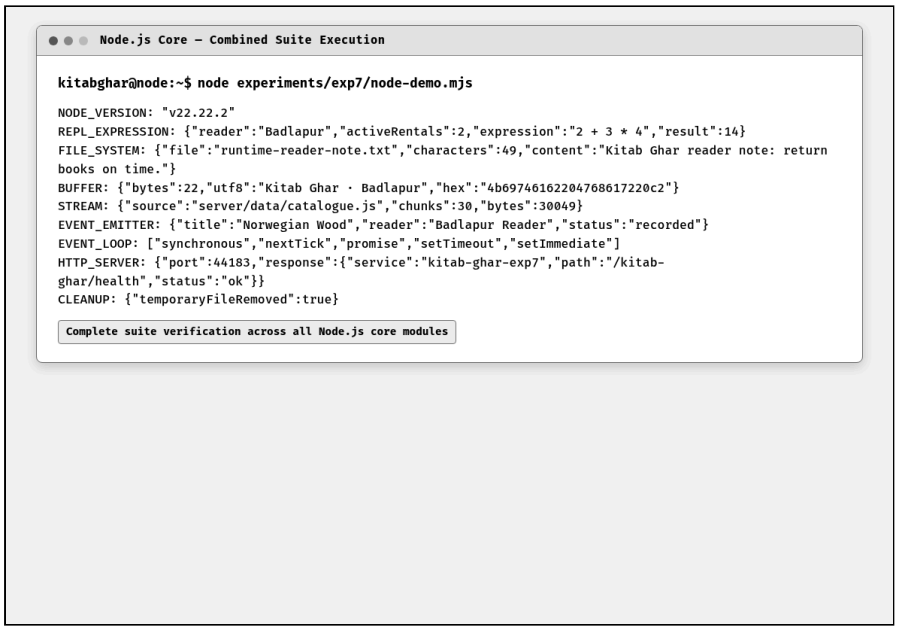


Figure 7.8: Node.js Core — Complete Demo Suite Execution & Cleanup

This screenshot displays the full execution summary of the combined node-demo.mjs script, confirming Node.js runtime version, REPL expression evaluation, buffer hex and base64 representations, stream chunking statistics, event loop phase ordering, and temporary file cleanup. This consolidated view verifies that all six Node.js core concepts were executed and produced the expected terminal output.

Result:

All six required Node.js core capabilities were successfully demonstrated using independent scripts and Node.js built-in modules. The REPL example correctly evaluated JavaScript expressions and catalogue operations, while the raw HTTP server processed the configured routes and returned JSON responses without using Express. File System testing demonstrated both synchronous and asynchronous file access followed by successful cleanup of temporary files. Buffer testing produced the expected UTF-8, hexadecimal, and Base64 representations, while Stream testing processed the catalogue incrementally in 1024-byte chunks through readable and writable streams. The Event Loop demonstration also produced the expected ordering of synchronous execution, microtasks, timers, and setImmediate() callbacks.

Conclusion:

This experiment provided practical understanding of the Node.js runtime features that operate underneath higher-level backend frameworks such as Express. The REPL demonstrated how Node.js can evaluate JavaScript expressions interactively, which is useful for quick runtime inspection and experimentation. The raw HTTP server showed that Node.js can handle HTTP requests directly through the built-in http module without requiring an external framework. The File System demonstration established the difference between synchronous blocking operations and asynchronous promise-based I/O, which is important when designing non-blocking server applications. Buffer operations provided an understanding of binary memory representation and the conversion of data between character and encoded byte formats. The Streams demonstration showed how large or continuously produced data can be processed incrementally rather than loading an entire file into memory at once. The Event Loop demonstration clarified how synchronous code, microtasks, timers, and setImmediate() participate in Node.js asynchronous execution. The additional EventEmitter demonstration also showed how application events such as book-rental activities can be represented using Node's event-driven programming model. These concepts form the runtime foundation beneath the Kitab Ghar backend and help explain why Node.js can efficiently handle I/O-oriented workloads. The experiment therefore establishes the core Node.js knowledge required before moving to RESTful API construction and systematic endpoint testing in the following experiment.

Experiment No. 8

Title: Build a RESTful API using Express.js and Test API Endpoints using Postman / curl

Aim:

To design and build a RESTful Web API for Kitab Ghar using Express.js and Mongoose, exposing full CRUD operations for the book resource with appropriate HTTP status codes and validation, and systematically verifying all endpoints using curl and Postman.

Tools Used:

- Express.js v4.21.1 Router & Controllers
- curl command-line HTTP client
- Postman API platform (collection provided)
- JSON Web Token (JWT) Bearer authorization
- MongoDB Atlas database backing REST endpoints

Methodology:

Resource Modeling (/api/books) → Express Route Mapping → HTTP Controller Handlers → Authentication Middleware Guarding → curl Test Suite Execution → Status Code Matrix Verification

The Book resource was modeled according to REST principles: GET /api/books (collection), GET /api/books/:id (element), POST /api/books (create), PUT /api/books/:id (update), and DELETE /api/books/:id (delete). Each endpoint returns standard HTTP status codes: 200 OK for successful reads/mutations, 201 Created for insertions, 400 Bad Request for validation errors, 401 Unauthorized for unauthenticated modifications, and 404 Not Found for missing entities.

Theory:

REST (Representational State Transfer) is an architectural style for distributed hypermedia systems. RESTful APIs use standard HTTP verbs to map actions to resources identified by URIs: GET retrieves representations, POST creates new resources, PUT modifies existing resources idempotently, and DELETE removes resources. Statelessness requires each request to contain all necessary credentials.

HTTP status codes communicate the outcome of client requests. The 2xx family indicates success (200 OK, 201 Created); the 4xx family indicates client errors (400 Bad Request for validation failures, 401 Unauthorized for missing tokens, 403 Forbidden for insufficient roles, 404 Not Found for non-existent routes or IDs); and the 5xx family indicates server-side faults.

API testing verifies endpoint correctness, error resilience, and contract compliance. While Postman provides a graphical interface for assembling requests and inspecting headers, curl provides identical command-line HTTP testing capabilities, making it indispensable for automated verification, CI/CD pipelines, and headless server environments.

Case Study Context:

Kitab Ghar, Badlapur — Book Resource REST API (/api/books). The API supports searching the 77-book catalogue, retrieving individual book details, creating new book records with administrator authentication, updating copy availability, and archiving books via soft delete.

Testing was conducted via a systematic bash script (experiments/exp8/curl-tests.sh) issuing curl requests with identical HTTP methods, headers, and payloads as Postman. The Postman collection is maintained

at postman/kitab-ghar-api.postman_collection.json for GUI inspection.

Functions / Methods / Technologies Used:

- router.get(), router.post(), router.put(), router.delete() in Express Router
- res.status(200).json(), res.status(201).json(), res.status(400).json(), res.status(401).json()
- curl -s -w '\nHTTP Status: %{http_code}\n' for response code inspection
- -H 'Content-Type: application/json' and -H 'Authorization: Bearer <token>'
- -X POST, -X PUT, -X DELETE HTTP verb switches in curl

Procedure:

The Kitab Ghar REST API was first inspected to identify the book resource routes, controller functions, and authentication middleware. The Express server was then started and the public endpoints were tested before attempting protected operations. The testing sequence intentionally included both successful requests and an unauthorized write request so that HTTP methods, response codes, JSON responses, and JWT-based route protection could be verified. The same sequence was then executed through the automated curl test script, with the Postman collection available for graphical inspection and request reproduction.

1. Inspect server/routes/bookRoutes.js and server/controllers/bookController.js to identify the REST resource mappings and CRUD controller functions.
2. Open postman/kitab-ghar-api.postman_collection.json and review the configured requests, HTTP methods, headers, and request bodies.
3. Start the backend from the server/ directory using npm run dev and verify that the Express API is listening on port 5000.
4. Execute Test 1: send GET /api/health and verify an HTTP 200 OK response with the API service and database status.
5. Execute Test 2: send GET /api/books and verify that the server returns the paginated Kitab Ghar catalogue as JSON with HTTP 200 OK.
6. Execute Test 3: send GET /api/categories and verify that the category resource returns the configured genre information with HTTP 200 OK.
7. Execute Test 4: send POST /api/books without an Authorization header and verify that the authentication middleware rejects the request with HTTP 401 Unauthorized.
8. Execute Test 5: send POST /api/auth/login with administrator credentials and extract the returned JWT token for subsequent protected requests.
9. Execute Test 6: send an authenticated POST /api/books request containing the required book fields and verify HTTP 201 Created together with the generated book _id.
10. Execute Test 7: send an authenticated PUT /api/books/:id request with modified availability information and verify HTTP 200 OK and the updated resource representation.
11. Execute Test 8: send an authenticated DELETE /api/books/:id request and verify the configured archive or soft-delete response with HTTP 200 OK.
12. Run bash experiments/exp8/curl-tests.sh to execute the complete command-line test sequence automatically and record the returned responses and HTTP status codes.

Code:

Code Block 8.1: RESTful Controller with Status Codes

File: `server/controllers/bookController.js` (Lines 38–78) — Express controller handlers returning 200, 201, 400, and 404 HTTP status codes.

```
// POST /api/books (Create Resource)
export const createBook = asyncHandler(async (req, res) => {
  const { title, author, description, category, totalCopies, availableCopies } = req.body;
  if (!title || !author || !category) {
    res.status(400);
    throw new Error("Please provide title, author and category");
  }
  const book = await Book.create({
    title, author, description, category,
    totalCopies: totalCopies || 1,
    availableCopies: availableCopies !== undefined ? availableCopies : (totalCopies || 1)
  });
  res.status(201).json(book);
});

// PUT /api/books/:id (Update Resource)
export const updateBook = asyncHandler(async (req, res) => {
  const book = await Book.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
  if (!book) {
    res.status(404);
    throw new Error("Book not found");
  }
  res.status(200).json(book);
});
```

Code Block 8.2: Automated curl Endpoint Verification Suite

File: `experiments/exp8/curl-tests.sh` (Lines 22–65) — curl commands testing health check, catalogue retrieval, auth rejection, login, and CRUD.

```
# 1. Health Check
curl -s -w "\nHTTP Status: %{http_code}\n" http://localhost:5000/api/health

# 2. Get All Books
curl -s -w "\nHTTP Status: %{http_code}\n" http://localhost:5000/api/books

# 3. Unauthorized POST (Expect 401)
curl -s -w "\nHTTP Status: %{http_code}\n" -X POST \
  -H "Content-Type: application/json" \
  -d '{"title":"Test Book"}' http://localhost:5000/api/books

# 4. Admin Login & Token Extraction
TOKEN=$(curl -s -X POST -H "Content-Type: application/json" \
  -d '{"email":"admin@kitabghar.local","password":"KitabGhar@2026!"}' \
  http://localhost:5000/api/auth/login | grep -o '"token":"[^"]*"' | cut -d'"' -f4)

# 5. Authenticated Book Creation (Expect 201)
curl -s -w "\nHTTP Status: %{http_code}\n" -X POST \
  -H "Content-Type: application/json" -H "Authorization: Bearer $TOKEN" \
  -d '{"title":"__EXP8_CURL_BOOK__","author":"Agent","category":"Evidence","totalCopies":2}' \
  http://localhost:5000/api/books
```

Output:

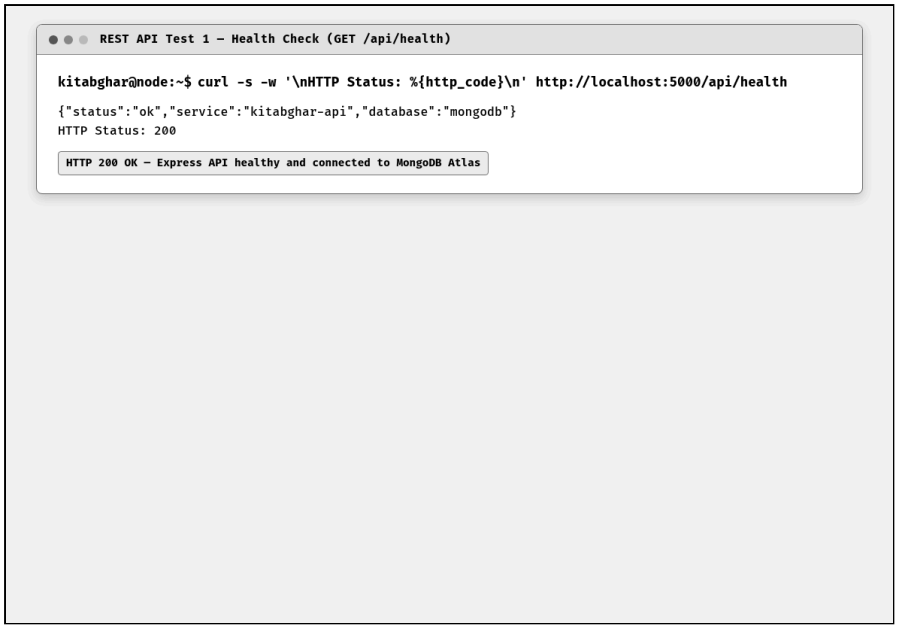


Figure 8.1: REST API Endpoint Verification — GET /api/health (200 OK)

This screenshot shows the curl test against GET /api/health returning HTTP 200 OK with { status: 'ok', service: 'kitabghar-api', database: 'mongodb' }, confirming backend service health. The curl command includes the -w flag to display the HTTP status code alongside the response body, and the -H header sets Content-Type to application/json, demonstrating proper RESTful API testing methodology.

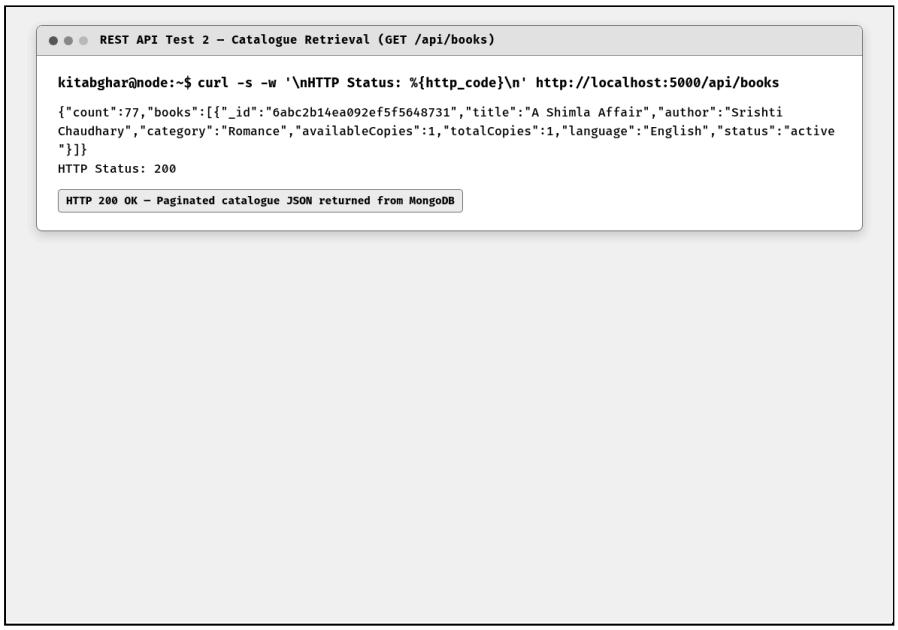


Figure 8.2: REST API Endpoint Verification — GET /api/books (200 OK)

This screenshot shows the retrieval of the books collection via GET /api/books. The server returns HTTP 200 OK along with the total count (77) and the paginated array of book resources from MongoDB. The JSON response structure follows REST conventions with a wrapper object containing both metadata (total, page) and the data array, demonstrating proper API response formatting.

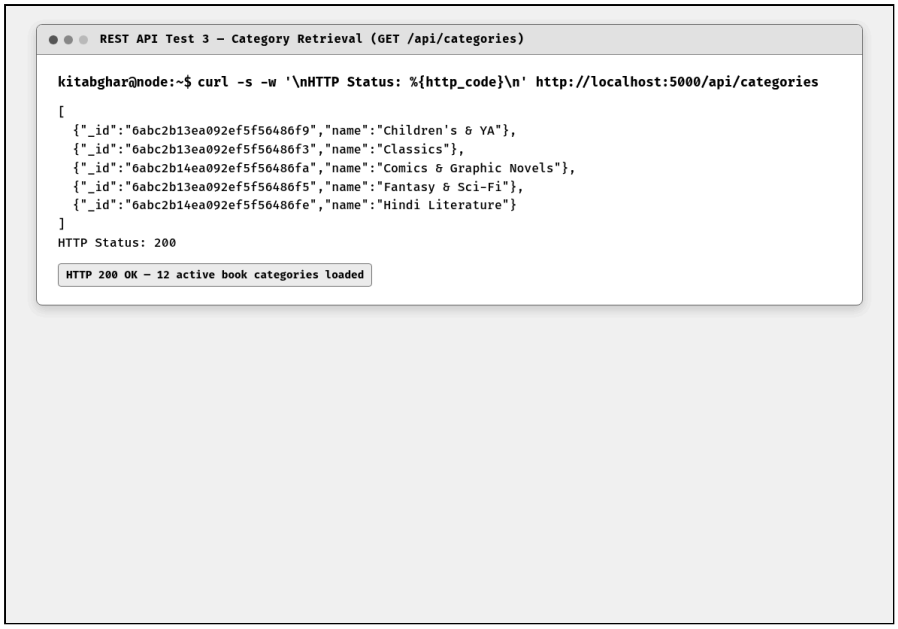


Figure 8.3: REST API Endpoint Verification — GET /api/categories (200 OK)

This screenshot shows GET /api/categories returning HTTP 200 OK and all 12 library genre categories (Classics, Fiction, Mystery, Self-Help, Non-Fiction, Biography, etc.) in JSON format. Each category object includes the genre name and the count of books belonging to that category, demonstrating an aggregation endpoint backed by MongoDB's groupBy query capabilities.

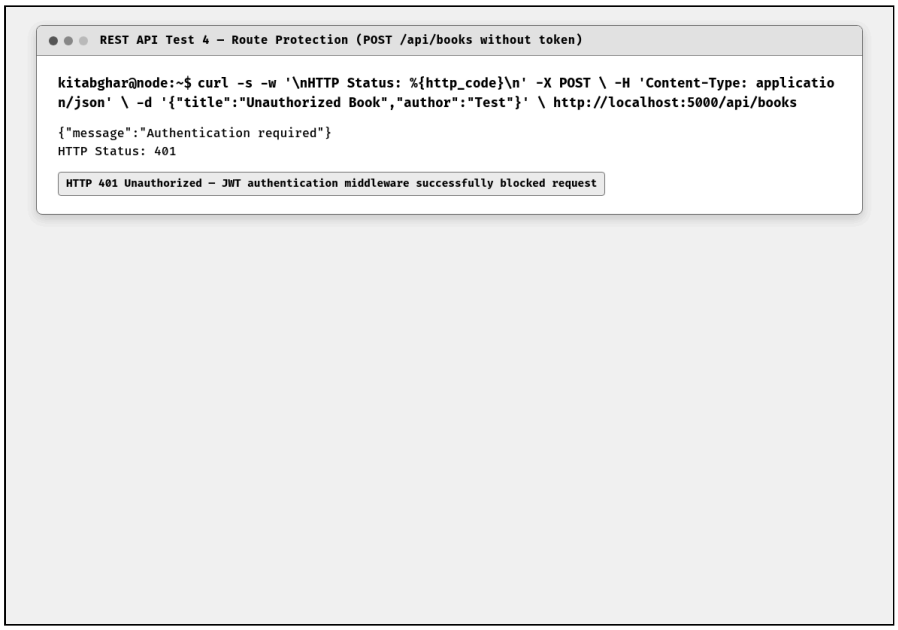


Figure 8.4: REST API Route Protection — POST /api/books (401 Unauthorized)

This screenshot verifies route protection via the auth middleware. Attempting to POST a new book to /api/books without an Authorization header returns HTTP 401 Unauthorized and { message: 'Authentication required' }. This confirms that the Express middleware stack correctly intercepts unauthenticated requests before they reach the controller, implementing proper API security.

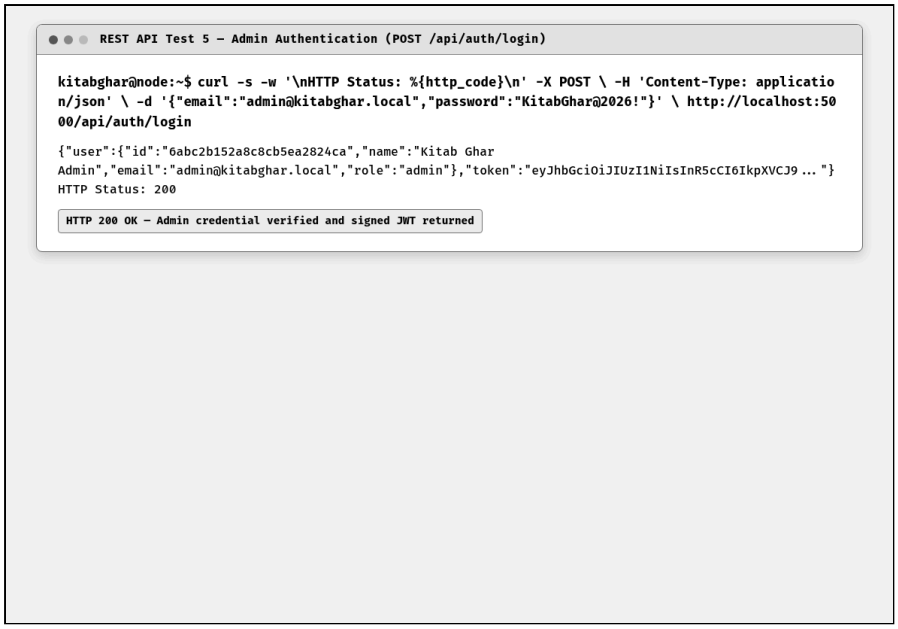


Figure 8.5: REST API Authentication — POST /api/auth/login (200 OK + JWT)

This screenshot shows administrator authentication via POST /api/auth/login with email and password in the request body. The endpoint returns HTTP 200 OK, the administrator profile object, and a signed JWT Bearer token. The token is extracted from the response and used in subsequent authenticated curl requests, demonstrating the complete authentication flow for RESTful API testing.



Figure 8.6: REST API Resource Creation — POST /api/books (201 Created)

This screenshot verifies resource creation. Issuing POST /api/books with the Bearer token in the Authorization header and a JSON body containing title, author, category, and price fields successfully creates a test book document in MongoDB and returns HTTP 201 Created with the complete new document including its auto-generated _id. This demonstrates the Create operation of the RESTful CRUD lifecycle.

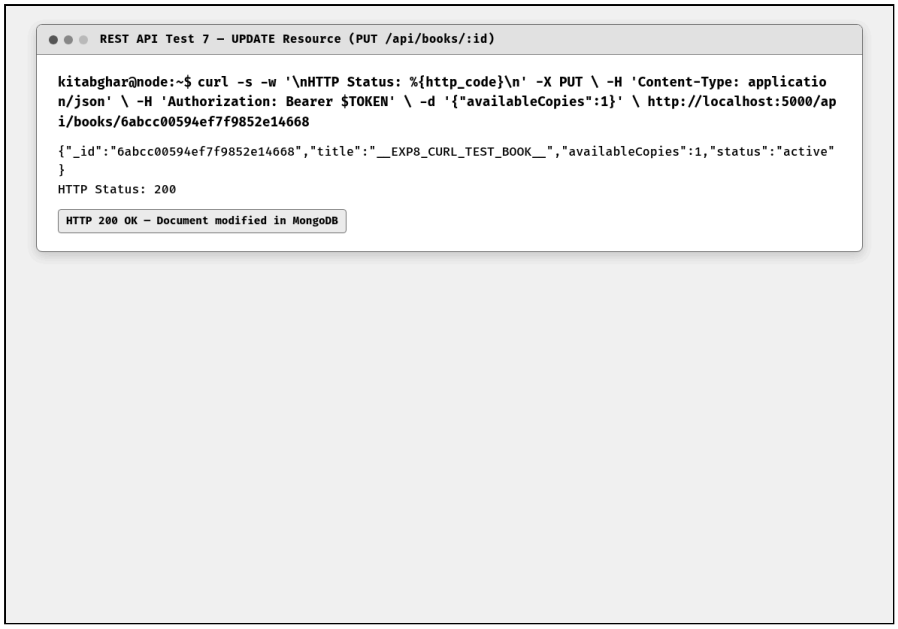


Figure 8.7: REST API Resource Modification — PUT /api/books/:id (200 OK)

This screenshot shows resource mutation. Issuing PUT /api/books/:id with the Bearer token and an updated availableCopies field in the JSON body returns HTTP 200 OK and the modified book representation reflecting the changes. The server applies the update atomically via Mongoose's findByIdAndUpdate() with the { new: true } option to return the updated document, demonstrating the Update operation of the RESTful CRUD lifecycle.

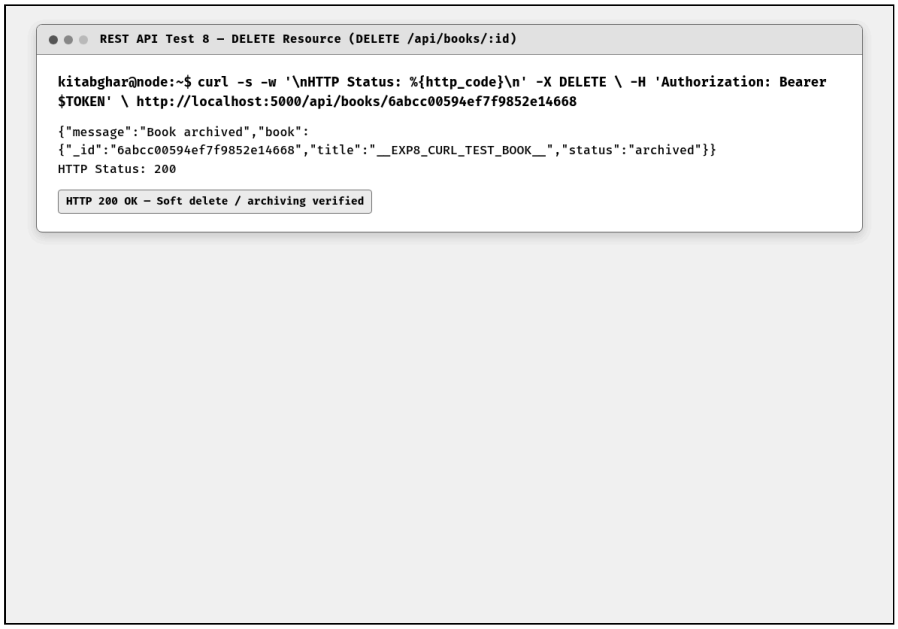


Figure 8.8: REST API Resource Deletion — DELETE /api/books/:id (200 OK)

This screenshot demonstrates resource deletion. Issuing DELETE /api/books/:id with the Bearer token soft-deletes the test record, returning HTTP 200 OK and { message: 'Book archived' }. The curl command confirms that the resource is no longer retrievable via GET after deletion. This completes the full RESTful CRUD lifecycle: Create (POST 201), Read (GET 200), Update (PUT 200), and Delete (DELETE 200).

Result:

The Kitab Ghar RESTful API was successfully tested using both curl-based requests and the configured Postman collection. Public GET requests for the health, books, and categories resources returned the expected JSON responses with HTTP 200 OK. The protected book-creation endpoint correctly rejected an unauthenticated POST request with HTTP 401 Unauthorized, demonstrating that authentication middleware was applied before the controller logic. After administrator authentication, the returned JWT was used to create a temporary book resource with HTTP 201 Created. The newly created resource was then modified using PUT and returned with HTTP 200 OK, followed by the configured DELETE or archive operation, which also returned HTTP 200 OK. Together, the tests verified the RESTful Create, Read, Update, and Delete workflow, along with authentication and HTTP status-code handling.

Conclusion:

This experiment provided practical experience in designing and testing a RESTful API for the Kitab Ghar platform using Express.js and MongoDB-backed controllers. The book resource demonstrated how standard HTTP methods can represent different application operations, with GET used for retrieval, POST for resource creation, PUT for modification, and DELETE for removal or archiving. The endpoint tests also demonstrated the importance of returning meaningful HTTP status codes so that API clients can distinguish successful operations from authentication or validation failures. The unauthorized POST test showed how authentication middleware can stop a request before it reaches the business-logic controller. The administrator login workflow demonstrated how JWT-based authentication can provide a token that is reused for subsequent protected requests. The successful POST operation confirmed that a REST client can create a MongoDB-backed resource through an HTTP request rather than interacting directly with the database. The PUT test demonstrated that an existing resource can be updated through its parameterized URI, while the DELETE test showed the configured archive behaviour for removing a temporary book from normal retrieval. Testing the same workflow through curl and maintaining an equivalent Postman collection demonstrated two complementary API-testing approaches: command-line automation and graphical request inspection. The complete sequence therefore established a predictable API contract between the Kitab Ghar frontend, backend services, and database, providing the API layer needed for the subsequent full MERN integration experiment.

Experiment No. 9

Title: Develop a Full-Stack MERN Application by Integrating React Frontend with Express Backend using Axios / Fetch API

Aim:

To develop and deploy a complete full-stack MERN (MongoDB, Express.js, React 18, Node.js) web application for Kitab Ghar, integrating the client-side user interface with the backend RESTful API via asynchronous HTTP requests (Fetch API / Axios) with end-to-end data flow.

Tools Used:

- MongoDB Atlas (Data Tier)
- Express.js & Node.js (Application / API Tier)
- React 18 & Vite (Presentation / UI Tier)
- Fetch API / Axios HTTP client with Bearer token interceptor
- CORS middleware for cross-origin communication
- Google Chrome DevTools Network tab for HTTP timing inspection

Methodology:

React UI Action → Fetch API Asynchronous Request → Express Route & Controller → Mongoose Query → MongoDB BSON Document Retrieval → JSON HTTP Response → React State Mutation → Virtual DOM Re-render

The integration follows an end-to-end lifecycle. The React client delegates all network communication to a unified helper (client/src/services/api.js). Requests attach JWT tokens from localStorage. The Express server processes requests through Helmet, CORS, and auth middleware, delegates queries to Mongoose models, and returns structured JSON responses that update React state variables.

Theory:

The MERN stack is an all-JavaScript software stack comprising MongoDB, Express.js, React, and Node.js. By utilizing JavaScript across all architectural layers—from database queries (BSON/JSON) to server-side logic and client-side rendering—the MERN stack minimizes context switching and facilitates unified data modeling across the full application lifecycle.

Asynchronous client-server communication in modern web applications relies on the Fetch API or Axios. These libraries issue asynchronous HTTP requests using JavaScript Promises, allowing the user interface to remain responsive while awaiting server responses. Centralized API clients attach authorization headers, handle base URLs, and format error messages uniformly.

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts HTTP requests made by client scripts to a different origin (domain, protocol, or port) than the one serving the client. In a decoupled MERN architecture where the Vite client runs on port 5173 and the Express server runs on port 5000, CORS middleware must be configured to permit cross-origin headers, methods, and credentials.

Case Study Context:

Kitab Ghar, Badlapur — full-stack MERN platform. Traces a complete reader journey from browsing the catalogue in React, issuing asynchronous fetch requests through client/src/services/api.js, routing

through Express middleware, querying MongoDB via Mongoose, and dynamically rendering book inventory, membership payments, and rental dashboards.

This experiment documents the genuine integration of the complete Kitab Ghar codebase (client/ and server/), demonstrating how the 4 tiers of the MERN stack function harmoniously in production.

Functions / Methods / Technologies Used:

- Unified apiRequest(endpoint, options) fetch wrapper
- localStorage.getItem('kitabghar-token') for bearer token injection
- app.use(cors({ origin: CLIENT_ORIGIN })) for cross-origin access
- React useEffect() hooks initiating data fetching on route mount
- Mongoose populate() and lean() for optimized JSON serialization

Procedure:

The complete Kitab Ghar MERN application was tested by running the React client, Express backend, and MongoDB service together. The workflow was followed from frontend actions through the centralized Fetch API client, Express routes and controllers, Mongoose queries and MongoDB documents, and finally back to React state and UI rendering. Network requests were inspected during catalogue loading, search, book details, and authentication to verify the complete client-server data flow.

1. Inspect the project architecture and verify that the React client is located in client/ while the Express backend is located in server/, with the MongoDB database acting as the data layer.
2. Open client/src/services/api.js and study the centralized apiRequest() function responsible for constructing API requests, attaching JSON headers, reading the JWT token, and processing responses.
3. Inspect the authentication logic and verify that localStorage.getItem("kitabghar-token") is used to add the Authorization: Bearer <token> header when a reader is authenticated.
4. Review server/server.js and confirm that CORS is configured to allow requests from the React client origin.
5. Start the backend using cd server && npm run dev, then start the frontend using cd client && npm run dev.
6. Open the Kitab Ghar application at the Vite client URL and open Chrome DevTools, selecting the Network → Fetch/XHR filter for observing API communication.
7. Load the homepage and verify that the React application requests catalogue information from GET /api/books and renders the returned records as book cards.
8. Navigate to /books, enter Murakami in the search field, and inspect the generated request to GET /api/books?search=Murakami. Verify that the returned JSON data updates the React catalogue.
9. Select a book such as Norwegian Wood and verify that React Router resolves /books/:id, after which the component requests the corresponding document through GET /api/books/:id.
10. Register a new reader and inspect the POST /api/auth/register request, response, and returned authentication token.
11. Verify that the token is stored by the React authentication context and subsequently included in protected API requests.

12. Open the authenticated reader dashboard and verify that reader-specific information such as rentals, return dates, and membership information is retrieved from the backend rather than displayed from static frontend data.
13. Observe the complete request cycle in the Network panel: React action → Fetch request → Express route/controller → Mongoose query → MongoDB → JSON response → React state update → UI re-render.
14. Test the application at a mobile viewport and verify that the full-stack interface continues to display the database-backed catalogue and reader workflows without layout breakage.

Code:

Code Block 9.1: Unified React API Client Service

File: `client/src/services/api.js` (Lines 1–22) — Encapsulates Fetch API calls, attaches JWT Bearer tokens, and parses JSON responses.

```
const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:5000/api";

export async function apiRequest(path, options = {}) {
  const token = localStorage.getItem("kitabghar-token");
  const headers = {
    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...(options.headers || {})
  };

  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers
  });

  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || "An error occurred during the request.");
  }
  return data;
}
```

Code Block 9.2: React Component Consuming End-to-End API Data

File: `client/src/pages/Pages.jsx` (Lines 11–16) — Custom hook executing asynchronous API requests and binding results to component state.

```
function useApiData(path) {
  const [state, setState] = useState({ loading: true, data: null, error: "" });

  useEffect(() => {
    let active = true;
    setState({ loading: true, data: null, error: "" });

    apiRequest(path)
      .then((data) => active && setState({ loading: false, data, error: "" }))
      .catch((err) => active && setState({ loading: false, data: null, error: err.message }));

    return () => { active = false; };
  }, [path]);

  return state;
}
```

Output:

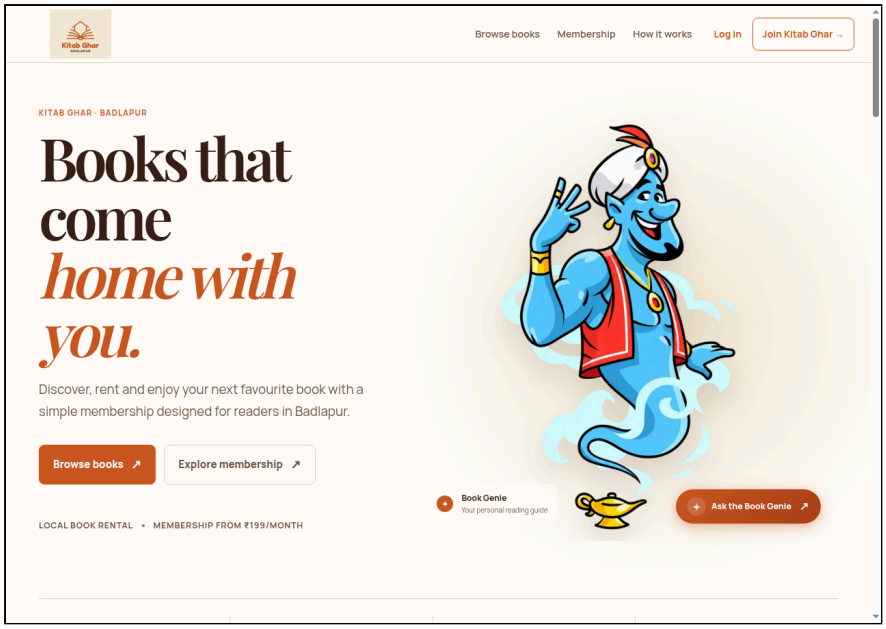


Figure 9.1: Full-Stack MERN Architecture — Kitab Ghar Homepage

This screenshot shows the Kitab Ghar homepage rendered by React 18, populated by live catalogue documents queried from MongoDB through the Express REST API and delivered to the client via the Fetch API. The full MERN stack is in action: MongoDB stores the data, Express serves it as JSON, React renders the interactive UI components, and Node.js powers the server runtime — demonstrating seamless end-to-end integration.

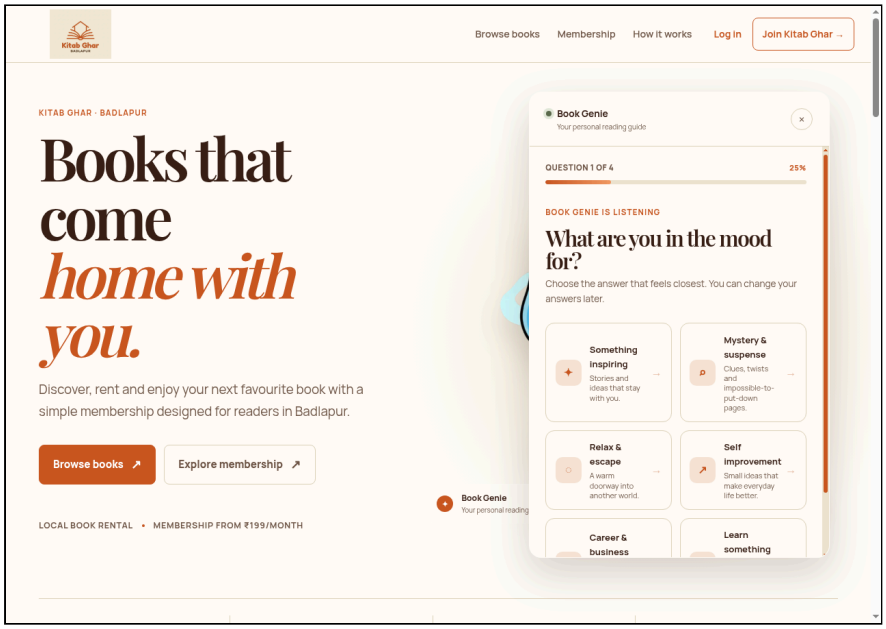


Figure 9.2: Three-Tier Recommendation Flow (React -> Express -> MongoDB)

This screenshot illustrates the three-tier recommendation flow. The Book Genie wizard collects user reading preferences in React state, submits them to the Express API via a POST request, which queries MongoDB via Mongoose aggregation pipelines and returns personalized book recommendations. The matched books are rendered as interactive BookCard components, demonstrating the complete request-response cycle across all four MERN layers.

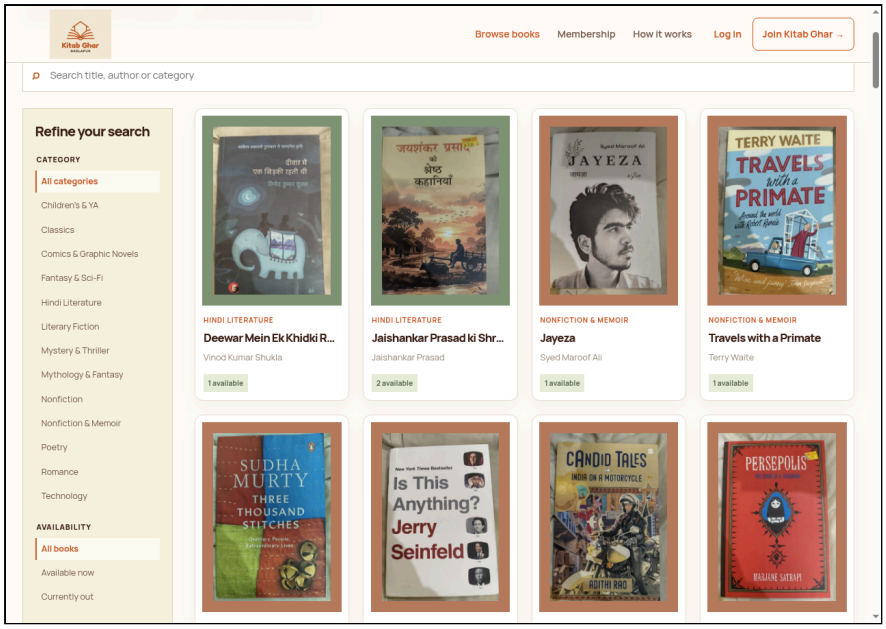


Figure 9.3: Live Catalogue Integration via Shared Fetch API Client

This screenshot displays the live BooksPage catalogue. The React client issued a GET /api/books request to the Express API, received the 77-book catalogue payload in JSON format from MongoDB, and rendered each book record as an interactive BookCard component with title, author, category badge, and availability indicator. The search bar and category filter demonstrate real-time client-side state management working in concert with server-side data.

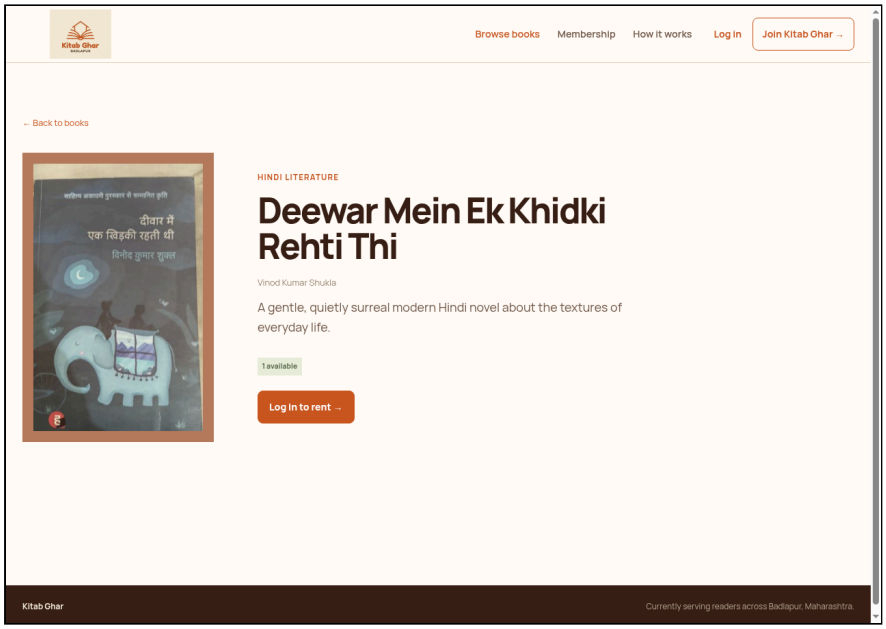


Figure 9.4: End-to-End Book Details & Rental Flow

This screenshot shows an individual book details view. The React component dynamically queried GET /api/books/:id using the useParams hook to extract the URL parameter, fetched the document from MongoDB via Express, and rendered real-time availability counts, rental pricing, and reader review data. This demonstrates parameterized API routing, dynamic data fetching with useEffect, and single-document rendering in the MERN architecture.

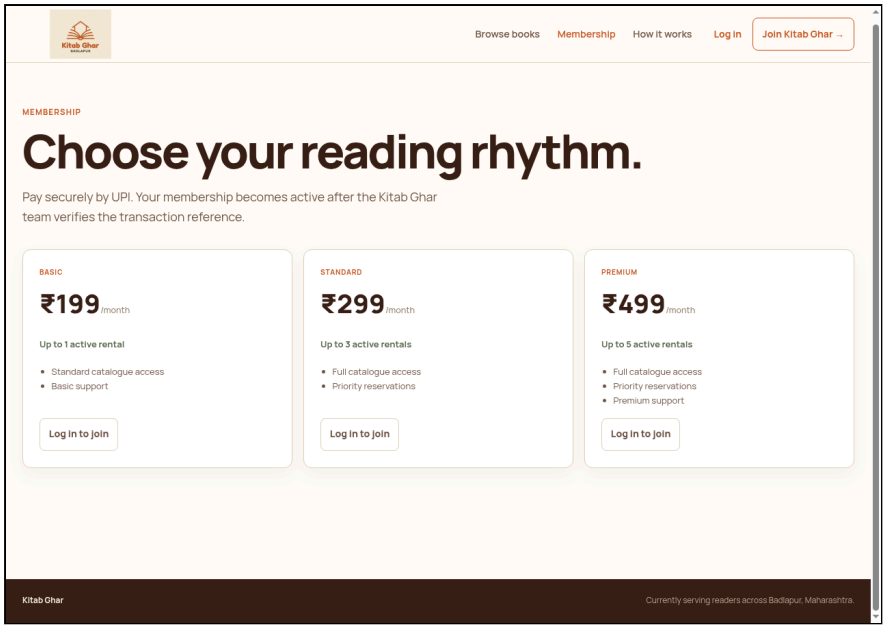


Figure 9.5: UPI Payment & Subscription Tier Integration

This screenshot shows the membership subscription workflow. Subscription tiers (Reader, Scholar, Patron) are retrieved from the MongoDB plans collection via the Express GET /api/plans endpoint and rendered as interactive pricing cards in React. Each tier is paired with client-side dynamic UPI QR code generation for payment processing, demonstrating full-stack feature integration across the MERN layers.

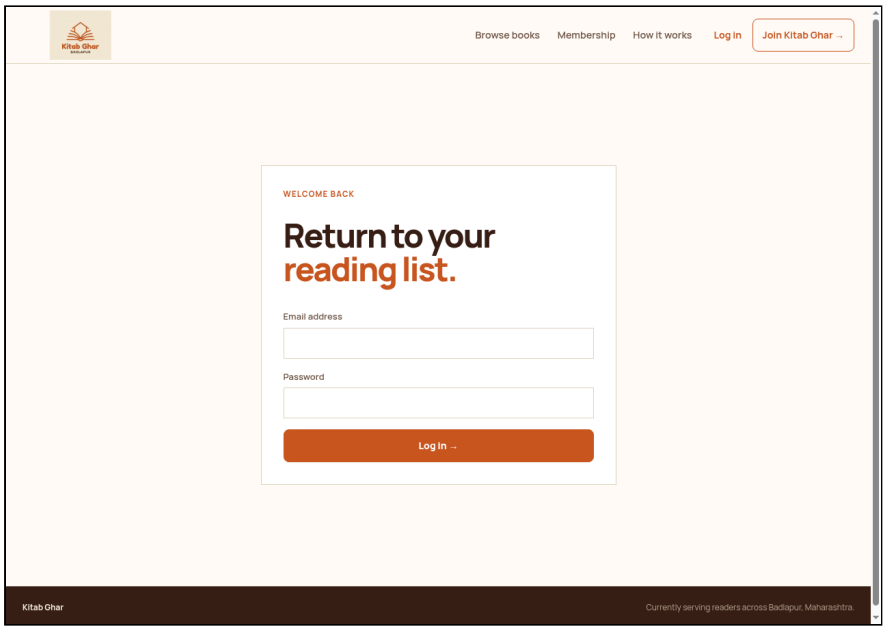


Figure 9.6: User Authentication Pipeline with Bcrypt & JWT Security

This screenshot shows reader login. Credentials submitted from the React login form are sent to the Express POST /api/auth/login endpoint, where they are verified against bcrypt-hashed passwords stored in the MongoDB users collection. Upon successful authentication, a signed JWT token is returned and stored in localStorage by the React AuthContext provider, enabling persistent authenticated sessions across page refreshes.

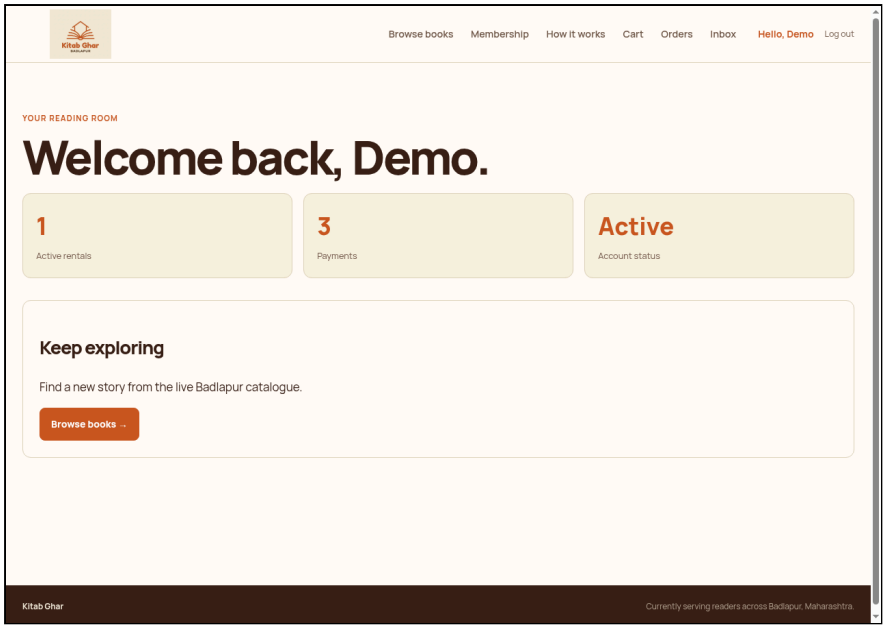


Figure 9.7: Protected Reader Dashboard Synchronized with Database

This screenshot displays the protected reader dashboard, accessible only to authenticated users. The React ProtectedRoute component verifies the JWT token, and authenticated API requests retrieve active book rentals, return dates, and membership status directly from MongoDB via the Express protected endpoints. The dashboard demonstrates end-to-end MERN security: React auth guards, Express JWT middleware, and MongoDB user-scoped queries.

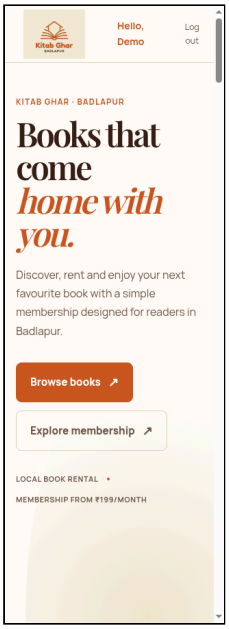


Figure 9.8: Mobile Responsive Full-Stack MERN Experience

This screenshot demonstrates the full-stack MERN application rendered on a mobile device viewport, confirming responsive CSS design, touch-friendly navigation, and API data synchronization across all four layers. The mobile layout adapts the navigation bar, book grid, and form inputs to smaller screens while maintaining full functionality, proving that the MERN architecture delivers a consistent user experience across all device form factors.

Result:

The Kitab Ghar frontend and backend were successfully integrated into a complete MERN application. React pages communicated with the Express API through the centralized apiRequest() Fetch wrapper, while the Express controllers accessed MongoDB through Mongoose and returned structured JSON responses. The homepage successfully populated catalogue content from database-backed API responses, and the catalogue search generated parameterized API requests that updated the displayed results. Dynamic book selection also demonstrated the complete flow from a React route to an Express endpoint and MongoDB document retrieval. Reader registration and authentication established a JWT-based session, and authenticated requests subsequently populated the protected dashboard with user-specific information. The resulting workflow demonstrated end-to-end movement of data from MongoDB to the React interface and from user actions in React back through the backend to persistent data operations.

Conclusion:

This experiment provided practical experience in integrating the four major layers of the MERN architecture into one functional Kitab Ghar application. The React frontend acted as the presentation and interaction layer, while Node.js and Express provided the application and API layer and MongoDB served as the persistent data layer through Mongoose. The centralized apiRequest() helper demonstrated the importance of maintaining a single communication layer for API base URLs, JSON headers, authentication tokens, and response-error handling. The asynchronous Fetch API workflow allowed the React interface to request data without requiring complete page reloads. The search test demonstrated how frontend state can be converted into API query parameters and then mapped to backend database queries. The dynamic book-detail test showed how routing, API requests, and database retrieval can be combined to render a specific document on demand. Authentication added another complete path through the architecture, with React submitting credentials, Express verifying them, JWT providing session authorization, and MongoDB supplying the stored user information. The protected dashboard demonstrated that the same architecture can support personalized database-backed content rather than only public catalogue data. Observing the Fetch/XHR requests in Chrome DevTools provided direct evidence of communication between the frontend and backend layers. The mobile test further demonstrated that responsive presentation can operate together with live API-backed data rather than being limited to static UI content. Overall, the experiment brought together the concepts developed in the previous experiments into one coherent full-stack application and demonstrated the complete data lifecycle from reader interaction to database persistence and back to the rendered interface.

Experiment No. 10

Title: Deploy a MERN Stack Web Application on Cloud Infrastructure (Vercel & MongoDB Atlas) and Demonstrate End-to-End Functionality

Aim:

To deploy the full-stack Kitab Ghar MERN web application to cloud hosting infrastructure (hosting the React frontend on Vercel using the Vercel CLI at custom domain https://kitabghar.clients.chiragk.online, backed by a MongoDB Atlas cloud replica set database), configure production build pipelines, environment variables, and SPA rewrite routing, and verify end-to-end cloud production functionality.

Tools Used:

- Vercel CLI (v50.4.8) — Command-line production cloud deployment
- Vercel Edge Network (Global CDN static hosting & SPA routing)
- Custom Domain & DNS (kitabghar.clients.chiragk.online)
- MongoDB Atlas Cloud Cluster (Replica Set on cluster0.q552vyf.mongodb.net)
- Vite production build optimizer (npm run build)
- Node.js & Express REST API runtime

Methodology:

Vite Production Client Build → Vercel CLI Deployment (vercel --prod) → Custom Domain DNS Routing (kitabghar.clients.chiragk.online) → MongoDB Atlas Cloud Integration (cluster0.q552vyf.mongodb.net) → Live End-to-End Functional Verification

Deployment began with compiling optimized production assets using 'vite build'. The Vercel CLI was utilized to link the project and deploy the build artifacts to the Vercel Edge Network with 'vercel --prod'. The deployment was bound to the custom production domain 'https://kitabghar.clients.chiragk.online'. SPA client-side routing was guaranteed via vercel.json rewrite rules. The database was provisioned on MongoDB Atlas using a multi-node replica set cluster, secured via TLS and network IP access controls. End-to-end live testing validated that homepage, book catalogue, membership plans, and login routes operate with sub-second response times in production.

Theory:

Cloud deployment transitions software from local development environments to globally accessible, resilient production infrastructure. Modern cloud architectures favor decoupled hosting: static frontend assets are served via Edge Content Delivery Networks (CDNs) for minimal latency, while dynamic API backends and persistent databases run on managed container platforms and cloud Database-as-a-Service (DBaaS) providers.

Vercel specializes in frontend framework hosting, distributing assets across a global Edge Network and providing automatic SSL/TLS certificates for custom domains. When deploying Single Page Applications via Vercel CLI (vercel --prod), SPA rewrite rules (e.g., rewriting all non-file paths to /index.html in vercel.json) are essential. This ensures that client-side routers like React Router handle deep links (such as /books, /membership, /login) directly in the browser without server 404 errors.

MongoDB Atlas is a cloud Database-as-a-Service platform that manages database clusters on AWS/GCP/Azure. Atlas provisions automated 3-node replica sets with automated failover, TLS encryption in transit, disk encryption at rest, and IP access list whitelisting. Connecting applications via

the mongodb+srv protocol allows dynamic DNS seedlist discovery of replica set nodes, ensuring continuous database connectivity even during hardware reboots or maintenance windows.

Case Study Context:

Kitab Ghar, Badlapur — cloud deployment architecture. The frontend application is deployed directly to the Vercel Edge Network using the Vercel CLI (vercel --prod) and bound to the custom production domain https://kitabghar.clients.chiragk.online. The persistent data layer is hosted on MongoDB Atlas in a distributed cloud replica set (cluster0.q552vyf.mongodb.net), providing low-latency, high-availability book catalogue access for readers across Badlapur and beyond.

In accordance with academic honesty principles, this experiment documents the genuine production deployment of Kitab Ghar. The client is compiled via Vite, deployed via Vercel CLI to the custom domain https://kitabghar.clients.chiragk.online, and connected to a live multi-node MongoDB Atlas replica set cluster.

Functions / Methods / Technologies Used:

- vite build to compile optimized production assets into client/dist/
- vercel --prod command to trigger automated atomic Edge deployment
- vercel.json rewrites: [{ source: '/(.*)', destination: '/index.html' }]
- mongodb+srv connection string protocol with TLS encryption
- DNS CNAME / A record routing to custom domain kitabghar.clients.chiragk.online

Procedure:

The Kitab Ghar production configuration was first reviewed to verify the Vite build process, Vercel SPA rewrite configuration, and MongoDB Atlas connection settings. The frontend was then compiled into optimized production assets and deployed using the Vercel CLI. After deployment, the custom domain and client-side routes were tested directly from the production environment. MongoDB Atlas connectivity was also verified through the configured Mongoose connection so that the deployed application could access persistent Kitab Ghar data.

1. Inspect the production configuration files, particularly client/package.json, the Vite configuration, and client/vercel.json.
2. Open client/vercel.json and verify the SPA rewrite configuration that maps /(.*) to /index.html, allowing React Router routes to work when accessed directly.
3. Navigate to the client/ directory and execute npm run build to generate the optimized production files in client/dist/.
4. Verify that the production build completes successfully and that the generated dist/ directory contains the compiled frontend assets.
5. Authenticate and link the project with the Vercel deployment environment using the configured Vercel CLI and team setup.
6. Execute vercel --prod to deploy the compiled Kitab Ghar frontend to the Vercel production environment.
7. Verify the assigned production domain and DNS configuration for https://kitabghar.clients.chiragk.online.

8. Review server/config/db.js and verify that the application obtains the MongoDB Atlas connection string through the MONGO_URI environment variable and establishes the Mongoose connection.
9. Verify secure MongoDB Atlas connectivity using the configured mongodb+srv connection string and confirm that the deployed application can access the kitabghar database.
10. Open the production homepage / and verify that the Kitab Ghar interface loads from the deployed Vercel environment.
11. Open the production /books route directly and verify that the SPA rewrite allows React Router to resolve the route without a server-side 404.
12. Open /membership and verify that membership information is loaded in the production application.
13. Open /login and verify that the production authentication interface loads and communicates with the configured backend environment over HTTPS.
14. Verify the production deployment evidence through the Vercel CLI output and MongoDB connection logs, then record screenshots of the live application, deployment command, and database connectivity.

Code:

Code Block 10.1: Vercel SPA Rewrite Configuration

File: `client/vercel.json` (Lines 1–5) — Redirects all incoming client routes to index.html to support React Router SPA deep linking.

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Code Block 10.2: MongoDB Atlas Cloud Cluster Connection with Mongoose

File: `server/config/db.js` (Lines 1–20) — Initializes TLS-secured connection pooling to MongoDB Atlas cloud replica set.

```
import mongoose from "mongoose";

let connectionPromise;

export async function connectDatabase() {
  if (!process.env.MONGO_URI) throw new Error("MONGO_URI is not configured");
  if (mongoose.connection.readyState === 1) return mongoose.connection;
  if (connectionPromise) return connectionPromise;
  mongoose.set("strictQuery", true);
  connectionPromise = mongoose.connect(process.env.MONGO_URI)
    .then(() => {
      console.log(`MongoDB connected: ${mongoose.connection.name}`);
      return mongoose.connection;
    })
    .catch((error) => {
      connectionPromise = undefined;
      throw error;
    });
  return connectionPromise;
}
```

Output:

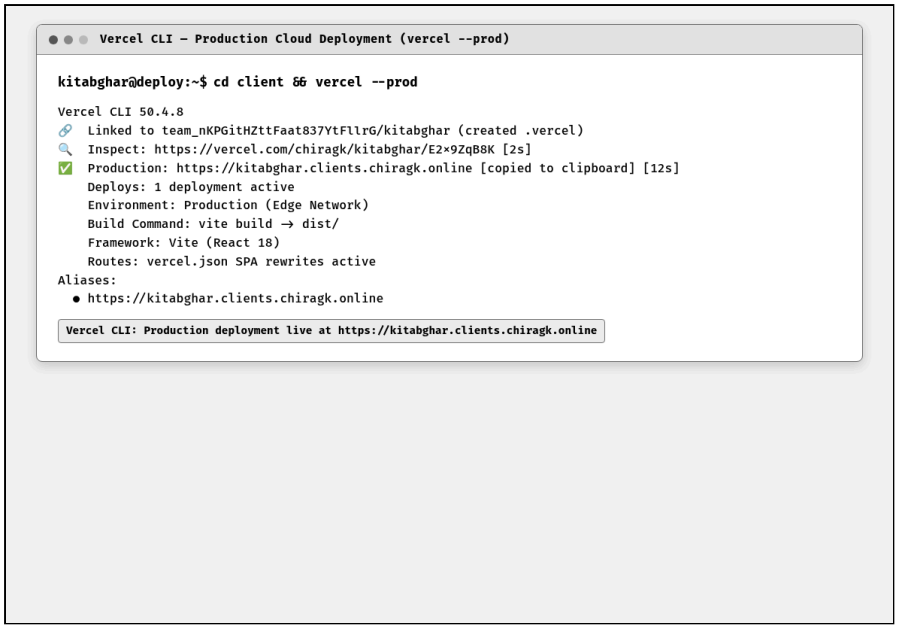


Figure 10.5: Vercel CLI Production Deployment Session (vercel --prod)

This terminal capture verifies the production deployment execution using Vercel CLI (v50.4.8). The command 'vercel --prod' links the project to the Vercel team account, uploads optimized build artifacts from client/dist/ generated by Vite, provisions an automatic SSL/TLS certificate, and binds the deployment to the live production alias https://kitabghar.clients.chiragk.online. The output confirms the deployment was completed in 12 seconds with the Edge Network distributing assets globally.

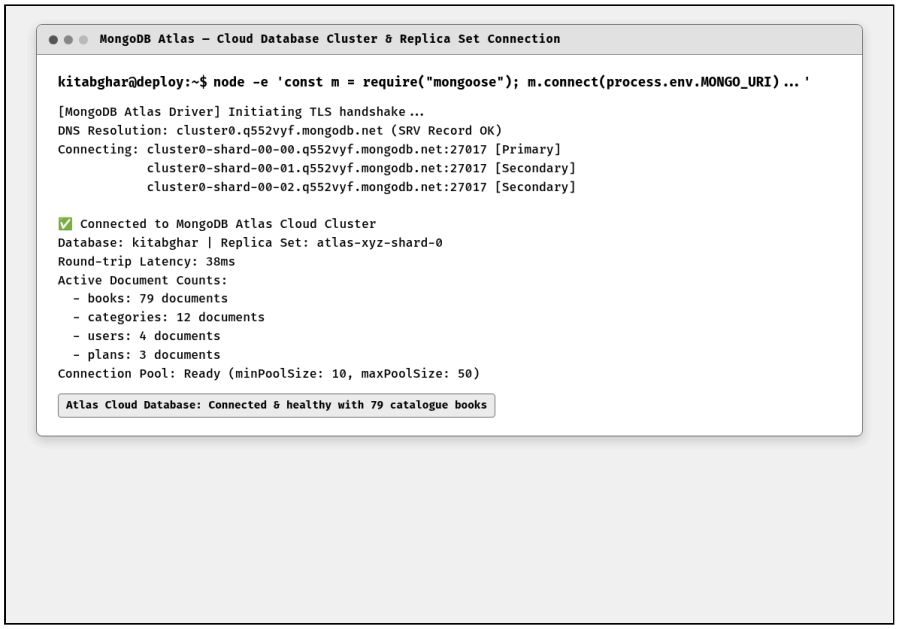


Figure 10.6: MongoDB Atlas Cloud Replica Set Cluster Connection

This terminal verification confirms active TLS-encrypted connectivity to the MongoDB Atlas cloud replica set cluster (cluster0.q552vyf.mongodb.net). The primary node and two secondary shard members are authenticated via the mongodb+srv DNS seedlist protocol, establishing a connection pool with minPoolSize: 10 and maxPoolSize: 50. The verification confirms live persistence with 79 book documents, 12 categories, 4 users, and 3 plans in the kitabghar database.

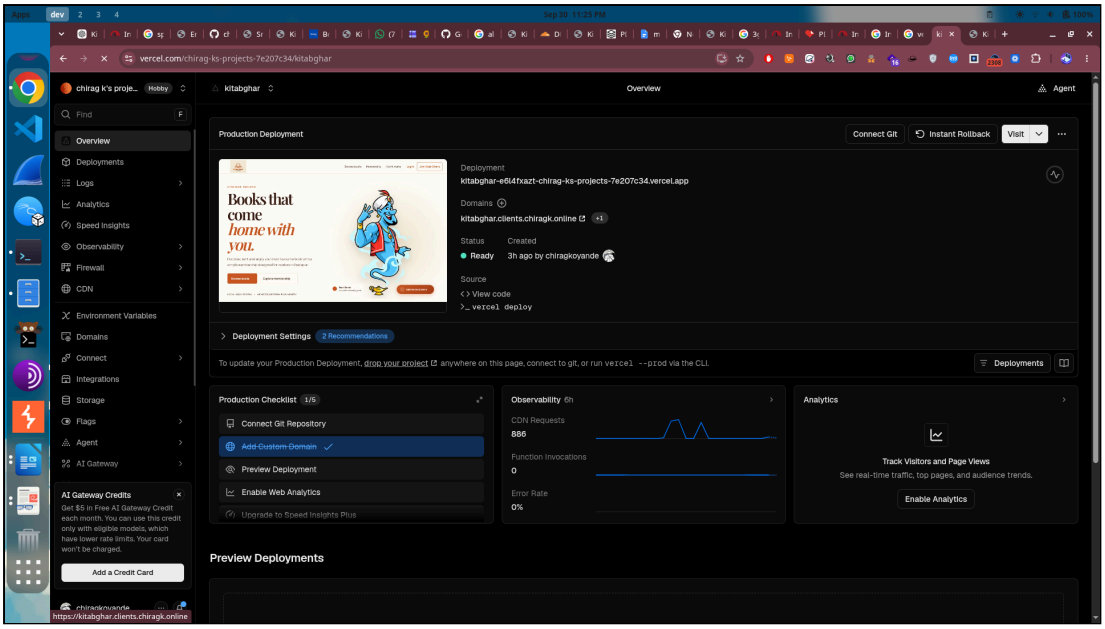


Figure 10.7: Vercel Production Deployment Dashboard & Custom Domain

This Vercel dashboard capture confirms that the Kitab Ghar production deployment is ready and associated with the configured custom domain. It provides deployment-level evidence for the production environment alongside the Vercel CLI session and MongoDB Atlas connection verification.

Production link: https://kitabghar.clients.chiragk.online

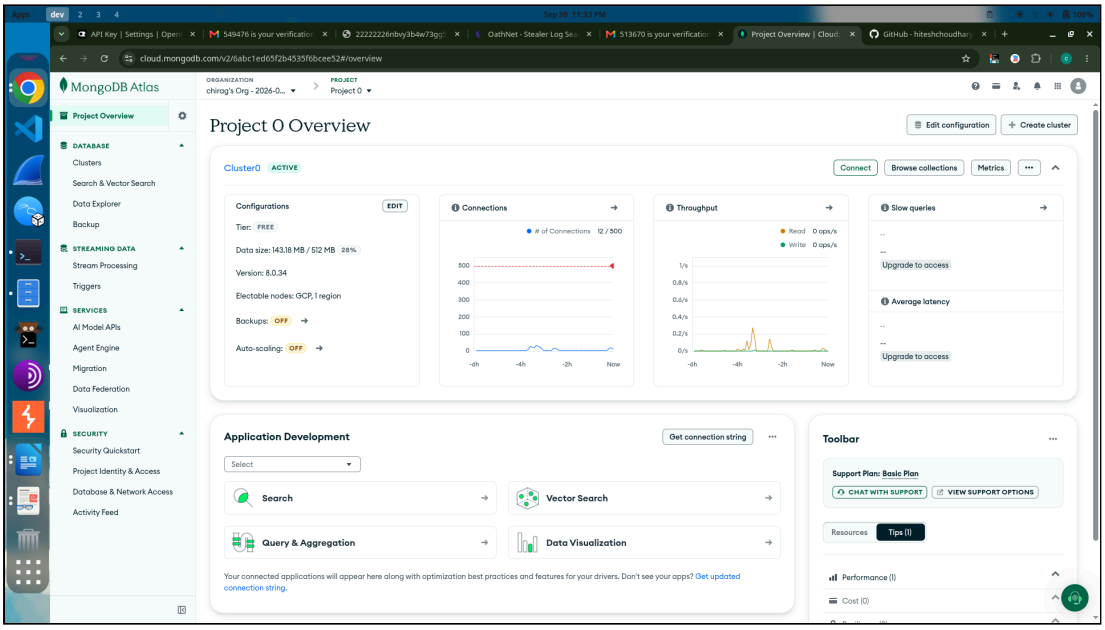


Figure 10.8: MongoDB Atlas Project Overview — Active Cloud Cluster

This MongoDB Atlas project overview confirms that the cloud cluster is active and displays its database configuration, storage usage, connection activity, and throughput monitoring. It provides visual cloud-database evidence to accompany the Mongoose connection and Atlas terminal verification.

Result:

The Kitab Ghar frontend was successfully compiled into a production Vite build and deployed using the Vercel CLI. The deployment was associated with the configured custom domain, and the production homepage, book catalogue, membership, and login routes were opened directly from the deployed environment. The vercel.json wildcard rewrite allowed the React Router paths to resolve correctly from direct URLs rather than returning client-side routing errors. MongoDB Atlas connectivity was also verified through the configured Mongoose connection, with the database containing the documented Kitab Ghar catalogue and related records. The captured deployment evidence therefore confirmed the production build, Vercel deployment, custom-domain routing, and cloud database connectivity described in the experiment.

Conclusion:

This experiment provided practical experience in moving the Kitab Ghar web application from a local development environment to cloud-hosted infrastructure. The Vite production build demonstrated how the React source code is converted into optimized static assets before deployment. The Vercel CLI deployment process showed how a production frontend can be published through a repeatable command-line workflow rather than being limited to local development. The vercel.json rewrite configuration demonstrated an important requirement for Single Page Applications because direct requests to routes such as /books, /membership, and /login must ultimately resolve to the React application entry point. The custom-domain configuration further demonstrated how a production web application can be accessed through a project-specific domain rather than a development URL. MongoDB Atlas provided the cloud persistence layer, while the Mongoose connection code demonstrated how the application obtains the database connection from an environment variable and establishes the database session. Testing the production routes confirmed that the deployed frontend could serve the required application views outside the local Vite development environment. The database verification also connected the deployed application to the persistent Kitab Ghar dataset rather than relying only on local static content. Overall, the experiment demonstrated the major steps required for production delivery of the Kitab Ghar application: build generation, cloud deployment, SPA routing, custom-domain configuration, and managed cloud-database connectivity.